

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2013

Huấn luyện viên: Hu Weidong, Tang Wenbin

Biên tập: Hu Weidong

China Computer Federation, 2013

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013

IOI China candidate team papers / National training team 2013 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2013. Bản dịch này chuyển ngữ thông tin đầu sách, mục lục và mười một bài đầu tiên: *Kế hoạch lên đỉnh*, của Peng Tianyi, *Lấy số trên lưới*, của Wang Kangning, *Thảo luận bài Two strings*, của Luo Gan, *Catch The Penguins*, của Zhang Wentao, *Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu*, của Luo Jianqiao, *Kỹ thuật meet in the middle trong bài toán tìm kiếm*, của Qiao Mingda, và *Phân tích cơ sở và ứng dụng của xác suất trong thi tin học*, của Hu Yuanming, và *Bàn về một số cách giải không kinh điển cho bài cấu trúc dữ liệu*, của Xu Haoran, *Ứng dụng cây cân bằng theo trọng lượng và cây cân bằng hậu tố trong Olympic Tin học*, của Chen Lijie, *Bàn về bài toán đếm trên vòng*, của Gao Shenghan, và *Ứng dụng của phương pháp chia khối*, của Wang Ziyu. Trạng thái hiện tại: mười một bài đầu đã được dịch; các bài còn lại mới có mục lục. Lớp văn bản của phần thân bị lỗi ánh xạ phông chữ, nên phần bài viết được đối chiếu từ OCR trang render và các công thức, ví dụ còn đọc được trong lớp văn bản.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013*. Huấn luyện viên là Hu Weidong và Tang Wenbin; biên tập là Hu Weidong. Đơn vị xuất bản ghi trên bìa là China Computer Federation.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Kế hoạch lên đỉnh	Peng Tianyi, Trường THPT trực thuộc Đại học Sư phạm Hồ Nam
9	Lấy số trên lưới	Wang Kangning, Trường THPT trực thuộc Đại học Sư phạm Đông Bắc, Cát Lâm
17	Thảo luận bài <i>Two strings</i>	Luo Gan, Trường Học Quân Hàng Châu, Chiết Giang
23	<i>Catch The Penguins</i>	Zhang Wentao, Trường Học Quân Hàng Châu, Chiết Giang
27	Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu	Luo Jianqiao, Trường THPT số 80 Bắc Kinh
43	Kỹ thuật <i>meet in the middle</i> trong bài toán tìm kiếm	Qiao Mingda, Trường Ngoại ngữ Nam Kinh, Giang Tô
57	Phân tích cơ sở và ứng dụng của xác suất trong thi tin học	Hu Yuanming, Trường Trung học Dương Châu, Giang Tô
71	Bàn về một số cách giải không kinh điển cho bài cấu trúc dữ liệu	Xu Haoran, Trường Ngoại ngữ Nam Kinh, Giang Tô
85	Ứng dụng cây cân bằng theo trọng lượng và cây cân bằng hậu tố trong Olympic Tin học	Chen Lijie, Trường Ngoại ngữ Hàng Châu, Chiết Giang
99	Bàn về bài toán đếm trên vòng	Gao Shenghan, Trường Trung học cao cấp Thường Châu, Giang Tô
109	Ứng dụng của phương pháp chia khối	Wang Ziyu, Trường Trung học số 1 Shengli, Sơn Đông
121	Bàn về nguyên lý bao hàm–loại trừ	Wang Di, Trường Trung học số 7 Thành Đô, Tứ Xuyên

3 Trạng thái và ghi chú trích xuất

Các trang in trong mục lục lệch 4 trang so với trang vật lý của PDF: trang vật lý 5 là trang in 1. Bài *Kế hoạch lên đỉnh* dùng các trang in 1–8, tức trang vật lý 5–12. Bài *Lấy số trên lưới* dùng các trang in 9–16, tức trang vật lý 13–20. Bài *Thảo luận bài Two strings* bắt đầu ở trang in 17, tức trang vật lý 21; bài kế tiếp bắt đầu ở trang in 23, tức trang vật lý 27. Trang in 22 là trang trắng, nên phần dịch bài thứ ba lấy nội dung ở các trang in 17–21. Bài *Catch The Penguins* dùng các trang in 23–26, tức trang vật lý 27–30. Bài *Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu* bắt đầu ở trang in 27, tức trang vật lý 31; bài kế tiếp bắt đầu ở trang in 43, tức trang vật lý 47. Trang in 42 là trang trắng, nên phần dịch bài thứ năm lấy nội dung ở các trang in 27–41. Bài *Kỹ thuật meet in the middle trong bài toán tìm kiếm* bắt đầu ở trang in 43, tức trang vật lý 47; bài kế tiếp bắt đầu ở trang in 57, tức trang vật lý 61. Phần dịch bài thứ sáu lấy nội dung ở các trang in 43–56. Bài *Phân tích cơ sở và ứng dụng của xác suất trong thi tin học* bắt đầu ở trang in 57, tức trang vật lý 61; bài kế tiếp bắt đầu ở trang in 71, tức trang vật lý 75. Phần dịch bài thứ bảy lấy nội dung ở các trang in 57–70. Bài *Bàn về một số cách giải không kinh điển cho bài cấu trúc dữ liệu* bắt đầu ở trang in 71, tức trang vật lý 75; bài kế tiếp bắt đầu ở trang in 85, tức trang vật lý 89. Phần dịch bài thứ tám lấy nội dung ở các trang in 71–84. Bài *Ứng dụng cây cân bằng theo trọng lượng và cây*

cân bằng hậu tố trong Olympic Tin học bắt đầu ở trang in 85, tức trang vật lý 89; bài kế tiếp bắt đầu ở trang in 99, tức trang vật lý 103. Trang in 98 là trang trắng, nên phần dịch bài thứ chín lấy nội dung ở các trang in 85–97. Bài *Bàn về bài toán đếm trên vòng* bắt đầu ở trang in 99, tức trang vật lý 103; bài kế tiếp bắt đầu ở trang in 109, tức trang vật lý 113. Phần dịch bài thứ mười lấy nội dung ở các trang in 99–108. Bài *Ứng dụng của phương pháp chia khối* bắt đầu ở trang in 109, tức trang vật lý 113; bài kế tiếp bắt đầu ở trang in 121, tức trang vật lý 125. Phần dịch bài thứ mười một lấy nội dung ở các trang in 109–120.

4 Kế hoạch lên đỉnh

Peng Tianyi

Trường THPT trực thuộc Đại học Sư phạm Hồ Nam

4.1 Bối cảnh bài toán

Leo núi là một môn vận động rất có ích cho sức khỏe thể chất và tinh thần. Nhưng khi leo núi, ta nên chọn chiến lược như thế nào để lên tới đỉnh núi nhanh nhất? Từ câu hỏi thú vị này, tác giả nghĩ ra bài toán dưới đây.

4.2 Mô tả bài toán

Giới hạn tài nguyên. Giới hạn thời gian: 2 giây. Giới hạn bộ nhớ: 512 MB.

Phát biểu. Trên mặt phẳng hai chiều, một dãy núi được xác định bởi một dãy đỉnh

$$(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n).$$

Hai đỉnh kề nhau được nối bằng đoạn thẳng, với bảo đảm x_i tăng nghiêm ngặt. Như vậy ta thu được một dãy núi liên tục; giá trị y của mỗi điểm là độ cao của điểm đó.

Ta định nghĩa phần trong của dãy núi là phần kẹp giữa đường gấp khúc nối các đỉnh và trục x , không tính chính đường gấp khúc đó. Nếu đoạn thẳng nối đỉnh A và đỉnh B không đi qua phần trong của dãy núi, ta nói đỉnh A nhìn thấy đỉnh B , hoặc đỉnh B nhìn thấy đỉnh A .

Bây giờ pty xuất phát từ một đỉnh nào đó và muốn lên tới đỉnh cao nhất của dãy núi. Anh chỉ có thể đi trên đường gấp khúc giữa các đỉnh. Sau khi suy nghĩ, anh chọn cách leo như sau:

1. Đứng tại điểm xuất phát và nhìn sang hai bên. Nếu đỉnh núi cao nhất nhìn thấy được lúc này nằm bên trái, anh đi sang trái; ngược lại anh đi sang phải.
2. Trong lúc đi, pty vẫn quan sát đỉnh cao nhất nhìn thấy được ở hai bên. Một khi phát hiện một đỉnh cao hơn tất cả những đỉnh đã thấy trước đó, anh sẽ đi về phía đỉnh cao nhất lúc này.
3. Nếu có một thời điểm mà vị trí pty đang đứng cao hơn mọi đỉnh cao nhất nhìn thấy được ở hai bên, thì anh đã tới đỉnh cao nhất của dãy núi và quá trình leo núi kết thúc.

pty muốn biết: nếu dùng chiến lược trên và xuất phát từ từng đỉnh, quãng đường để tới đỉnh cao nhất lần lượt là bao nhiêu? Khoảng cách giữa hai điểm trên mặt phẳng là độ dài đoạn thẳng nối chúng.

Dữ liệu vào. Dòng đầu tiên chứa một số nguyên n , là số đỉnh của dãy núi.

Tiếp theo là n dòng; dòng thứ i chứa hai số nguyên x_i, y_i , biểu diễn tọa độ của đỉnh thứ i . Bảo đảm x_i tăng nghiêm ngặt, các y_i đôi một khác nhau trừ y_1, y_n , và x_i, y_i đều là số nguyên không âm. Bảo đảm $y_1 = y_n = 0$.

Dữ liệu ra. In ra n dòng, mỗi dòng một số thực.

Số thực ở dòng thứ i biểu diễn quãng đường từ đỉnh thứ i tới đỉnh cao nhất. Nếu sai số giữa kết quả in ra và đáp án chuẩn không vượt quá 10^{-2} , test đó được điểm tối đa; ngược lại được 0 điểm.

Ví dụ vào.

```
6
0 0
1 6
2 4
3 1
5 5
6 0
```

Ví dụ ra.

```
6.08
0.00
2.24
6.52
9.87
14.97
```

Giải thích ví dụ. Các lộ trình là:

$A : A \rightarrow B,$
 $B : B,$
 $C : C \rightarrow B,$
 $D : D \rightarrow G \rightarrow D \rightarrow C \rightarrow B,$
 $E : E \rightarrow D \rightarrow C \rightarrow B,$
 $F : F \rightarrow E \rightarrow D \rightarrow C \rightarrow B.$

Khi xuất phát từ D , đỉnh cao nhất nhìn thấy được ban đầu là E . Khi đi tới điểm G , ptý phát hiện đỉnh cao hơn là B , nên quay đầu và tiếp tục đi về phía B . Với các điểm xuất phát còn lại, lộ trình không cần đổi hướng.

Quy mô dữ liệu và quy ước. Mọi dữ liệu đều thỏa mãn $x_i, y_i \leq 100000$.

Test	Ràng buộc
1–4	$n \leq 20$
5–8	$n \leq 70$
9–10	$n \leq 100000$, và mọi đỉnh đều nhìn thấy trực tiếp đỉnh cao nhất
11–14	$n \leq 30000$
15–20	$n \leq 100000$

4.3 Phân tích bài toán

4.3.1 Đơn giản hóa bài toán

Bài toán yêu cầu tại mọi thời điểm khi leo núi ta đều phải quan sát đỉnh cao nhất hiện tại ở hai bên. Trước hết hãy tạm đơn giản hóa vấn đề: chỉ khi tới một đỉnh ta mới quan sát đỉnh cao nhất lúc đó. Nói cách khác, điểm quan sát tạm thời chuyển từ vô hạn nhiều điểm thành hữu hạn $O(n)$ đỉnh.

4.3.2 Tìm đỉnh cao nhất mà một đỉnh nhìn thấy

Gọi đỉnh cao nhất mà đỉnh i nhìn thấy được về bên trái là $L[i]$, và đỉnh cao nhất mà nó nhìn thấy được về bên phải là $R[i]$. Hãy tưởng tượng ta đứng tại một điểm nào đó; ban đầu tia nhìn nằm ngang về bên trái, sau đó ta từ từ ngẩng đầu lên cho tới khi tia nhìn không còn bị một ngọn núi nào che khuất. Khi đó ta tìm được $L[i]$.

Giả sử đỉnh hiện tại là A , còn $L[i]$ là B . Hai tính chất sau là hiển nhiên:

- i điểm đầu đều nằm phía dưới đoạn AB , theo nghĩa không nghiêm ngặt;
- A và B là hai điểm kề nhau cuối cùng trên bao lồi trên của i điểm đầu.

Vì A chắc chắn là điểm cuối cùng của bao lồi trên của i điểm đầu, B chính là điểm áp chót trên bao lồi trên đó.¹ Do đó ta chỉ cần dùng một stack để duy trì bao lồi của các điểm đầu tiên. Mỗi lần thêm điểm thứ $i + 1$, ta đồng thời duy trì tính chất của bao lồi. Vì tổng cộng chỉ có $O(n)$ đỉnh, ta có thể tìm toàn bộ $L[i]$ và $R[i]$ trong thời gian $O(n)$.

4.3.3 Xây dựng mô hình đồ thị

Cách làm vét cạn. Sau khi có $L[i]$ và $R[i]$, ta có một thuật toán khá thô: với từng đỉnh xuất phát, mô phỏng sự thay đổi của lộ trình. Độ phức tạp là $O(n^2)$ hoặc cao hơn. Điều này cho thấy ta cần tiếp tục khai thác phần thông tin bị tính lặp.

Dựng cây. Gọi $H[i]$ là độ cao của đỉnh thứ i , và đặt

$$T[i] = \max(H[L[i]], H[R[i]]).$$

Khi đó $T[i]$ biểu diễn độ cao của đỉnh cao nhất mà điểm i nhìn thấy được.

Khi ta xuất phát từ A , đến điểm đầu tiên B thỏa mãn

$$T[B] \geq T[A],$$

thì đoạn đường còn lại cần đi sẽ hoàn toàn giống như khi xuất phát từ B . Vì vậy khoảng cách từ A tới đỉnh cao nhất bằng khoảng cách từ B tới đỉnh cao nhất cộng thêm khoảng cách từ A tới B .

Lúc này cho B nối tới A bằng một cạnh có hướng, trọng số là độ dài AB . Làm như vậy với mọi đỉnh, ta thu được một cây: đỉnh cao nhất không có bậc vào, còn mọi đỉnh khác có bậc vào bằng 1. Tổng trọng số trên đường đi từ gốc cây tới một đỉnh bất kỳ chính là quãng đường từ đỉnh đó tới đỉnh cao nhất. Chỉ cần DFS một lần là có đáp án cho mọi đỉnh.

¹Một kết luận đẹp như vậy thật sự làm tác giả ngạc nhiên.

Vấn đề cần giải quyết. Với một điểm A , làm thế nào nhanh chóng tìm được điểm đầu tiên về bên trái hoặc bên phải thỏa mãn $T[B] \geq T[A]$? Đây là một bài toán cấu trúc dữ liệu rất kinh điển. Ta có thể giải bằng nhị phân đáp án kết hợp RMQ trên đoạn. Dưới đây là một cách có hằng số nhỏ hơn.

Lập danh sách liên kết hai chiều của toàn bộ đỉnh theo thứ tự từ trái sang phải. Sau đó duyệt tất cả các điểm theo thứ tự T tăng dần. Mỗi khi duyệt tới một điểm A , điểm đầu tiên bên trái thỏa mãn $T[B] \geq T[A]$ chính là hàng xóm bên trái của A còn tồn tại trong danh sách liên kết tại thời điểm đó. Nối cạnh tương ứng, rồi xóa A khỏi danh sách liên kết. Phía bên phải được xử lý tương tự. Độ phức tạp của thuật toán này chính là độ phức tạp sắp xếp.

4.3.4 Quay lại bài toán ban đầu

Tới đây ta đã giải xong phiên bản đơn giản hóa trong thời gian sắp xếp $O(n \log n)$. Bây giờ hãy quay lại bài toán ban đầu.

Ta có khả năng quan sát đỉnh cao nhất tại mọi thời điểm. Điều này làm bài toán thay đổi như thế nào?

Giả sử hiện tại ta xuất phát từ điểm A , đi sang phải, tới một điểm B trên một đoạn nào đó thì phát hiện một đỉnh cao hơn ở bên trái, nên quay sang đi về bên trái. Điểm rẽ B phải có những tính chất gì? Gọi đoạn chứa B là P_1P_2 . Khi đó B thỏa mãn:

- B là giao điểm giữa P_1P_2 và đường thẳng nối hai điểm nào đó bên trái A , ký hiệu là T_1, T_2 ;
- các điểm trước A đều nằm phía dưới đường thẳng T_1T_2 , theo nghĩa không nghiêm ngặt;
- T_1, T_2 là hai điểm kề nhau trên bao lồi trên của các điểm trước A .

Điều này có nghĩa: một điểm quan sát có ý nghĩa B sẽ là giao điểm đầu tiên giữa đường kéo dài của một cạnh nào đó trên bao lồi trên và đường gấp khúc của dãy núi. Vì tổng số cạnh được sinh ra trong quá trình duy trì bao lồi trên là $O(n)$, số điểm quan sát có ý nghĩa cũng là $O(n)$.²

Vậy làm thế nào nhanh chóng tìm ra các điểm quan sát đó?

Nếu đường thẳng T_1T_2 cắt đoạn P_1P_2 , điều này có nghĩa là khi P_2 được thêm vào bao lồi, T_2 sẽ bị bật ra; còn khi P_1 được thêm vào bao lồi thì T_2 chưa bị bật ra. Vì thế trong lúc duy trì bao lồi, ta chỉ cần lấy giao điểm giữa các đường thẳng liên quan tới điểm bị bật ra và điểm vừa chèn, là có thể thu được toàn bộ điểm quan sát. Độ phức tạp vẫn là $O(n)$.

Bài toán với hữu hạn điểm quan sát đã được giải ở trên. Vì thế toàn bộ bài toán được giải trọn vẹn trong thời gian $O(n \log n)$.

4.4 Tổng kết

4.4.1 Nguồn cảm hứng

Cảm hứng của bài toán đến từ suy nghĩ về việc leo núi trong đời sống thực. Dùng kiến thức thuật toán để phân tích một vấn đề thực tế không chỉ nâng cao năng lực phân tích thuật toán, mà còn phản ánh mục đích cốt lõi của việc học là “học để dùng”. Nó cũng có thể khơi dậy hứng thú đối với thi Tin học và làm ta thấy vui trong quá trình đó. Hy vọng cách suy nghĩ này đem lại gợi ý cho mọi người.

4.4.2 Tư duy giải bài

Nhìn lại quá trình giải bài: trước hết ta đơn giản hóa bài toán, biến vô hạn thành hữu hạn; tiếp đó chia bài toán thành các phần nhỏ hơn và lần lượt giải quyết. Đây đều là những phương pháp thường dùng trong

²Đây cũng là một tính chất gây phấn khích.

quá trình làm bài.

Dưới đây là vài cảm nhận của tác giả về phương pháp giải bài, xin chia sẻ với mọi người.

Trong quá trình giải bài thông thường, ta cần dựng một chiếc cầu giữa điều kiện đã biết và vấn đề cần tìm. Có thể dùng công cụ đã học để suy luận xuôi từ điều kiện đã biết, xem ta còn thu được gì. Cũng có thể suy luận ngược từ vấn đề cần tìm, xem ta còn cần biết gì. Khi cây cầu suy luận xuôi và cây cầu suy luận ngược có một điểm trùng nhau, cây cầu nối từ điều kiện tới bài toán đã được dựng xong, và bài toán cũng được giải quyết.

Tuy nhiên phương pháp giải bài biến hóa muôn hình vạn trạng; không tồn tại bí kíp vạn năng nào. Điều đáng mừng là: khi số bài bạn đã giải càng nhiều, khả năng bạn giải được bài tiếp theo càng lớn. Vì vậy, chỉ cần chăm luyện tập và chịu suy nghĩ, năng lực giải bài chắc chắn sẽ không ngừng bước lên những nấc thang mới.

5 Lấy số trên lưới

Wang Kangning

Trường THPT trực thuộc Đại học Sư phạm Đông Bắc, Cát Lâm

Tóm tắt

Đây là báo cáo lời giải cho một bài toán thú vị trong đợt làm đề đối kháng của đội tuyển quốc gia năm 2013.

5.1 Đề bài

5.1.1 Mô tả bài toán

Cho một bảng ô vuông $N \times N$. Mỗi ô chứa một số; số ở hàng i , cột j bằng $A_i \cdot B_j$.

Một người đi từ góc trên bên trái của một hình chữ nhật con tới góc dưới bên phải của hình chữ nhật đó. Mỗi bước chỉ được đi sang phải hoặc đi xuống một ô. Anh ta cộng các số đi qua trên đường đi và muốn tổng này nhỏ nhất.

Anh ta quyết định đi T lần. Lần thứ i , anh đi từ ô hàng P_i , cột Q_i tới ô hàng R_i , cột S_i . Hãy hỏi tổng nhỏ nhất của các số đi qua trong mỗi lần.

5.1.2 Dữ liệu vào

Dòng đầu tiên gồm hai số nguyên N, T , lần lượt là kích thước bảng và số lượng truy vấn.

Dòng thứ hai gồm N số nguyên $A_1, A_2, \dots, A_N$. Dòng này được sinh ngẫu nhiên.

Dòng thứ ba gồm N số nguyên $B_1, B_2, \dots, B_N$. Dòng này cũng được sinh ngẫu nhiên.

Tiếp theo là T dòng, mỗi dòng gồm bốn số nguyên P_i, Q_i, R_i, S_i , có ý nghĩa như trên và thỏa mãn $P_i \leq R_i, Q_i \leq S_i$.

5.1.3 Dữ liệu ra

In ra T dòng; mỗi dòng là một số nguyên, biểu diễn đáp án của truy vấn tương ứng.

5.1.4 Ví dụ vào

```
3 4
1 2 3
2 3 4
1 1 1 1
1 3 1 3
1 1 3 3
1 1 3 1
```

5.1.5 Ví dụ ra

```
2
4
29
12
```

5.1.6 Phạm vi dữ liệu

- Với 20% số test: $N \leq 100$, $T \leq 100$.
- Với 50% số test: $N \leq 3000$, $T \leq 1000$.
- Với 70% số test: $N \leq 50000$, $T \leq 20000$.
- Với 100% số test: $N \leq 200000$, $T \leq 50000$, $1 \leq A_i, B_i \leq 10^6$.

Bảo đảm mọi truy vấn đều hợp lệ. Ngoài ví dụ, tất cả các giá trị A_i, B_i đều được sinh ngẫu nhiên.

Giới hạn tài nguyên. Giới hạn thời gian là 2 giây, giới hạn bộ nhớ là 512 MB.

5.2 Hướng giải

5.2.1 Thuật toán một

Với mỗi truy vấn, dùng quy hoạch động trực tiếp trên toàn bộ hình chữ nhật của truy vấn.

Độ phức tạp thời gian là $O(T \cdot N^2)$, độ phức tạp bộ nhớ là $O(N^2)$. Cách này chỉ kỳ vọng đạt khoảng 20 điểm.

5.2.2 Thuật toán hai

Thuật toán trên trả lời truy vấn quá chậm, nên ta cần tăng tốc phần truy vấn.

Xét tư tưởng chia để trị. Mỗi lần ta dùng đường giữa để chia hình chữ nhật thành hai nửa, dùng quy hoạch động tính đáp án từ từng điểm trên đường giữa tới từng điểm trong hình chữ nhật hiện tại, rồi đệ quy xử lý hai hình chữ nhật ở hai bên đường giữa. Nếu điểm đầu và điểm cuối của một truy vấn nằm ở hai phía của đường giữa, ta liệt kê điểm trên đường giữa mà đường đi đi qua; nếu không, truy vấn được đẩy xuống một hình chữ nhật con.

Nếu lần nào cũng cắt theo đường thẳng đứng, đặt độ phức tạp tiền xử lý là $\text{Time}(x)$, ta có

$$\text{Time}(x) = 2 \text{Time}(x/2) + O(N^2x).$$

Khi đó độ phức tạp tiền xử lý là $O(N^3 \log N)$. Nếu luân phiên cắt theo chiều ngang và chiều dọc, ta có

$$\text{Time}(x) = 2 \text{Time}(x/2) + O(x^3),$$

nên độ phức tạp tiền xử lý là $O(N^3)$.

Tổng độ phức tạp thời gian là $O(N^3 + TN)$, độ phức tạp bộ nhớ là $O(N^3)$. Chi phí tiền xử lý vẫn quá cao, vì vậy cần tìm phương án khác.

5.2.3 Thuật toán ba

Xét thuật toán chia khối. Nếu chia hình vuông thành các khối $S \times S$, ta tiền xử lý đáp án từ mỗi điểm tới mỗi điểm trên biên khối, đồng thời tiền xử lý đáp án giữa mọi cặp điểm trong cùng một khối.

Khi trả lời truy vấn, nếu hai điểm không nằm trong cùng một khối, ta liệt kê vị trí đầu tiên mà đường đi đi qua biên khối; nếu chúng nằm trong cùng một khối thì trả lời trực tiếp.

Độ phức tạp thời gian là

$$O\left(N^2 \left(S + \frac{N}{S^2}\right) + T \cdot \frac{N}{S}\right),$$

và độ phức tạp bộ nhớ là

$$O\left(N^2 \left(S + \frac{N}{S^2}\right)\right).$$

Độ phức tạp này cũng còn quá cao.

5.2.4 Cải tiến

Một nguyên nhân chính khiến các thuật toán trên kém hiệu quả là chúng chưa dùng tới điều kiện quan trọng trong đề: hai dãy A và B đều được sinh ngẫu nhiên.

Qua thử nghiệm và quan sát, ta thấy số điểm rẽ của đường đi tối ưu không nhiều. Hãy xét một điểm rẽ và hai đoạn đường đi kề với nó. Nếu điểm rẽ nằm ở một cột có trọng số không nhỏ hơn một cột nào đó bên trái trong vùng đang xét, đồng thời cũng không nhỏ hơn một cột nào đó bên phải, thì luôn tồn tại một cách biến đổi đường đi không tệ hơn và làm mất điểm rẽ đó.

Cụ thể, giả sử trong hình minh họa của bài gốc, cột thứ năm trong khung có trọng số không nhỏ hơn trọng số của cột ngoài cùng bên trái và cột ngoài cùng bên phải của khung. Ta xét hai cách biến đổi đường đi quanh điểm rẽ:

- Nếu trọng số của hàng phía trên trong vùng nhỏ hơn trọng số của hàng phía dưới, dùng biến đổi thứ nhất sẽ cho đáp án tốt hơn.
- Nếu trọng số của hàng phía trên lớn hơn trọng số của hàng phía dưới, dùng biến đổi thứ hai sẽ cho đáp án tốt hơn.
- Nếu hai trọng số hàng bằng nhau, dùng cách biến đổi nào thì đáp án cũng không xấu đi.

Như vậy ta có thể loại bỏ điểm rẽ đang xét.

Từ lập luận trên suy ra: với một điểm rẽ bất kỳ, trọng số của hàng hoặc cột chứa nó phải nhỏ hơn tất cả trọng số của các hàng hoặc cột ở một phía trong vùng truy vấn. Nói cách khác, nếu nhìn theo một phía của đoạn, điểm rẽ phải là một kỷ lục nhỏ mới.

Với dữ liệu ngẫu nhiên, xác suất để giá trị thứ i nhỏ hơn mọi giá trị đứng trước nó là $1/i$. Vì vậy số lượng điểm rẽ kỳ vọng là

$$1 + \frac{1}{2} + \dots + \frac{1}{N} = O(\log N).$$

5.2.5 Thuật toán bốn

Ta tìm vết cạn mọi hàng và cột có thể chứa điểm rẽ. Sau đó dùng quy hoạch động để giải truy vấn, trong đó chỉ những hàng và cột có khả năng chứa điểm rẽ mới được xem là trạng thái hợp lệ.

Theo phân tích kỳ vọng ở trên, số hàng và số cột cần xét đều là $O(\log N)$; kỳ vọng của tích cũng bằng tích của các kỳ vọng, nên số trạng thái kỳ vọng là $O(\log^2 N)$. Ngoài ra, ta lưu tổng tiền tố của hai dãy A và B để tăng tốc các phép chuyển trạng thái dọc theo một đoạn thẳng.

Độ phức tạp thời gian kỳ vọng là

$$O(T \cdot (N + \log^2 N)) = O(TN).$$

Cách này kỳ vọng đạt khoảng 50 điểm.

5.2.6 Thuật toán năm

Nút thắt của thuật toán trước là việc tìm các hàng và cột có khả năng chứa điểm rẽ. Bài toán con này có thể phát biểu như sau: trong một khoảng của một dãy, tìm các số nhỏ hơn mọi số ở phía trước hoặc nhỏ hơn mọi số ở phía sau. Chỉ cần biết, với mỗi vị trí, phần tử đầu tiên ở phía sau hoặc phía trước nhỏ hơn nó là đủ. Việc này có thể thực hiện bằng hàng đợi đơn điệu.

Chẳng hạn, để tìm phần tử đầu tiên ở phía sau nhỏ hơn mỗi phần tử, ta quét dãy từ phải sang trái. Với phần tử hiện tại, loại khỏi đuôi hàng đợi đơn điệu mọi phần tử không nhỏ hơn nó. Khi đó phần tử đầu tiên ở phía sau nhỏ hơn nó chính là phần tử ở đuôi hàng đợi hiện tại; sau đó đưa phần tử hiện tại vào hàng đợi.

Tiền xử lý chỉ cần thời gian tuyến tính. Mỗi truy vấn sau đó được trả lời trong thời gian kỳ vọng $O(\log^2 N)$.

Độ phức tạp thời gian kỳ vọng là

$$O(N + T \log^2 N).$$

Cách này kỳ vọng đạt 100 điểm. Tới đây bài toán đã được giải quyết trọn vẹn.

5.3 Lời cảm ơn

Xin cảm ơn CCF đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn cô Kong Weiling và thầy Gu Fangming vì sự chỉ dẫn.

Xin cảm ơn các bạn học đã giúp đỡ tôi.

6 Thảo luận bài *Two strings*

Luo Gan

Trường Học Quân Hàng Châu, Chiết Giang

6.1 Mô tả bài toán

Cho hai xâu A và B . Có năm loại thao tác:

1. Thêm một ký tự vào đầu xâu A .
2. Thêm một ký tự vào cuối xâu A .
3. Thêm một ký tự vào đầu xâu B .

4. Thêm một ký tự vào cuối xâu B .
5. Hỏi số lần xâu B hiện tại xuất hiện trong xâu A hiện tại.

Bảo đảm mọi ký tự đều là chữ cái thường, và ban đầu hai xâu A, B đều không rỗng.

6.1.1 Dữ liệu vào

Dòng thứ nhất và dòng thứ hai lần lượt là xâu A và xâu B ban đầu. Dòng thứ ba chứa một số nguyên m , biểu diễn số thao tác. Tiếp theo là m dòng, mỗi dòng biểu diễn một thao tác. Số đầu tiên của mỗi dòng là một số trong khoảng 1–5; nếu giá trị này khác 5, sau số đó sẽ có một chữ cái thường.

6.1.2 Dữ liệu ra

Với mỗi truy vấn, in ra một số nguyên trên một dòng, biểu diễn số lần xâu B xuất hiện trong xâu A .

6.1.3 Quy mô dữ liệu

- Với 10% dữ liệu, tổng độ dài cuối cùng của hai xâu A và B không vượt quá 200, số thao tác không vượt quá 10.
- Với 30% dữ liệu, tổng độ dài cuối cùng của hai xâu A và B không vượt quá 2000, số thao tác không vượt quá 1000.
- Với 100% dữ liệu, tổng độ dài cuối cùng của hai xâu A và B không vượt quá 200000, số thao tác không vượt quá 200000.

6.2 Ví dụ

6.2.1 Dữ liệu vào

```
ababc
a
7
5
4 b
5
3 a
1 a
5
5
```

6.2.2 Dữ liệu ra

```
2
2
1
1
```

6.2.3 Giải thích ví dụ

Lần hỏi	Xâu A	Xâu B	Số lần B xuất hiện trong A
Lần 1	ababc	a	2
Lần 2	ababc	ab	2
Lần 3	aababc	aab	1
Lần 4	aababc	aab	1

6.3 Thuật toán đơn giản

6.3.1 Mô phỏng

Mô phỏng từng thao tác. Với mỗi truy vấn, trực tiếp liệt kê các vị trí mà chuỗi B có thể xuất hiện trong chuỗi A , rồi so sánh trực tiếp.

6.3.2 Mô phỏng tối ưu bằng KMP

Trên cơ sở mô phỏng, với mỗi truy vấn ta dùng thuật toán KMP để thực hiện phép khớp chuỗi.

6.4 Tối ưu mô phỏng bằng xử lý ngoại tuyến

Đọc trước toàn bộ thao tác liên quan tới chuỗi A , rồi ghép chúng thành một “chuỗi A lớn”.

Khi đó, ở mỗi thời điểm truy vấn, chuỗi A hiện tại tương đương với một đoạn nào đó của chuỗi A lớn. Bài toán trở thành hỏi trong đoạn đó có bao nhiêu chuỗi B .

Mỗi khi gặp thao tác loại 3 hoặc loại 4, ta chạy lại KMP một lần và ghi nhận những vị trí khớp với chuỗi B . Như vậy truy vấn được chuyển thành: trong các vị trí đó, có bao nhiêu vị trí nằm trong đoạn từ L tới $R - \text{Len}(B) + 1$.

Vì L giảm dần còn R tăng dần, mỗi truy vấn có thể được giải trong thời gian $O(1)$.

6.4.1 Độ phức tạp thời gian của từng thao tác

Thao tác	Độ phức tạp
Thao tác 1	$O(n) \rightarrow O(1)$
Thao tác 2	$O(n) \rightarrow O(1)$
Thao tác 3	$O(n)$
Thao tác 4	$O(n)$
Thao tác 5	$O(n) \rightarrow O(1)$

6.5 Thuật toán xử lý ngoại tuyến bằng mảng hậu tố

6.5.1 Từ trực tuyến sang ngoại tuyến

Sử dụng tư tưởng ngoại tuyến ở trên: lấy kết quả cuối cùng của hai chuỗi A, B , ghép chúng lại với nhau, đồng thời chèn một ký tự phân cách ở giữa, chẳng hạn $+$. Sau đó dùng mảng hậu tố để duy trì quan hệ giữa các hậu tố.

Ví dụ:

Xâu A lớn: ABABC
 Xâu B lớn: ABA
 Xâu cuối cùng: ABABC+ABA

Sau khi sắp xếp mảng hậu tố:

A
 ABA
 ABC+ABA
 ABABC+ABA
 BA
 BABC+ABA
 BC+ABA
 C+ABA
 +ABA

6.5.2 Tìm khoảng hậu tố khả thi

Xâu B hiện tại xuất hiện trong xâu A ở các vị trí từ i tới $i + \text{Len}(B) - 1$ khi và chỉ khi độ dài tiền tố chung dài nhất Lcp của xâu B hiện tại và hậu tố thứ i của xâu A thỏa mãn

$$Lcp \geq \text{Len}(B).$$

Ta có thể dùng phương pháp nhân đôi để tiền xử lý mảng height , rồi tìm khoảng hậu tố này trong thời gian $O(\log n)$.

6.5.3 Thống kê vị trí bắt đầu hợp lệ trong xâu A

Ta định nghĩa một vị trí hợp lệ như sau: với vị trí i trong xâu A lớn, toàn bộ đoạn

$$[i, i + \text{Len}(B) - 1]$$

đều nằm trong xâu A hiện tại.

6.5.4 Duy trì bằng cây BIT

Ta có thể nhận thấy: mỗi thao tác bất kỳ chỉ làm thay đổi tính hợp lệ của nhiều nhất hai vị trí. Vì vậy có thể dùng cây BIT để duy trì, trên một đoạn của mảng hậu tố, có bao nhiêu vị trí hợp lệ.

6.5.5 Độ phức tạp thời gian của từng thao tác

Thao tác	Độ phức tạp
Thao tác 1	$O(\log n)$
Thao tác 2	$O(\log n)$
Thao tác 3	$O(\log n)$
Thao tác 4	$O(\log n)$
Thao tác 5	$O(\log n)$

6.6 Các điểm kiểm tra

- Suy nghĩ sâu về bản chất của bài toán.
- Nắm vững nhiều loại cấu trúc dữ liệu.
- Khả năng tối ưu nhiều loại thuật toán mô phỏng.
- Khả năng vận dụng thuật toán ngoại tuyến.

7 *Catch The Penguins*

Zhang Wentao

Trường Học Quân Hàng Châu, Chiết Giang

7.1 Tóm tắt bài toán

Cho tọa độ của N con chim cánh cụt trong không gian bốn chiều.

Có Q truy vấn. Mỗi truy vấn cho một điểm và hỏi có bao nhiêu chim cánh cụt có tọa độ ở mọi chiều đều nhỏ hơn hoặc bằng tọa độ tương ứng của điểm đó.

Ví dụ vào.

```
1
0 0 0 0
2
1 1 1 0
1 1 1 -1
```

Ví dụ ra.

```
1
0
```

Quy mô dữ liệu. Có tất cả 20 bộ dữ liệu.

Dữ liệu	Ràng buộc
1–2	$N, Q \leq 5000$
3–6	$N, Q \leq 10000$
7–14	$N \leq 30000, Q \leq 10000$
15–18	$N \leq 10000, Q \leq 30000$
19–20	$N, Q \leq 30000$

7.2 Thuật toán 0

Trực tiếp vét cạn. Với mỗi truy vấn, dùng một vòng lặp for để liệt kê từng chim cánh cụt, rồi kiểm tra xem cả bốn tọa độ của nó có đều nhỏ hơn hoặc bằng tọa độ của truy vấn hay không.

Độ phức tạp thời gian là $O(n^2)$.

Kỳ vọng đạt 10 điểm.

7.3 Tối ưu thuật toán 0

Sắp xếp riêng các chim cánh cụt theo từng chiều trong bốn chiều tọa độ.

Với mỗi truy vấn, trên mỗi chiều ta tìm nhị phân trong mảng đã sắp xếp tương ứng để biết số chim cánh cụt có tọa độ ở chiều đó nhỏ hơn hoặc bằng tọa độ của truy vấn. Lấy giá trị nhỏ nhất trong bốn kết quả này để chọn chiều đem ra liệt kê, như một tối ưu hằng số.

Kỳ vọng đạt 30 điểm.

7.4 Tiếp tục tối ưu?

Nếu muốn dùng thuật toán trực tuyến, dường như không có cấu trúc dữ liệu phù hợp để trả lời truy vấn tọa độ bốn chiều, nên rất khó tiếp tục theo hướng đó.

Ta hãy thử chuyển sang xét cách xử lý truy vấn ngoại tuyến.

7.5 Thuật toán chia khối

Khi đã có ý tưởng ngoại tuyến, kết hợp thêm tư tưởng chia khối thì không khó nghĩ ra một thuật toán có độ phức tạp tốt hơn một chút so với vét cạn.

7.5.1 Rời rạc hóa và xử lý ngoại tuyến

Sắp xếp riêng các truy vấn và các chim cánh cụt theo tọa độ chiều thứ nhất.

Sau đó liệt kê các truy vấn.

7.5.2 Với truy vấn đang được liệt kê

Thêm vào tất cả các chim cánh cụt có tọa độ chiều thứ nhất nhỏ hơn hoặc bằng tọa độ chiều thứ nhất của truy vấn.

Sắp xếp theo chiều thứ hai. Mỗi khi thêm $\sqrt{n}$ chim cánh cụt thì thực hiện một lần sắp xếp bằng vét cạn; phần còn lại cũng xử lý vét cạn.

Độ phức tạp của phần này là $O(n\sqrt{n})$.

7.5.3 Khi thực hiện một lần sắp xếp vét cạn

Giả sử số chim cánh cụt đã được thêm vào và đã sắp xếp là M .

Ta chia khối, sắp xếp theo tọa độ chiều thứ ba, rồi theo chiều thứ tư duy trì cây đoạn lưu trọng số tiền tố.

Cụ thể, có thể dùng cây đoạn hàm thức.

7.5.4 Truy vấn

Với các khối đầy đủ: dùng nhị phân cộng truy vấn.

Với các phần không trọn khối: xử lý vét cạn.

Tổng độ phức tạp là $O(n\sqrt{n} \log n)$.

Kỳ vọng đạt 60–95 điểm.

7.6 Thuật toán tốt hơn

Thông thường, xử lý chia khối là một phương án thay thế hiệu quả khi tồn tại thuật toán có độ phức tạp tốt hơn nhưng độ phức tạp tư duy lại khá cao. Ở đây, chỉ cần suy nghĩ thêm một chút là ta có thể tìm được một thuật toán chia để trị tốt hơn.

7.6.1 Bước 1

Sắp xếp chung các truy vấn và chim cánh cụt theo tọa độ chiều thứ nhất.

Sau đó thực hiện “trộn” theo tọa độ chiều thứ hai.

7.6.2 Bước 2

Giả sử dãy chỉ số sau khi sắp xếp là $id[L]$ tới $id[R]$, trong đó có phần tử là chim cánh cụt và có phần tử là truy vấn. Gọi MID là điểm cuối của đoạn bên trái.

7.6.3 Bước 3

Đệ quy xử lý đáp án của hai phần $L-MID$ và $MID+1-R$. Dĩ nhiên, khi L và R bằng nhau thì có thể trả về ngay.

7.6.4 Bước 4

Ta thấy trường hợp chưa được xử lý chỉ còn là ảnh hưởng của chim cánh cụt ở nửa trái, tức $L-MID$, lên truy vấn ở nửa phải, tức $MID+1-R$. Lúc này cả hai nửa đều đã được sắp xếp theo tọa độ chiều thứ hai (lý do được giải thích ngay sau đây), và thứ tự theo chiều thứ nhất của chúng không còn cần thiết nữa.

Ta trộn nửa trái và nửa phải theo tọa độ chiều thứ hai, tức gộp hai mảng tuyến tính đã có thứ tự. Khi tạo mảng mới, nếu phần tử được thêm là chim cánh cụt ở nửa trái, ta chèn điểm tạo bởi tọa độ chiều thứ ba và chiều thứ tư của nó vào một “tập hợp”. Nếu phần tử được thêm là truy vấn ở nửa phải, ta hỏi trong “tập hợp” có bao nhiêu điểm nằm ở góc dưới bên trái của điểm tạo bởi hai tọa độ chiều thứ ba và chiều thứ tư của truy vấn đó, rồi cộng số này vào đáp án của truy vấn.

7.6.5 Bước 5

Về cách cài đặt “tập hợp”, ta có thể dùng cây BIT (hoặc cây đoạn) lồng cây cân bằng.

Tổng độ phức tạp thời gian là $O(n \log^3 n)$.

7.7 Các điểm kiểm tra

- Chuyển đổi tư duy từ trực tuyến sang ngoại tuyến.
- Vận dụng các tư tưởng chia khối, chia để trị.
- Xử lý thứ tự dữ liệu một cách khéo léo.
- Nắm vững cấu trúc dữ liệu.

8 Bàn về ứng dụng tư tưởng chia khối trong một lớp bài xử lý dữ liệu

Luo Jianqiao
Trường THPT số 80 Bắc Kinh

Tóm tắt

Trong thi đấu và thực tiễn, làm thế nào để tổ chức và xử lý dữ liệu quy mô lớn một cách hiệu quả là một vấn đề quan trọng. Bài viết này chủ yếu thảo luận một lớp bài xử lý dữ liệu trong thi đấu, liên quan tới dãy tuyến tính và cấu trúc cây. Tác giả đưa vào khái niệm tư tưởng chia khối, khảo sát ứng dụng của nó trong tổ chức dữ liệu, tiền xử lý dữ liệu và thiết kế thuật toán ngoại tuyến; đồng thời dùng ví dụ để minh họa cách chia dữ liệu thành khối nhằm thu được thuật toán có độ phức tạp thời gian thấp.

8.1 Dẫn nhập

Trong một lớp bài xử lý dữ liệu, ta cần thường xuyên thống kê thông tin về những phần tử thỏa mãn điều kiện nào đó trong một tập dữ liệu rất lớn, chẳng hạn số lượng hoặc giá trị cực trị. Các cấu trúc dữ liệu truyền thống, như cây đoạn, tổ chức toàn bộ dữ liệu trong tập theo một cách có thứ tự, để mỗi truy vấn có thể được tách thành tổng thông tin của một số phần có thứ tự, qua đó nâng cao hiệu quả.

Bài viết này giới thiệu một tư tưởng chia khối. Nó cũng làm dữ liệu có thứ tự và có tầng bậc, nhưng phương thức khác đi, khả năng mở rộng mạnh hơn và phạm vi áp dụng rộng hơn.

Cốt lõi của tư tưởng chia khối là chia một tập hợp thành một số tập con có quy mô nhỏ hơn. Nói chung, ta có xu hướng đưa các phần tử có tính chất gần nhau vào cùng một tập con, rồi tổ chức các phần tử trong mỗi tập con theo một cấu trúc nhất định.

Trong các bài xử lý dữ liệu, tư tưởng chia khối có phạm vi ứng dụng rộng vì cách chia khối có vài tính chất tốt:

- Quy mô của mỗi tập con nhỏ, nên có thể dùng thuật toán có độ phức tạp cao theo quy mô tập con.
- Số lượng tập con ít, nên có thể dùng thuật toán liên quan tới quy mô toàn cục để xử lý riêng từng tập con.
- Các phần tử trong mỗi tập con có tính chất chung, nên có thể thiết kế thuật toán có tính nhắm đích cho từng tập con.

Dễ thấy hai tính chất đầu mâu thuẫn với nhau: để cân bằng, ta không thể để tập con quá lớn, nhưng cũng không thể để số tập con quá lớn. Đây là lý do độ phức tạp thời gian của các thuật toán chia khối thường liên quan tới $\sqrt{N}$.

Bài viết chủ yếu bàn về ứng dụng của chia khối trong các bài xử lý dữ liệu liên quan tới dãy tuyến tính và cấu trúc cây. Mục 2 giới thiệu phương pháp chia khối chung trong các cấu trúc dữ liệu điển hình. Mục 3 giới thiệu ứng dụng của chia khối trong tiền xử lý dữ liệu. Mục 4 giới thiệu ứng dụng của chia khối khi thiết kế thuật toán ngoại tuyến.

8.2 Chia khối dữ liệu

Mục này giới thiệu phương pháp chia khối chung cho hai cấu trúc dữ liệu thường gặp là dãy tuyến tính và cây, rồi lần lượt đưa ra ví dụ dùng chia khối để giải.

8.2.1 Chia khối dãy tuyến tính

Trong dãy tuyến tính, mọi phần tử của tập dữ liệu được sắp xếp theo một thứ tự nhất định. Một bài điển hình là thường xuyên thống kê “tổng” thông tin của các phần tử thỏa mãn một tính chất nào đó trong một dãy con liên tiếp; ở đây “tổng” nghĩa là đáp án có thể được tách trực tiếp hoặc gián tiếp thành tổng đáp án của các phần khác nhau. Trước hết ta giới thiệu một cách chia khối thường dùng, sau đó minh họa cách sử dụng nó để giải hiệu quả loại bài này.

Cách chia khối như sau: bắt đầu từ phần tử đầu tiên của dãy, cứ mỗi S phần tử liên tiếp tạo thành một khối. Nếu cuối cùng còn ít hơn S phần tử, cho các phần tử còn lại tạo thành một khối.

Tính chất 2.1. Giả sử N là độ dài dãy. Khi dùng cách trên, số khối không vượt quá $\lceil N/S \rceil + 1$.

Tính chất 2.2. Giả sử một truy vấn hỏi thông tin của dãy con liên tiếp từ phần tử thứ L tới phần tử thứ R , gọi tắt là đoạn $[L, R]$. Khi đó, ngoài các phần tử nằm trong những khối được chứa trọn vẹn, số phần tử của đoạn $[L, R]$ còn dính tới không vượt quá $2S$.

Theo hai tính chất trên, nếu ta có thể duy trì ở từng khối tổng thông tin của toàn bộ phần tử trong một khối, thì chỉ cần chọn kích thước khối S thích hợp, với mọi đoạn ta đều có thể nhanh chóng tìm ra những khối trọn vẹn và những phần tử thừa, rồi gộp chúng để thu được thông tin của cả đoạn.

Ví dụ một. Cho một dãy gồm N số nguyên. Cần thực hiện M thao tác thuộc hai loại:

- ADD D_i X_i : bắt đầu từ số thứ nhất, cứ cách D_i vị trí thì cộng thêm X_i vào phần tử đó.
- QUERY L_i R_i : hỏi tổng các phần tử từ vị trí L_i tới R_i .

Quy mô dữ liệu: $1 \leq N, M \leq 10^5$; ở mọi thời điểm, giá trị tuyệt đối của mỗi số trong dãy không vượt quá 10^9 .

Phân tích thuật toán. Trong trường hợp xấu nhất, mỗi thao tác ADD và QUERY đều có thể liên quan tới N phần tử, nên mô phỏng đơn thuần sẽ rất chậm. Ta xét chia dãy thành khối và ghi ở từng khối tổng các phần tử trong từng khối.

Bổ đề 2.1. Nếu đặt kích thước khối là $\sqrt{N}$, mỗi truy vấn có thể xử lý trong $O(\sqrt{N})$ thời gian.

Cách làm cụ thể là khi truy vấn, liệt kê từng khối; nếu khối nằm trọn trong đoạn truy vấn thì cộng trực tiếp tổng của khối vào đáp án. Số phần tử còn lại không vượt quá $2\sqrt{N}$, nên có thể liệt kê trực tiếp.

Tuy nhiên, độ phức tạp của thao tác sửa vẫn chưa được cải thiện. Lúc này ta tiếp tục dùng tư tưởng chia khối và chú ý tới sự thật sau:

Bổ đề 2.2. Khi và chỉ khi thao tác ADD có $D_i < \sqrt{N}$, số phần tử cần cập nhật mới vượt quá $\sqrt{N}$.

Vì vậy ta phân loại thao tác ADD. Nếu $D_i \geq \sqrt{N}$, cập nhật trực tiếp các phần tử bị ảnh hưởng và tổng khối tương ứng, độ phức tạp là $O(\sqrt{N})$. Nếu $D_i < \sqrt{N}$, chỉ cần dùng mảng để ghi tổng lượng sửa của mọi thao tác ADD có cùng D_i ; lúc này một thao tác sửa là $O(1)$.

Khi đó, giá trị của từng phần tử và tổng từng khối chỉ xét loại sửa thứ nhất. Khi truy vấn, ta còn phải liệt kê mọi D_i thuộc loại thứ hai và tính ảnh hưởng của tổng lượng sửa với D_i đó lên đáp án. Vì số D_i thuộc loại thứ hai không vượt quá $\sqrt{N}$, độ phức tạp một truy vấn vẫn là $O(\sqrt{N})$.

Định lý 2.3. Bài toán trên có thể giải trong thời gian $O((N + M)\sqrt{N})$.

8.2.2 Chia khối cấu trúc cây

Trong cấu trúc cây, các đỉnh kề nhau có tương quan rất mạnh. Một ý tưởng trực quan là mong muốn chia cây thành một số khối liên thông có quy mô nhỏ. Dưới đây là một cách chia khối trên cây: với S thích hợp, nó có thể chia cây thành $O(N/S)$ khối liên thông kích thước $O(S)$, trong đó N là số đỉnh của cây.

Trước hết cần khái niệm dãy DFS. Với cây vô căn, có thể chọn tùy ý một đỉnh làm gốc để biến nó thành cây có gốc.

Định nghĩa 2.1 (Dãy DFS của cây). Duy trì một dãy, ban đầu rỗng. Từ gốc, thực hiện duyệt sâu; mỗi khi gặp một đỉnh lúc vào stack hoặc ra khỏi stack thì thêm nó vào cuối dãy. Dãy thu được sau khi duyệt xong gọi là dãy DFS.

Như vậy, với một cây bất kỳ ta thu được một dãy DFS có $2N$ phần tử; mỗi đỉnh của cây tương ứng với hai phần tử trong dãy. Dãy DFS có những tính chất tốt.

Bổ đề 2.4. Trong dãy DFS, quan hệ giữa hai phần tử kề nhau chỉ có ba loại: hoặc là cùng một đỉnh, hoặc là quan hệ cha con, hoặc là quan hệ anh em. Loại thứ ba xảy ra khi và chỉ khi trong DFS, phần tử trước là đỉnh được đưa ra khỏi stack, còn phần tử sau là đỉnh vừa được đưa vào stack.

Từ bổ đề 2.4 có thể chứng minh tính chất sau.

Bổ đề 2.5. Một dãy con liên tiếp từ hạng u tới hạng v trong dãy DFS vừa đúng tương ứng với một đường đi từ đỉnh ứng với hạng u tới đỉnh ứng với hạng v . Ngoài LCA của hai đầu mút ra, mọi đỉnh trên đường đi đều xuất hiện ít nhất một lần trong dãy con đó.

Từ đó có kết luận ta cần:

Định lý 2.6. Dùng phương pháp chia khối của mục trước để chia dãy DFS của cây thành các khối kích thước S . Với mỗi khối của dãy, tập các đỉnh tương ứng với toàn bộ phần tử trong khối cùng LCA của chúng tạo thành một khối liên thông trên cây. Số khối liên thông không vượt quá $2N/S$.

Phương pháp chia khối này cài đặt đơn giản, đồng thời có tính chất chung của tư tưởng chia khối: mỗi khối không lớn và số khối cũng không nhiều. Đáng tiếc là có thể tồn tại một số đỉnh xuất hiện trong nhiều hơn hai khối liên thông.

Ví dụ hai. Cho một cây N đỉnh và một số nguyên K . Mỗi cạnh có trọng số. Cần tính với mỗi đỉnh i , trong các khoảng cách từ $N - 1$ đỉnh còn lại tới đỉnh i , giá trị nhỏ thứ K là bao nhiêu.

Quy mô dữ liệu: $1 \leq K < N \leq 50000$; trọng số cạnh là số nguyên có giá trị tuyệt đối nhỏ hơn 1000.

Phân tích thuật toán. Thuật toán đơn giản là lần lượt chọn mỗi đỉnh làm gốc, tính khoảng cách từ mọi đỉnh khác tới nó, rồi sắp xếp để lấy đáp án. Độ phức tạp thời gian là $O(N^2 \log N)$. Nếu dùng sắp xếp cơ số, có thể giảm xuống $O(N^2)$.

Ta chia cây theo phương pháp ở trên thành một số khối liên thông kích thước $O(S)$. Xét việc lợi dụng tính liên quan giữa các đỉnh kề nhau để nhanh chóng tính đáp án khi từng đỉnh trong cùng một khối liên thông được chọn làm gốc.

Bổ đề 2.7. Giả sử u là một đỉnh bất kỳ không nằm trong khối liên thông hiện tại. Dù chọn đỉnh nào trong khối làm gốc, trên đường từ u tới gốc, đỉnh đầu tiên nằm trong khối là không đổi.

Theo bổ đề 2.7, ta phân loại mọi đỉnh ngoài khối theo đỉnh đầu tiên trong khối mà đường đi tới gốc đi qua, gọi tắt là tổ tiên gần nhất trong khối. Rõ ràng các đỉnh ngoài khối được chia thành nhiều nhất S loại.

Bổ đề 2.8. Giả sử u và v là hai đỉnh bất kỳ không nằm trong khối hiện tại và thuộc cùng một loại. Gọi p là tổ tiên gần nhất trong khối của chúng. Khi chọn bất kỳ đỉnh r nào trong khối làm gốc, quan hệ thứ tự giữa khoảng cách từ u tới r và từ v tới r đúng bằng quan hệ thứ tự giữa khoảng cách từ u tới p và từ v tới p .

Vì vậy, với mỗi loại, ta sắp xếp các đỉnh của loại đó theo khoảng cách tới tổ tiên gần nhất trong khối từ nhỏ tới lớn. Sau đó liệt kê từng đỉnh trong khối làm gốc và lần lượt tính đáp án.

Định lý 2.9. Khi chọn một đỉnh r bất kỳ trong khối liên thông làm gốc, có thể trong $O(S \log^2 N)$ thời gian tính được giá trị nhỏ thứ K trong các khoảng cách từ các đỉnh khác tới r .

Cách làm cụ thể như sau. Để tính giá trị nhỏ thứ K , ta nhị phân đáp án. Giả sử giá trị nhị phân là X . Từ r , liệt kê mọi đỉnh trong khối, tính khoảng cách của chúng tới r và đếm có bao nhiêu khoảng cách không vượt quá X . Với mỗi đỉnh p trong khối, giả sử khoảng cách từ p tới r là D_p ; trong bảng có thứ tự của loại nhận p làm tổ tiên gần nhất trong khối, nhị phân vị trí lớn nhất có giá trị không vượt quá $X - D_p$, qua đó tính số đỉnh ngoài khối thuộc loại này và cách r không quá X .

Sau khi xử lý mọi p , giả sử số đỉnh cách r không quá X là cnt . Nếu $\text{cnt} < K$, đáp án thật lớn hơn X ; ngược lại đáp án thật không lớn hơn X . Mỗi lần nhị phân thu hẹp một nửa phạm vi đáp án, do đó đạt độ phức tạp $O(S \log^2 N)$.

Bổ đề 2.10. Tính cả thời gian sắp xếp và số đỉnh trong khối, ta có thể tính đáp án cho mọi đỉnh trong một khối liên thông trong

$$O(N \log N + S^2 \log^2 N)$$

thời gian.

Định lý 2.11. Nếu chọn kích thước khối $S = \Theta(\sqrt{N / \log N})$, độ phức tạp thời gian tổng cộng của thuật toán là $O(N^{1.5} \log^{1.5} N)$.

8.2.3 Mở rộng thêm

Bạn đọc quan tâm có thể tự suy nghĩ cách mở rộng phương pháp chia khối dãy tuyến tính lên trường hợp d chiều, cũng như việc phương pháp chia khối cây có áp dụng được cho các cấu trúc phức tạp hơn hay không.

8.3 Chia khối và tiền xử lý

Tiền xử lý nghĩa là khi duy trì một tập dữ liệu, trước mọi thao tác truy vấn ta xử lý dữ liệu trước, thường ghi lại nhiều thông tin phụ, rồi khi truy vấn thì dùng thông tin đã xử lý để tăng hiệu quả thời gian. Nếu có thao tác sửa, còn phải xét ảnh hưởng của sửa lên thông tin đã tiền xử lý.

Kết hợp tư tưởng chia khối với tiền xử lý có thể giải hiệu quả một lớp bài truy vấn tập hợp rất rộng. Mục này trước hết nêu các tính chất mà loại bài có thể tối ưu bằng tiền xử lý cần thỏa mãn, sau đó giới thiệu chi tiết phương pháp tiền xử lý chung trên dãy và trên cây.

8.3.1 Điều kiện để tiền xử lý

Giả sử tập dữ liệu đang duy trì là tập S .

Điều kiện 3.1 (Có thể tăng lượng). Giả sử hai lần truy vấn tương ứng với hai tập con A và B của tập S . Độ phức tạp để chuyển thông tin liên quan của tập truy vấn A thành thông tin liên quan của tập truy vấn B là một hàm theo kích thước hiệu đối xứng của A và B , tức

$$(A - B) \cup (B - A).$$

Nói cách khác, độ phức tạp chuyển trạng thái chỉ phụ thuộc vào số phần tử thuộc hiệu đối xứng.

Gọi $\mathcal{P}$ là tập hợp mọi tập con của S có thể xuất hiện trong truy vấn.

Điều kiện 3.2 (Có thể tiền xử lý). Với các số nguyên K và L cho trước, tồn tại một tập con $\mathcal{Q} \subseteq \mathcal{P}$ có kích thước không vượt quá K , sao cho với mọi phần tử $X \in \mathcal{P}$, tồn tại một phần tử $Y \in \mathcal{Q}$ mà kích thước hiệu đối xứng giữa X và Y không vượt quá L .

Nếu truy vấn trên tập dữ liệu thỏa mãn điều kiện 3.1, đồng thời tồn tại K, L thích hợp để tập dữ liệu thỏa mãn điều kiện 3.2, ta có thể dùng phương pháp chia khối và tiền xử lý để tăng hiệu quả thời gian của thao tác truy vấn.

Khi đó ta tìm tập $\mathcal{Q}$ trong điều kiện 3.2 và tiền xử lý thông tin liên quan của các phần tử trong $\mathcal{Q}$. Mỗi khi truy vấn phần tử X , tìm phần tử $Y \in \mathcal{Q}$ tương ứng. Vì thông tin của Y đã được tiền xử lý, và truy vấn thỏa mãn điều kiện 3.1, ta có thể nhanh chóng chuyển thông tin của Y thành thông tin của X trong $O(f(L))$ thời gian.

8.3.2 Tiền xử lý trên dãy tuyến tính

Mục nhỏ này xét một lớp bài trên dãy tuyến tính: thường xuyên truy vấn thông tin của một dãy con liên tiếp và thỏa mãn điều kiện tiền xử lý. Trước hết là phương pháp tiền xử lý, sau đó là một ví dụ.

Ta chia dãy thành khối. Giả sử kích thước mỗi khối là S , số khối là $O(N/S)$. Theo điều kiện 3.1, với bất kỳ đoạn có len phần tử, ta có thể thu được thông tin liên quan trong thời gian $O(f(\text{len}))$. Do đó ta tiền xử lý thông tin liên quan giữa hai khối bất kỳ như sau.

Lấy một khối A bất kỳ, xét việc lần lượt tính thông tin giữa khối A và từng khối B sau nó theo thứ tự vị trí trong dãy. Sau khi tính được thông tin từ khối A tới khối B_i , có thể trong $O(f(S))$ thời gian chuyển thành thông tin từ khối A tới khối B_{i+1} .

Định lý 3.1. Chỉ xét các khối B_i nằm sau khối A , nếu không tính độ phức tạp ghi thông tin, có thể trong $O(Nf(S)/S)$ thời gian tính thông tin giữa khối A và mọi khối B_i .

Nếu độ phức tạp ghi thông tin giữa hai khối bất kỳ có thể bỏ qua, ta liệt kê khối A và hoàn thành toàn bộ quá trình chia khối, tiền xử lý trong $O(N^2f(S)/S^2)$ thời gian.

Theo tính chất 2.2, thông tin của một đoạn bất kỳ $[L, R]$ có thể được suy ra từ thông tin của một dãy con giữa hai khối đã tiền xử lý bằng cách thêm không quá $2S$ phần tử. Vì vậy:

Định lý 3.2. Với bài thống kê đoạn trên dãy tuyến tính, nếu bài toán thỏa mãn điều kiện tiền xử lý, trước hết chia khối và tiền xử lý dãy; sau đó mỗi truy vấn đoạn $[L, R]$ có thể trả lời trong $O(f(S))$ thời gian.

Ví dụ ba. Cho một dãy gồm N số, mỗi số là một số nguyên trong khoảng 1 tới N . Cần trả lời M truy vấn. Mỗi truy vấn cho ba số nguyên L_i, R_i, K_i , hỏi trong đoạn $[L_i, R_i]$ có bao nhiêu số xuất hiện đúng K_i lần.

Quy mô dữ liệu: $1 \leq N, M \leq 10^5$.

Phân tích thuật toán. Trong bài này, đáp án của các đoạn kề nhau không thể nhanh chóng gộp thành đáp án của đoạn hợp bằng cách ghi vài thông tin phụ, nên không thể dùng những cấu trúc dữ liệu như cây đoạn để duy trì. Nhưng truy vấn có tính chất sau:

Bổ đề 3.3. Với hai đoạn bất kỳ $[L_1, R_1]$ và $[L_2, R_2]$, nếu ta đã ghi số lần xuất hiện trong $[L_1, R_1]$ của từng số từ 1 tới N , cũng như số lượng các số có tần suất 1 tới N , thì có thể trong

$$O(|L_1 - L_2| + |R_1 - R_2|)$$

thời gian chuyển chúng thành thông tin tương ứng của đoạn $[L_2, R_2]$.

Vì vậy có thể dùng phương pháp tiền xử lý. Chia dãy thành khối với kích thước mỗi khối là $O(\sqrt{N})$, số khối cũng là $O(\sqrt{N})$. Ta có thể trong $O(N\sqrt{N})$ thời gian tính số lần xuất hiện của mỗi số từ 1 tới N , và số lượng các số có tần suất 1 tới N , giữa hai khối bất kỳ. Nhưng nếu ghi toàn bộ thông tin như vậy thì dung lượng là $O(N^2)$, nên cần giảm lượng thông tin phải ghi.

Bổ đề 3.4. Đánh số các khối từ trái sang phải là 1 tới $\sqrt{N}$. Số lần xuất hiện của một số x trong các khối từ i tới j bằng số lần x xuất hiện trong j khối đầu trừ số lần x xuất hiện trong $i - 1$ khối đầu.

Ta chỉ duy trì số lần xuất hiện của mỗi số từ 1 tới N trong tiền tố từ 1 tới $\sqrt{N}$ khối, giảm độ phức tạp không gian xuống $O(N\sqrt{N})$. Việc này tương đương với duy trì số lần xuất hiện giữa hai khối bất kỳ.

Bổ đề 3.5. Số giá trị xuất hiện trong dãy ít nhất $\sqrt{N}$ lần không vượt quá $\sqrt{N}$.

Khi tiền xử lý, ta chỉ ghi số lượng các giá trị có số lần xuất hiện từ 1 tới $\sqrt{N}$ giữa hai khối. Do đó độ phức tạp ghi thông tin là $O(N\sqrt{N})$. Theo định lý 3.2, ta có thể trả lời đúng mọi truy vấn có $K_i < \sqrt{N}$ trong $O(\sqrt{N})$ thời gian.

Với truy vấn có $K_i \geq \sqrt{N}$, ta xử lý trước tổng tiền tố tần suất của mọi giá trị xuất hiện nhiều hơn $\sqrt{N}$ lần. Khi truy vấn, liệt kê những giá trị này trong $O(\sqrt{N})$ thời gian, và $O(1)$ để kiểm tra tần suất của từng giá trị trong đoạn truy vấn có bằng K_i hay không.

Định lý 3.6. Thuật toán trên có thể hoàn thành tiền xử lý trong $O(N\sqrt{N})$ thời gian. Mỗi truy vấn được trả lời trong $O(\sqrt{N})$ thời gian. Tổng độ phức tạp thời gian và không gian là $O((N + M)\sqrt{N})$.

8.3.3 Tiền xử lý trên cấu trúc cây

Mục nhỏ này xét một lớp bài trên cây: thường xuyên truy vấn thông tin của đường đi giữa hai đỉnh và thỏa mãn điều kiện tiền xử lý. Ta mở rộng phương pháp chia khối tiền xử lý từ dãy tuyến tính lên cây, mô tả cách đánh dấu các đỉnh then chốt trên cây, rồi đưa ra một ví dụ.

Bổ đề 3.7. Trong cây, đường đi đơn giữa hai đỉnh bất kỳ là duy nhất.

Bổ đề 3.8. Với hai đỉnh bất kỳ u, v trong cây có gốc, gọi p là LCA của chúng. Đường đi từ u tới v vừa đúng bằng đường đi từ u tới p và đường đi từ p tới v .

Ta muốn đánh dấu một số đỉnh của cây làm đỉnh then chốt, đồng thời tiền xử lý đường đi giữa mọi cặp đỉnh then chốt, sao cho mọi đường đi đủ dài trên cây đều chứa một đường đi đã được xử lý, còn phần còn lại ngoài đường đi đó có số đỉnh ít. Dưới đây là một thuật toán tham lam.

Với cây có gốc, liệt kê các đỉnh theo thứ tự độ sâu giảm dần và quyết định có đánh dấu chúng làm đỉnh then chốt hay không. Đánh dấu một đỉnh u khi và chỉ khi trong cây con của u tồn tại một đỉnh mà trên đường từ đỉnh đó tới u chưa có đỉnh then chốt, đồng thời số đỉnh trên đường này lớn hơn S . Ở đây S là số cho trước.

Bổ đề 3.9. Giả sử N là số đỉnh của cây. Thuật toán tham lam đánh dấu không quá N/S đỉnh then chốt.

Lý do là mỗi đỉnh then chốt đều là đỉnh then chốt đầu tiên trên đường tới gốc của ít nhất S đỉnh không then chốt.

Bổ đề 3.10. Giả sử độ sâu của gốc là 0. Với bất kỳ đỉnh nào có độ sâu không nhỏ hơn S , trong $S + 1$ tổ tiên gần nhất của nó, kể cả chính nó, có ít nhất một đỉnh then chốt.

Hệ quả 3.11. Với bất kỳ đường đi đơn nào trong cây có số đỉnh lớn hơn $3S$, gọi u và v là hai đầu mút. Khoảng cách từ u tới đỉnh then chốt gần nhất trên đường đi không vượt quá $2S - 2$; tổng khoảng cách từ u và v tới các đỉnh then chốt gần nhất trên đường đi không vượt quá $3S - 3$.

Trong thuật toán tham lam đánh dấu đỉnh then chốt, chỉ cần ghi với mỗi đỉnh giá trị lớn nhất của số đỉnh trên một đường trong cây con của nó, chỉ gồm các đỉnh không then chốt và kết thúc tại nó, là có thể quyết định có đánh dấu nó hay không.

Định lý 3.12. Trong $O(N)$ thời gian, có thể đánh dấu $O(N/S)$ đỉnh then chốt trong cây. Nếu việc tiền xử lý giữa các cặp đỉnh then chốt có độ phức tạp $O((N/S)^2 f(N))$, thì với mọi đường đi đơn, sau khi bỏ phần đã được xử lý, số đỉnh còn lại là $O(S)$.

Ví dụ bốn. Cho một cây N đỉnh. Màu của mỗi đỉnh là một số nguyên từ 1 tới M .

Định nghĩa lợi ích du lịch từ đỉnh u tới đỉnh v là

$$\sum_i V_i W_{C_i},$$

trong đó i chạy qua các màu, C_i là số lần màu i xuất hiện trên đường đi đơn từ u tới v , và $W_0, W_1, \dots, W_N$ là dãy số nguyên.

Cần thực hiện Q thao tác thuộc hai loại:

- CHANGE $x_i \ y_i$: đổi màu của đỉnh x_i thành y_i .
- QUERY $u_i \ v_i$: hỏi lợi ích du lịch từ u_i tới v_i .

Quy mô dữ liệu: $1 \leq N, M, Q \leq 10^5$, $1 \leq V_i, W_i \leq 10^6$. Giới hạn thời gian của bài là 10 giây. Bài gốc là *Candy Park* của NOI Winter Camp 2013.

Phân tích thuật toán. Chọn một đỉnh bất kỳ làm gốc. Đặt $S = \Theta(N^{1/3})$, dùng phương pháp đánh dấu đỉnh then chốt để tiền xử lý trên cây, ghi số lần xuất hiện của các màu 1 tới M và lợi ích du lịch trên đường đi giữa mọi cặp đỉnh then chốt. Độ phức tạp tiền xử lý là $O(N^{5/3})$.

Với thao tác truy vấn đường đi từ u_i tới v_i , trước hết dùng nhân đôi hoặc thuật toán hiệu quả khác để tìm LCA của hai đỉnh. Sau đó lần lượt đi từ u_i và v_i dọc theo đường đi để tìm các đỉnh then chốt gần u_i và v_i nhất trên đường. Khi đó có thể dùng thông tin đã ghi và một số thao tác tăng lượng để thu được đáp án trong $O(N^{1/3})$ thời gian.

Với thao tác đổi màu đỉnh x_i , liệt kê mọi cặp đỉnh then chốt và kiểm tra x_i có nằm trên đường đi giữa chúng hay không. Nếu có thì cập nhật thông tin đã ghi.

Bổ đề 3.13. Đỉnh x_i nằm trên đường đi từ u tới v khi và chỉ khi LCA của u và v là tổ tiên của x_i , đồng thời x_i là tổ tiên của u hoặc v .

Nếu tiền xử lý thời điểm vào stack và ra khỏi stack trong DFS cho mỗi đỉnh, thì x là tổ tiên của y khi và chỉ khi thời điểm vào stack của y nằm giữa thời điểm vào và ra của x . Trong lúc tiền xử lý, ta ghi LCA của mọi cặp đỉnh then chốt, nên có thể $O(1)$ kiểm tra một đỉnh có nằm trên đường đi giữa chúng hay không. Vì vậy thao tác sửa có độ phức tạp $O(N^{2/3})$.

Định lý 3.14. Bài toán này có thể giải trong $O((N + Q)N^{2/3})$ thời gian.

8.4 Chia khối và thuật toán ngoại tuyến

Một số bài xử lý dữ liệu không yêu cầu trả lời từng truy vấn ngay lập tức, mà cho trước rất nhiều truy vấn và yêu cầu chương trình nhanh chóng tính toàn bộ kết quả rồi trả về một lần. Thuật toán ngoại tuyến là thuật toán biết trước nội dung của mọi truy vấn, vì vậy có thể giải chúng cùng nhau.

Trong bài ngoại tuyến, vì đã biết toàn bộ truy vấn, ta có thể tổ chức các truy vấn theo một cách nhất định, tận dụng đầy đủ thông tin trùng lặp giữa chúng. Về bản chất, ta xem toàn bộ truy vấn như một phần của tập dữ liệu, rồi dùng phương pháp xử lý tập dữ liệu để xử lý truy vấn.

Mục này dùng hai ví dụ để minh họa ứng dụng của tư tưởng chia khối khi thiết kế thuật toán ngoại tuyến.

Ví dụ năm. Cho một dãy gồm N số, mỗi số là một số nguyên trong khoảng 1 tới N . Cần trả lời M truy vấn. Mỗi truy vấn cho ba số nguyên L_i, R_i, K_i , hỏi trong đoạn $[L_i, R_i]$ có bao nhiêu số xuất hiện đúng K_i lần.

Quy mô dữ liệu: $1 \leq N, M \leq 10^5$.

Phân tích bài toán. Chia dãy thành khối, đặt kích thước mỗi khối là $O(\sqrt{N})$. Số khối cũng là $O(\sqrt{N})$.

Đọc toàn bộ truy vấn một lần và phân loại truy vấn theo khối chứa đầu trái L_i , sau đó xử lý riêng từng loại.

Xét mọi truy vấn có đầu trái nằm trong khối A_i . Sắp xếp các truy vấn này theo vị trí đầu phải trong dãy từ nhỏ tới lớn. Sau đó quét lần lượt mọi phần tử từ đầu trái của khối A_i tới cuối dãy, dùng mảng duy trì tại từng thời điểm số lượng các giá trị có tần suất 1 tới N trong các phần tử đã quét, và tần suất của từng giá trị từ 1 tới N .

Với mỗi truy vấn $[L_i, R_i]$, khi quét tới đầu phải R_i , ta có thể trong $O(\sqrt{N})$ thời gian chuyển thông tin đang duy trì thành thông tin của đoạn $[L_i, R_i]$. Sau khi ghi đáp án của truy vấn, khôi phục thông tin. Mỗi truy vấn được xử lý đúng một lần.

Định lý 4.1. Thuật toán này có độ phức tạp thời gian $O((N + M)\sqrt{N})$ và độ phức tạp không gian $O(N)$.

Trong ví dụ này, ta dùng cách ngoại tuyến để giải quyết vấn đề dung lượng ghi thông tin quá lớn của ví dụ ba. Có thể thấy kết hợp chia khối với thuật toán ngoại tuyến có phần tương tự phương pháp tiền xử lý; hơn nữa, những bài có thể giải bằng chia khối và tiền xử lý thường có thể giảm độ phức tạp không gian trong tình huống ngoại tuyến.

Ví dụ sáu. Cho một cây N đỉnh. Mỗi đỉnh có trọng số là một số nguyên không âm nhỏ hơn 2^{31} . Cần trả lời M truy vấn; mỗi truy vấn hỏi số lượng trọng số đỉnh phân biệt xuất hiện trên đường đi đơn từ đỉnh u_i tới đỉnh v_i .

Quy mô dữ liệu: $1 \leq N \leq 40000$, $1 \leq M \leq 100000$. Bài gốc là *Count On A Tree II* trên Sphere Online Judge.

Phân tích bài toán. Trước hết rời rạc hóa mọi trọng số của đỉnh; sau đó có thể xem trọng số là số nguyên không vượt quá N . Dùng phương pháp đánh dấu đỉnh then chốt trên cây đã giới thiệu ở mục 3.3. Đặt $S = \Theta(\sqrt{N})$; khi đó số đỉnh then chốt cũng là $O(\sqrt{N})$. Theo hệ quả 3.11, khoảng cách từ mỗi đỉnh tới đỉnh then chốt gần nhất là $O(\sqrt{N})$.

Đọc toàn bộ truy vấn và phân loại mỗi truy vấn theo đỉnh then chốt gần nhất với đầu nút u_i tương ứng. Sau đó xử lý riêng từng loại.

Xét một loại truy vấn có chung đỉnh then chốt r_i , là đỉnh then chốt gần nhất với các đầu nút u_i . Lấy r_i làm gốc và DFS toàn bộ cây; đồng thời duy trì, trong các đỉnh đã vào stack nhưng chưa ra stack, số lần xuất hiện của từng trọng số và số lượng trọng số phân biệt.

Bổ đề 4.2. Khi đang thăm một đỉnh x_i , mọi đỉnh đã vào stack nhưng chưa ra stack chính là toàn bộ đỉnh trên đường từ x_i tới gốc.

Bổ đề 4.3. Đường đi từ một đỉnh u tới một đỉnh v tương đương với đường từ u tới gốc cộng với đường từ v tới gốc, rồi trừ đường từ $LCA(u, v)$ tới gốc.

Với mỗi truy vấn (u_i, v_i) , khi DFS tới v_i , thông tin trong stack chính là thông tin của các đỉnh trên đường từ v_i tới gốc. Lúc này liệt kê từng đỉnh x trên đường từ u_i tới gốc r_i . Nếu x là $LCA(u_i, v_i)$ thì không thay đổi; nếu x nằm trên đường từ $LCA(u_i, v_i)$ tới gốc r_i thì trừ trọng số của x ; ngược lại cộng trọng số của x . Sau khi thu được đáp án, hoàn tác các thay đổi tạm thời.

Bổ đề 4.4. Có thể trả lời một truy vấn trong $O(\sqrt{N})$ thời gian.

Định lý 4.5. Bài toán này có thể giải trong $O((N + M)\sqrt{N})$ thời gian và $O(N)$ không gian.

8.5 Tổng kết

Nói một cách hình tượng, bản chất của tư tưởng chia khối là chuyển một thuật toán đơn giản có độ phức tạp $O(f(N))$ thành một thuật toán có độ phức tạp

$$O\left(g(S) + \frac{N}{S}h(S)\right),$$

trong đó S là kích thước khối, còn N/S là số khối. Với các bài khác nhau, biểu hiện cụ thể của điều này cũng khác nhau.

Lấy ví dụ một làm mẫu: thuật toán tham khảo của bài đó thể hiện tư tưởng chia khối ở hai chỗ. Một chỗ là dùng hai cấu trúc dữ liệu khác nhau để duy trì các sửa có $D_i < \sqrt{N}$ và các sửa có $D_i \geq \sqrt{N}$. Chỗ còn lại là khi thao tác sửa rất thường xuyên, ta dùng cấu trúc chia khối để duy trì tổng đoạn của dãy. Trong bài đặc biệt này, cách đó hiệu quả hơn cây đoạn, vì số lượng thao tác sửa có thể đạt tới $O(\sqrt{N})$ lần số lượng truy vấn. Các ví dụ sau cũng tương tự. Tuy nhiên, có khi ta chia một bài toán thành nhiều bài con có bản chất giống nhau, có khi lại tách một bài toán thành nhiều bài con khác nhau khá lớn.

Bài viết đưa ra các mô hình cơ bản của chia khối trên dãy tuyến tính và cấu trúc cây; chúng có hiệu lực với một lớp bài rất rộng. Nhưng áp dụng tư tưởng chia khối như thế nào còn tùy từng bài cụ thể. Các bạn không nên bị bó buộc bởi vài hình thức được nêu ở đây, mà cần mạnh dạn sáng tạo. Tất nhiên, nếu

tồn tại thuật toán hiệu quả hơn chia khối, cũng phải biết lựa chọn, không nên bảo thủ. Hy vọng các ví dụ này đem lại gợi ý cho người đọc.

8.6 Tài liệu tham khảo

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*, 2nd edition.
2. Jon Kleinberg, Eva Tardos. *Algorithm Design*.
3. Wu Wenhui, Wang Jiande. *Giáo trình nâng cao cuộc thi lập trình sinh viên quốc tế ACM/ICPC*, tập 1.
4. Zheng Dun. *Quy hoạch cân bằng: phân tích ứng dụng của một lớp tư tưởng cân bằng*.
5. Qi Zichao. *Ứng dụng thuật toán chia để trị trong các bài toán đường đi trên cây*.
6. Tài liệu về *Candy Park*, NOI Winter Camp 2013.
7. Báo cáo ra đề *JZPLCM*.
8. Xu Jie. Báo cáo ra đề *Đồ thị trong đồ thị*.
9. Xu Haoran. *Bàn rộng về cấu trúc dữ liệu*.

8.7 Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Xin cảm ơn thầy hướng dẫn Gong Zhiyong đã quan tâm và giúp đỡ tác giả trong nhiều năm.

Xin cảm ơn các huấn luyện viên đội tuyển quốc gia Hu Weidong và Tang Wenbin vì sự giúp đỡ.

Đặc biệt cảm ơn bạn Fan Haoqiang ở Bắc Kinh đã kiên nhẫn giúp đỡ tác giả trong quá trình viết bài. Tác giả học được rất nhiều và nhận được không ít gợi ý qua các trao đổi với bạn ấy.

Xin cảm ơn đàn anh Huang Jiyuan vì sự giúp đỡ.

Xin cảm ơn các thầy cô và bạn học khác từng giúp đỡ hoặc gợi ý cho tác giả.

Cuối cùng, xin cảm ơn cha mẹ tôi vì sự quan tâm và chăm sóc chu đáo.

9 Kỹ thuật *meet in the middle* trong bài toán tìm kiếm

Qiao Mingda

Trường Ngoại ngữ Nam Kinh, Giang Tô

9.1 Tóm tắt

Meet in the middle là một kỹ thuật tối ưu hóa tìm kiếm thường gặp. Một số bài toán tìm kiếm có thể dùng kỹ thuật này để giảm mạnh độ phức tạp thời gian, nhờ đó tìm được kết quả trong giới hạn thời gian.

Bài viết gồm ba phần. Phần thứ nhất đưa ra một mô hình đồ thị có hướng trừu tượng, dựa vào đó giới thiệu tư tưởng của kỹ thuật *meet in the middle*, và dùng kỹ thuật này để thiết kế thuật toán cho mô hình đồ thị có hướng. Phần thứ hai chứng minh ngắn gọn tính đúng đắn, độ phức tạp thời gian và điều kiện sử dụng của thuật toán trong phần thứ nhất. Phần thứ ba khảo sát vài bài ví dụ dùng kỹ thuật này; từ các ví dụ đó khái quát thành một mô hình phương trình, rồi phân tích một số biến thể của kỹ thuật.

9.2 Phần thứ nhất: mô hình đồ thị có hướng

9.2.1 Bản chất bài toán

Bản chất của một phần các bài toán tìm kiếm là: trong đồ thị có hướng G , tính số đường đi khác nhau có độ dài L từ đỉnh A tới đỉnh B . Ở đây, một “đường đi độ dài L ” được định nghĩa là một dãy cạnh có hướng

$$(E_1, E_2, \dots, E_L),$$

trong đó điểm đầu của E_1 là A , điểm cuối của E_L là B , và với mọi $i = 1, 2, \dots, L - 1$, điểm cuối của E_i bằng điểm đầu của E_{i+1} . Hai đường đi

$$(E_1, E_2, \dots, E_L) \quad \text{và} \quad (E'_1, E'_2, \dots, E'_L)$$

khác nhau khi và chỉ khi tồn tại $1 \leq i \leq L$ sao cho $E_i \neq E'_i$.

Giả sử bậc ra lớn nhất của một đỉnh trong G là hằng số D , không gian cần để mô tả một đỉnh là M , và thời gian cần để tính một hậu kế xác định của một đỉnh là T .

Ví dụ, mỗi đỉnh có nhãn là một dãy độ dài N ,

$$P_1, P_2, \dots, P_N.$$

Từ một đỉnh, ta có thể lật dãy $P_1, P_2, \dots, P_{N-1}$ hoặc dãy $P_2, P_3, \dots, P_N$ để thu được hậu kế. Ở đây, “lật $P_1, P_2, \dots, P_{N-1}$ ” nghĩa là đổi chỗ P_1 với P_{N-1} , P_2 với P_{N-2} , v.v., cho tới $P_{\lfloor N/2 \rfloor}$ và $P_{\lfloor N/2 \rfloor + 1}$. Như vậy mỗi đỉnh có đúng hai hậu kế; mô tả một đỉnh cần $O(N)$ không gian để lưu dãy; tính một hậu kế cần $O(N)$ thời gian để lật dãy. Trong ví dụ này $D = 2$, $M = O(N)$, và $T = O(N)$.

Theo các giả thiết trên, ta phân tích thuật toán DFS thô để giải bài toán này. Thuật toán giới hạn độ sâu DFS không vượt quá L ; khi độ sâu tìm kiếm đạt L , nó so sánh đỉnh hiện tại với B , nếu là cùng một đỉnh thì tăng đáp án thêm 1. Để chứng minh số đỉnh đi qua trong quá trình tìm kiếm không vượt quá

$$1 + D + D^2 + \dots + D^L = \frac{D^{L+1} - 1}{D - 1} = O(D^L),$$

và số đỉnh đạt được sau khi tìm L bước không vượt quá $D^L = O(D^L)$. Lưu ý rằng nếu cùng một đỉnh được đi qua nhiều lần thì phải tính nhiều lần.

Vì một đỉnh chiếm $O(M)$ không gian, so sánh một đỉnh có phải B hay không cần $O(M)$ thời gian. Mặt khác, trong quá trình tìm kiếm, tính mỗi hậu kế cần $O(T)$ thời gian. Vì vậy độ phức tạp thời gian của DFS thô là

$$O(D^L(M + T)).$$

Khi L lớn, thời gian chạy của thuật toán thô sẽ rất dài; trong đa số tình huống nó không thể tính ra đáp án trong giới hạn thời gian. Ta cần xét cách tối ưu hóa tìm kiếm.

Trong lớp bài này, số đỉnh của G rất lớn, thậm chí G có thể là đồ thị vô hạn, nên số bài toán con trùng lặp trong quá trình tìm kiếm khá ít; quy hoạch động không phải một phương pháp hiệu quả. Vì trong trường hợp xấu nhất ta buộc phải duyệt cả cây tìm kiếm, các kỹ thuật tối ưu hóa tìm kiếm nghiệm tối ưu cũng không phù hợp với loại bài đếm số đường đi này. Ta cần thay đổi cách tìm kiếm ở mức tổng thể.

9.2.2 Tư tưởng *meet in the middle*

Nghĩa đen của *meet in the middle* là “gặp nhau ở giữa”. Giả sử có hai người, một người xuất phát từ A , người kia xuất phát từ B , cùng đi về phía điểm xuất phát của người còn lại. Nếu khi cả hai đều đi được $L/2$ mét họ vừa đúng “gặp nhau ở giữa”, thì có thể khẳng định ta đã tìm được một đường đi từ A tới B có độ dài L mét.

Dưới đây ta xét cách áp dụng tư tưởng này vào mô hình đồ thị có hướng.

9.2.3 Quy trình thuật toán cho mô hình đồ thị có hướng

Để thuận tiện mô tả, ta dựng một đồ thị mới G' . Các đỉnh của G' giống với G , nhưng mỗi cạnh có hướng từ X tới Y trong G tương ứng với một cạnh có hướng từ Y tới X trong G' . Ngoài ra, đặt

$$L_1 = \left\lfloor \frac{L}{2} \right\rfloor, \quad L_2 = \left\lceil \frac{L}{2} \right\rceil.$$

Xét riêng tính chẵn lẻ của L sẽ dễ chứng minh $L_1 + L_2 = L$.

Phần thứ nhất của thuật toán thực hiện DFS trong G từ đỉnh A , giới hạn độ sâu tìm kiếm không vượt quá L_1 . Ta ghi lại toàn bộ các đỉnh đạt được sau đúng L_1 bước và số lần đạt được từng đỉnh. Ký hiệu số lần đi L_1 bước tới đỉnh P là $\text{Count}(P)$. Nếu sau L_1 bước chưa từng tới P , quy ước $\text{Count}(P) = 0$.

Phần thứ hai của thuật toán thực hiện DFS trong G' từ đỉnh B , giới hạn độ sâu tìm kiếm không vượt quá L_2 . Nếu sau đúng L_2 bước tìm kiếm tới đỉnh Q , ta tuyên bố đã tìm được $\text{Count}(Q)$ đường đi có độ dài $L_1 + L_2 = L$, và cộng $\text{Count}(Q)$ vào đáp án. Lưu ý rằng sau L_2 bước tìm kiếm ta có thể tới cùng một đỉnh nhiều lần; mỗi lần tới đều phải cộng $\text{Count}(Q)$.

Từ quy trình trên có thể thấy, *meet in the middle* là một kỹ thuật “tìm kiếm hai chiều”.

9.3 Phần thứ hai: phân tích thuật toán

9.3.1 Tính đúng đắn

Đặt

$$L_1 = \left\lfloor \frac{L}{2} \right\rfloor, \quad L_2 = \left\lceil \frac{L}{2} \right\rceil.$$

Với một đỉnh P trong G , gọi $f(P)$ là số đường đi từ A tới P có độ dài L_1 , và gọi $g(P)$ là số đường đi từ P tới B có độ dài L_2 .

Bổ đề 1. Số đường đi từ A tới B có độ dài L là

$$\sum_P f(P)g(P).$$

Chứng minh. Xét một đường đi bất kỳ có độ dài L ,

$$(E_1, \dots, E_L),$$

trong đó E_1 bắt đầu ở A , còn E_L kết thúc ở B . Lấy P là điểm cuối của E_{L_1} ; ta nói đường đi này “đi qua” P .

Ngược lại, nếu chọn một đường đi từ A tới P độ dài L_1 , rồi chọn một đường đi từ P tới B độ dài L_2 , ghép hai dãy cạnh theo thứ tự thì thu được một đường đi từ A tới B độ dài L . Vì vậy, với một đỉnh P cố định, số đường đi “đi qua” P là $f(P)g(P)$. Lấy tổng trên mọi P , ta được số đường đi cần tính.

Bổ đề 2. Thuật toán trong phần thứ nhất cho ra kết quả

$$\sum_P f(P)g(P).$$

Chứng minh. Theo hai lần DFS, với mọi đỉnh P , trong phần thứ nhất của thuật toán ta có $\text{Count}(P) = f(P)$; trong phần thứ hai, số lần tìm kiếm sau L_2 bước đạt tới P là $g(P)$.

Do đó, với các đỉnh có $g(P) = 0$, thuật toán không tăng đáp án, tương đương cộng thêm $f(P)g(P) = 0$. Với các đỉnh có $g(P) > 0$, thuật toán cộng $f(P)$ vào đáp án đúng $g(P)$ lần, tức cộng tổng cộng $f(P)g(P)$. Cộng theo mọi đỉnh P , kết quả của thuật toán là $\sum_P f(P)g(P)$.

Định lý 1. Thuật toán trong phần thứ nhất có thể tính đúng tổng số đường đi khác nhau từ đỉnh A tới đỉnh B có độ dài L trong đồ thị G .

Chứng minh. Kết hợp Bổ đề 1 và Bổ đề 2, đáp án thuật toán đưa ra và số đường đi thực tế đều bằng $\sum_P f(P)g(P)$.

9.3.2 Độ phức tạp thời gian

Dưới đây ta phân tích độ phức tạp thời gian của thuật toán. Ở trên ta đã giả sử bậc ra lớn nhất của một đỉnh trong G là D ; bây giờ giả sử thêm rằng bậc ra của các đỉnh trong G' cũng không vượt quá D . Mặt khác, giống mô tả trước đó, ta vẫn giả sử trong cả G và G' , không gian mô tả một đỉnh là M , và thời gian tính hậu kế xác định của một đỉnh là T .

Độ phức tạp thời gian của thuật toán chủ yếu phụ thuộc vào hai phần: thời gian DFS và thời gian chương trình tính, truy vấn $\text{Count}(P)$. Trong phân tích dưới đây, vì D là hằng số, ta có thể bỏ qua các dấu làm tròn trên số mũ, tức xem

$$O(D^{\lceil L/2 \rceil}) = O(D^{\lfloor L/2 \rfloor}) = O(D^{L/2}).$$

Trong phần thứ nhất, chi phí DFS là $O(D^{L/2}T)$; trong phần thứ hai, chi phí DFS cũng là $O(D^{L/2}T)$. Vì vậy tổng thời gian dùng cho DFS là

$$O(D^{L/2}T) + O(D^{L/2}T) = O(D^{L/2}T).$$

Trong phần thứ nhất, ta cần thống kê số lần $\text{Count}(P)$ của mỗi đỉnh P đạt được sau L_1 bước; trong phần thứ hai, ta cần truy vấn $\text{Count}(Q)$ với mỗi đỉnh Q đạt được sau L_2 bước. Điều này đòi hỏi một cấu trúc dữ liệu có thể chèn dữ liệu nhanh và truy vấn nhanh số lần xuất hiện của một dữ liệu. Có thể thấy bảng băm là lựa chọn lý tưởng. Theo tính chất của bảng băm, nếu dùng băm đều đơn giản (*simple uniform hashing*) và dùng danh sách liên kết để xử lý va chạm, thì trung bình một thao tác chèn hoặc truy vấn chỉ cần $O(M)$ thời gian. Nguyên nhân là trong bài toán này, so sánh hai đỉnh có giống nhau hay không cần $O(M)$ thời gian.

Trong phần thứ nhất, ta cần thực hiện $O(D^{L/2})$ thao tác chèn vào bảng băm; trong phần thứ hai, ta cần thực hiện $O(D^{L/2})$ thao tác truy vấn. Vì vậy thời gian tính và truy vấn $\text{Count}(P)$ là

$$[O(D^{L/2}) + O(D^{L/2})] \cdot O(M) = O(D^{L/2}M).$$

Kết hợp hai phần, độ phức tạp thời gian của thuật toán là

$$O(D^{L/2}(M + T)).$$

9.3.3 Điều kiện sử dụng thuật toán

Muốn dùng thuật toán trong phần thứ nhất cho một bài toán, trước hết bài toán đó phải phù hợp với mô hình đồ thị có hướng. Tiếp theo, trong quá trình phân tích độ phức tạp thời gian của thuật toán, thực ra ta đã thêm một số điều kiện hạn chế.

Trong phân tích trước đó, ta giả sử bậc ra của các đỉnh trong G' không vượt quá bậc ra lớn nhất trong G . Mặt khác, bậc ra của một đỉnh trong G' thực chất bằng bậc vào của đỉnh đó trong G . Vì vậy thông thường cần bảo đảm bậc vào và bậc ra của các đỉnh trong G cùng bậc độ lớn.

Mặt khác, ta giả sử có thể truy cập một hậu kế xác định của một đỉnh trong G' trong $O(T)$ thời gian. Điều kiện này tương đương với yêu cầu có thể truy cập một tiền nhiệm xác định của một đỉnh trong G trong $O(T)$ thời gian.

Về chi phí không gian, để bảo đảm bảng băm có độ phức tạp thao tác kỳ vọng $O(M)$, ta cần tiêu tốn

$$O(D^{L/2}M)$$

không gian.

9.4 Phần thứ ba: ứng dụng

9.4.1 Ví dụ 1: số nghiệm của phương trình

Nội dung bài toán. Cho các số nguyên $M, k_1, k_2, \dots, k_N$ và $p_1, p_2, \dots, p_N$. Hãy tính số nghiệm nguyên của phương trình

$$\sum_{i=1}^N k_i x_i^{p_i} = 0$$

thỏa mãn $1 \leq x_i \leq M$. Bài gốc là NOI 2001, vòng hai.

Quy mô dữ liệu.

$$N \leq 6, \quad M \leq 150.$$

Phân tích. Ta có thể chuyển bài này thành mô hình đồ thị có hướng, rồi dùng thuật toán trong phần thứ nhất để giải.

Dùng cặp (Depth, Sum) để biểu diễn một đỉnh trong G , nên mô tả một đỉnh chỉ cần $O(1)$ không gian. Ở đây Depth biểu diễn số hạng đã xử lý, còn Sum biểu diễn tổng của Depth số hạng đầu. Đỉnh xuất phát là $(0, 0)$, đỉnh kết thúc là $(N, 0)$.

Với một đỉnh (Depth, Sum) có $\text{Depth} < N$, đặt

$$k' = k_{\text{Depth}+1}, \quad p' = p_{\text{Depth}+1}.$$

Từ đỉnh này có M hậu kế:

$$\begin{aligned} &(\text{Depth}+1, \text{Sum} + k' \cdot 1^{p'}), \\ &(\text{Depth}+1, \text{Sum} + k' \cdot 2^{p'}), \\ &\dots, \\ &(\text{Depth}+1, \text{Sum} + k' \cdot M^{p'}). \end{aligned}$$

Nếu xét đồ thị đảo, thì với một đỉnh (Depth, Sum) có $\text{Depth} > 0$, nó có M tiền nhiệm. Đặt

$$k'' = k_{\text{Depth}}, \quad p'' = p_{\text{Depth}}.$$

Các tiền nhiệm là

$$\begin{aligned} &(\text{Depth}-1, \text{Sum} - k'' \cdot 1^{p''}), \\ &(\text{Depth}-1, \text{Sum} - k'' \cdot 2^{p''}), \\ &\dots, \\ &(\text{Depth}-1, \text{Sum} - k'' \cdot M^{p''}). \end{aligned}$$

Để tính một hậu kế hoặc tiền nhiệm của một đỉnh, ta cần tính một lũy thừa. Dùng lũy thừa nhanh thì độ phức tạp thời gian là $O(\log \text{MaxP})$, trong đó MaxP là giá trị lớn nhất trong $p_1, p_2, \dots, p_N$.

Qua phân tích trên, ta có thể dùng trực tiếp thuật toán trong phần thứ nhất, với độ phức tạp thời gian

$$O(M^{N/2} \log \text{MaxP}).$$

Hãy nhìn lại ý nghĩa thực tế của thuật toán trên trong ví dụ này. Xét trường hợp $N = 6$:

$$k_1x_1^{p_1} + k_2x_2^{p_2} + k_3x_3^{p_3} + k_4x_4^{p_4} + k_5x_5^{p_5} + k_6x_6^{p_6} = 0.$$

Thuật toán thô là liệt kê sáu ẩn ở vế trái rồi kiểm tra kết quả của vế trái có bằng hằng số ở vế phải hay không; độ phức tạp thời gian như vậy là $O(M^6 \log \text{MaxP})$, rất kém.

Suy nghĩ kỹ hơn, phương trình trên có vẻ rất mất cân bằng: cả sáu ẩn đều nằm ở vế trái, khiến ta buộc phải liệt kê sáu ẩn. Chuyển về phương trình, ta có

$$k_1x_1^{p_1} + k_2x_2^{p_2} + k_3x_3^{p_3} = -k_4x_4^{p_4} - k_5x_5^{p_5} - k_6x_6^{p_6}.$$

Lúc này hai vế đã “cân bằng”. Giả sử ta liệt kê trước ba số ở vế trái và lưu mọi kết quả liệt kê được vào một bảng, rồi liệt kê ba số ở vế phải. Mỗi khi tính được một kết quả trùng với một số trong bảng, ta thu được một nghiệm của phương trình. Khi so sánh kết quả tính toán, ta không cần so sánh từng cặp số; có thể dùng cấu trúc dữ liệu, chẳng hạn bảng băm, để tăng tốc việc thống kê. Từ đó ta có thuật toán độ phức tạp $O(M^3 \log \text{MaxP})$. Thực chất, điểm then chốt của *meet in the middle* nằm ở việc giảm lượng tính toán bằng cách tối ưu hóa quá trình hợp nhất.

9.4.2 Ví dụ 2: *ABCDEF*

Nội dung bài toán. Cho một tập số nguyên S . Hãy tính có bao nhiêu bộ (a, b, c, d, e, f) thỏa mãn $a, b, c, d, e, f \in S$, $d \neq 0$, và

$$ab + c = d(e + f).$$

Bài gốc là *ABCDEF* trên SPOJ.

Quy mô dữ liệu.

$$|S| \leq 100.$$

Phân tích. Bài này chỉ cần chuyển về phương trình thành

$$ab + c = d(e + f),$$

vì vậy về bản chất nó giống ví dụ 1. Ta thu được thuật toán có độ phức tạp

$$O(|S|^3).$$

9.4.3 Ví dụ 3: *EllysBulls*

Nội dung bài toán. Có một số thập phân N chữ số A , có thể có chữ số 0 ở đầu. Bây giờ cho M điều kiện; điều kiện thứ i gồm một số N chữ số X_i và giá trị $\text{Same}(A, X_i)$. Ở đây $\text{Same}(A, B)$ biểu diễn số vị trí mà hai số A và B có chữ số giống nhau. Ví dụ,

$$\text{Same}(\text{"0312"}, \text{"0321"}) = 2.$$

Nếu có duy nhất một A thỏa mãn yêu cầu thì in nghiệm đó; ngược lại in thông tin biểu thị vô nghiệm hoặc nhiều nghiệm. Bài gốc là TopCoder Single Round Match 572, Division I, Level Two.

Quy mô dữ liệu.

$$N \leq 9, \quad M \leq 50.$$

Phân tích. Để tiện trình bày, định nghĩa vector hàng

$$\text{Sum} = (\text{Same}(A, X_1), \text{Same}(A, X_2), \dots, \text{Same}(A, X_M)).$$

Gọi $X_{i,j}$ là chữ số thứ j của X_i . Định nghĩa hàm $S(a, b)$ với $0 \leq a, b \leq 9$: nếu $a = b$ thì $S(a, b) = 1$, nếu $a \neq b$ thì $S(a, b) = 0$. Tiếp tục định nghĩa

$$\text{Cost}(i, j) = (S(X_{1,i}, j), S(X_{2,i}, j), \dots, S(X_{M,i}, j)).$$

Vector này cho biết: nếu chữ số thứ i của A là j , thì riêng vị trí này khớp với những điều kiện nào.

Theo điều kiện của đề bài, ta có phương trình

$$\text{Cost}(1, a_1) + \text{Cost}(2, a_2) + \dots + \text{Cost}(N, a_N) = \text{Sum}.$$

Ta vẫn chuyển về giống hai bài trước. Đặt

$$\text{Mid} = \left\lfloor \frac{N}{2} \right\rfloor,$$

thu được

$$\begin{aligned} & \text{Cost}(1, a_1) + \dots + \text{Cost}(\text{Mid}, a_{\text{Mid}}) \\ &= \text{Sum} - \text{Cost}(\text{Mid} + 1, a_{\text{Mid} + 1}) - \dots - \text{Cost}(N, a_N). \end{aligned}$$

Vì mỗi vị trí có 10 khả năng, nên tìm kiếm về trái có $O(10^{N/2})$ kết quả khác nhau, và về phải cũng có $O(10^{N/2})$ kết quả khác nhau. Hai vế của phương trình đều là vector hàng $1 \times M$, nên mỗi bước tính toán và thao tác trong bảng băm cần $O(M)$ thời gian. Vì vậy ta có thuật toán độ phức tạp $O(10^{N/2}M)$, đủ để vượt qua dữ liệu kiểm thử của bài này.

9.4.4 Tiểu kết: mô hình phương trình

Tổng kết ba ví dụ trên, ta thấy các bài toán đều có thể chuyển thành một mô hình phương trình:

$$\sum_{i=1}^N f(i, x_i) = S.$$

Ở ví dụ 1 và 2, $f(i, x_i)$ và S là số nguyên; ở ví dụ 3, $f(i, x_i)$ và S là vector. Trong cả ba bài, cách làm đều dựa trên việc biến đổi phương trình thành

$$\sum_{i=1}^{\lfloor N/2 \rfloor} f(i, x_i) = S - \sum_{i=\lfloor N/2 \rfloor + 1}^N f(i, x_i).$$

Ta tìm kiếm về trái trước và ghi kết quả vào bảng băm; sau đó tìm kiếm về phải và truy vấn các kết quả đã lưu trong bảng băm. Có thể thấy quy trình giải ngắn gọn này rất giống thuật toán trong phần thứ nhất. Hơn nữa, từ phân tích của ví dụ 1, mô hình phương trình có thể chuyển thành mô hình đồ thị có hướng trong phần thứ nhất, nên mô hình phương trình là một trường hợp đặc biệt của mô hình đồ thị có hướng. Với các bài thuộc mô hình phương trình, ta dễ nghĩ tới kỹ thuật *meet in the middle*.

Vận dụng hai mô hình trên, ta có thể nhanh chóng mô hình hóa bài toán cụ thể và dùng kỹ thuật *meet in the middle* để giải. Tuy nhiên, một số bài không thể chuyển thành mô hình phương trình hoặc mô hình đồ thị có hướng, do đó không thể dùng trực tiếp thuật toán của phần thứ nhất. Dù vậy, chúng vẫn có điểm tương tự với hai mô hình trên; ta vẫn có thể sửa thuật toán trong phần thứ nhất để dùng kỹ thuật *meet in the middle* tối ưu các bài này. Dưới đây ta phân tích vài ví dụ như vậy.

9.4.5 Ví dụ 4: *Balanced Cow Subsets*

Nội dung bài toán. Cho các số nguyên dương $a_1, \dots, a_N$. Trước hết chọn ra một số số trong N số, ít nhất một số; sau đó chia các số đã chọn thành hai phần sao cho tổng của hai phần bằng nhau. Hãy tính số phương án chọn số ở bước đầu.

Lưu ý: có thể tồn tại một phương án chọn số cho phép nhiều cách chia khác nhau, nhưng phương án chọn số đó chỉ được tính một lần. Bài gốc là USACO 2012 US Open, Gold Division.

Quy mô dữ liệu.

$$N \leq 20.$$

Phân tích. Ta đổi bài toán sang một cách mô tả tương đương: với dãy $a_1, a_2, \dots, a_N$, đặt $x_i \in \{-1, 0, 1\}$. Dưới điều kiện

$$\sum_{i=1}^N x_i a_i = 0,$$

cần tính có bao nhiêu tập khác nhau, không rỗng,

$$S = \{i \mid 1 \leq i \leq N, x_i \neq 0\}.$$

Bài này giống mô hình phương trình đã nêu, nhưng điều ta cần không phải số nghiệm của phương trình, mà là số tập tương ứng với các nghiệm của phương trình. Vì vậy thuật toán cần sửa đôi chút.

Định nghĩa một dãy mới $y_1, y_2, \dots, y_N$: nếu $x_i = 0$ thì $y_i = 0$, còn nếu $x_i \neq 0$ thì $y_i = 2^{i-1}$. Khi đó tập S có thể biểu diễn bằng $\sum y_i$.

Chọn một số nguyên M trong đoạn $[0, N]$, biến đổi $\sum x_i a_i = 0$ thành

$$x_1 a_1 + x_2 a_2 + \dots + x_M a_M = -x_{M+1} a_{M+1} - x_{M+2} a_{M+2} - \dots - x_N a_N.$$

Ký hiệu vế trái là LeftSum, vế phải là RightSum. Tương tự, ký hiệu

$$\text{LeftSet} = y_1 + y_2 + \dots + y_M, \quad \text{RightSet} = y_{M+1} + y_{M+2} + \dots + y_N.$$

Trong phần thứ nhất của thuật toán, ta vẫn tìm kiếm M ở vế trái như các bài trước, tổng cộng có 3^M trường hợp. Ta không chỉ ghi LeftSum, mà còn ghi cả LeftSet tương ứng.

Trong phần thứ hai của thuật toán, nếu phát hiện RightSum bằng một LeftSum nào đó, thì tập được biểu diễn bởi

$$\text{LeftSet} + \text{RightSet}$$

là một tập hợp hợp lệ. Ta có thể dùng một mảng boolean đánh số từ 0 tới $2^N - 1$ để ghi nhận các tập này.

Cuối cùng, quét toàn bộ mảng boolean một lần là thống kê được đáp án.

Dễ thấy phần thứ nhất có độ phức tạp thời gian $O(3^M)$. Phần thứ hai liệt kê $O(3^{N-M})$ trường hợp, nhưng với mỗi trường hợp cần truy cập tất cả LeftSet tương ứng. Trong trường hợp xấu nhất, ta cần truy cập $O(2^M)$ LeftSet, nên độ phức tạp thời gian của phần thứ hai là $O(3^{N-M} 2^M)$. Thời gian quét cuối cùng là $O(2^N)$.

Tóm lại, độ phức tạp thời gian là

$$O(3^M + 3^{N-M} 2^M + 2^N) = O(3^M + 3^{N-M} 2^M).$$

Nếu chọn $M = \lfloor N/2 \rfloor$, ta thu được

$$O(6^{N/2}) \approx O(2.45^N),$$

đủ để vượt qua dữ liệu kiểm thử của bài này. Nhưng thực tế, nếu chọn

$$M = N \log_{9/2} 3,$$

ta có được độ phức tạp tốt hơn:

$$O((3^{\log_{9/2} 3})^N) \approx O(2.23^N).$$

Ví dụ 4 cho thấy khi dùng kỹ thuật *meet in the middle*, ta có thể lưu thêm một số thông tin khác trong bảng băm để thuận tiện cho việc giải bài. Mặt khác, khi dùng kỹ thuật này, nếu chia số ẩn cần tìm kiểm thành hai phần không đều nhau, đôi khi ta có thể thu được độ phức tạp thời gian tốt hơn.

9.4.6 Ví dụ 5: *AlphabetPaths*

Nội dung bài toán. Cho một ma trận $R \times C$. Mỗi ô của ma trận hoặc là ô trống, hoặc chứa một số nguyên từ 0 tới 20. Hãy tính có bao nhiêu đường đi gồm 21 ô thỏa mãn: hai ô kề nhau trên đường đi phải có một cạnh chung, và mỗi số từ 0 tới 20 xuất hiện đúng một lần trên đường đi. Bài gốc là TopCoder Single Round Match 523, Division I, Level Three.

Quy mô dữ liệu.

$$R, C \leq 21.$$

Phân tích. Bài này nhìn qua không phải mô hình phương trình đã nêu, mà giống mô hình đồ thị có hướng trong phần thứ nhất. Tuy nhiên, trong bài này điểm đầu và điểm cuối của đường đi đều không xác định, mỗi loại có $O(RC)$ cách chọn. Nếu liệt kê cả điểm đầu và điểm cuối thì ta đã mất $O(R^2C^2)$ lần liệt kê; nhân thêm thời gian tìm kiếm thì hoàn toàn không thể vượt qua.

Ý tưởng vừa nêu là liệt kê điểm đầu và điểm cuối, rồi tìm từ hai đầu đường đi vào giữa. Ta nhận thấy lựa chọn “điểm giữa” thật ra cũng chỉ có $O(RC)$ cách. Vậy tại sao không liệt kê điểm giữa, rồi tìm kiếm từ giữa ra hai đầu?

Do đó ta liệt kê điểm giữa của đường đi, gọi là Mid, tức ô thứ 11. Từ Mid tìm kiếm 10 bước, ta có thể thu được một đường đi P gồm 11 ô. Gọi $\text{Num}(P)$ là tập các giá trị của 10 ô không tính Mid. Gọi giá trị trong Mid là X . Nếu tồn tại hai đường đi P_1 và P_2 thỏa mãn

$$\text{Num}(P_1) \cup \text{Num}(P_2) \cup \{X\} = \{0, 1, \dots, 20\},$$

thì P_1 và P_2 ghép lại là một đường đi thỏa mãn yêu cầu. Chú ý rằng vì các số 0 tới 20 đều xuất hiện đúng một lần, nên có thể khẳng định P_1 và P_2 chỉ giao nhau tại Mid. Giống bài trước, ta dùng một số nhị phân để biểu diễn tập $\text{Num}(P)$, nhờ đó có thể nhanh chóng kiểm tra

$$\text{Num}(P_1) \cup \text{Num}(P_2) \cup \{X\} = \{0, 1, \dots, 20\}.$$

Ước lượng lượng tính toán của thuật toán trên. Trước hết số cách liệt kê Mid là $O(RC)$. Tiếp theo, vì tìm kiếm 10 bước và mỗi bước có thể đi theo bốn hướng, số đường đi là 4^{10} . Thực ra ước lượng này có thể chính xác hơn: từ bước tìm kiếm thứ hai trở đi, “đi ngược lại” hiển nhiên không hợp lệ, nên từ bước thứ hai chỉ còn ba hướng có thể đi. Vì vậy số đường đi thực tế không vượt quá $4 \cdot 3^9$. Với mỗi đường đi, thao tác bảng băm và so sánh đều chỉ cần thời gian hằng số. Tóm lại, ta thu được thuật toán có độ phức tạp

$$O(RC \cdot 4 \cdot 3^9).$$

Dù $RC \cdot 4 \cdot 3^9$ có thể đạt

$$21^2 \cdot 4 \cdot 3^9 = 34720812,$$

nhưng do đường đi không được vượt khỏi biên ma trận hoặc đi tới ô không có số, và nếu một số xuất hiện hai lần trên đường đi thì có thể cắt nhánh ngay, lượng tìm kiếm thực tế sẽ nhỏ hơn rất nhiều so với ước lượng trên. Trong kiểm thử thực tế, chương trình cài đặt theo thuật toán trên vượt qua toàn bộ dữ liệu kiểm thử.

Ví dụ 5 cho thấy khi dùng kỹ thuật *meet in the middle*, nếu điểm đầu và điểm cuối chưa biết, đồng thời số điểm giữa khá ít, ta có thể thử đổi “gặp nhau ở giữa” thành “xuất phát từ giữa”.

9.4.7 Ví dụ 6: *FencingGarden*

Nội dung bài toán. Có N thanh gỗ, độ dài lần lượt là $L_1, L_2, \dots, L_N$. Bạn có thể chọn trước một thanh gỗ và cắt nó thành hai đoạn, độ dài không nhất thiết là số nguyên. Từ các thanh gỗ thu được, chọn một phần để tựa vào một bức tường dài vô hạn và rào một vùng hình chữ nhật. Hãy tìm độ dài cạnh song song với tường của hình chữ nhật khi diện tích vùng hình chữ nhật đạt giá trị lớn nhất. Bài gốc là TopCoder Single Round Match 461, Division I, Level Three.

Quy mô dữ liệu.

$$N \leq 40, \quad L_i \leq 10^8.$$

Phân tích. Bài này khác mọi ví dụ trước: nó không yêu cầu đếm số phương án, mà yêu cầu một nghiệm tối ưu.

Một khó khăn của bài là có rất nhiều cách cắt thanh gỗ; liệt kê cách cắt hiển nhiên là không thực tế.

Vì ta cắt một thanh gỗ thành hai đoạn, còn hình chữ nhật được rào bởi ba cạnh, chắc chắn có một cạnh hoàn toàn được ghép từ các thanh gỗ nguyên vẹn. Cạnh này có hai khả năng: song song với tường hoặc vuông góc với tường. Gọi độ dài cạnh đó là Len .

Trước hết xét trường hợp cạnh này song song với tường. Đặt

$$S = \sum_{i=1}^N L_i.$$

Khi đó hai cạnh còn lại của hình chữ nhật phải có độ dài bằng nhau. Ta xếp các thanh gỗ chưa dùng lại với nhau; tổng độ dài của chúng là $S - Len$. Cắt một nhát tại vị trí $(S - Len)/2$. Nếu vị trí này đúng là chỗ nối của hai thanh gỗ, ta không làm thay đổi số lượng thanh gỗ; nếu đây là một điểm ở giữa một thanh gỗ, ta cắt thanh đó thành hai đoạn. Dù thế nào, ta luôn có thể chia các thanh gỗ còn lại thành hai phần có độ dài đều là $(S - Len)/2$, nhờ đó thu được hình chữ nhật diện tích

$$Len \cdot \frac{S - Len}{2}.$$

Tương tự, nếu một cạnh độ dài Len vuông góc với tường và hoàn toàn được ghép từ các thanh gỗ nguyên vẹn, ta có thể thu được một hình chữ nhật diện tích

$$Len(S - 2Len).$$

Theo tính chất của hàm bậc hai, trong trường hợp thứ nhất $Len = S/2$ là tối ưu; trong trường hợp thứ hai $Len = S/4$ là tối ưu. Vì vậy ta cần tìm, trong các độ dài có thể ghép từ các thanh gỗ nguyên vẹn, những độ dài gần $S/2$ và $S/4$ nhất.

Giống các bài trước, ta vẫn chia N thanh gỗ thành hai phần, lần lượt có M thanh và $N - M$ thanh. Ta tìm kiếm trước M thanh và ghi lại các độ dài thu được. Sau đó tìm kiếm $N - M$ thanh còn lại; giả sử thu được một phần có độ dài T , thì cần tìm trong các độ dài đã tìm trước đó một x sao cho $x + T$ gần $S/2$

hoặc $S/4$ nhất, tức làm cho x gần $S/2 - T$ hoặc $S/4 - T$ nhất. Một cách xử lý tốt là sắp xếp trước $O(2^M)$ độ dài thu được ở phần thứ nhất; khi truy vấn ở phần thứ hai, dùng hai lần tìm kiếm nhị phân.

Sắp xếp $O(2^M)$ số có độ phức tạp thời gian $O(2^M M)$. Thực hiện $O(2^{N-M})$ lần tìm kiếm nhị phân trong đó có độ phức tạp $O(2^{N-M} M)$. Nếu chọn $M = \lfloor N/2 \rfloor$, độ phức tạp thời gian của thuật toán là

$$O(2^{N/2} N).$$

Ví dụ 6 cho thấy kỹ thuật *meet in the middle* không chỉ giải được các bài toán đếm, mà còn có thể kết hợp với tìm kiếm nhị phân và các phương pháp khác để xử lý một số bài tối ưu đặc biệt.

9.5 Tổng kết

Từ các phần trên có thể thấy, *meet in the middle* là một kỹ thuật tối ưu hóa tìm kiếm rất hiệu quả. Tư tưởng cốt lõi của nó là chia bài toán thành hai phần để tìm kiếm riêng, rồi nhanh chóng hợp nhất kết quả của hai phần tìm kiếm nhằm tối ưu thuật toán. Tuy ta không nhờ đó thu được một thuật toán đa thức, nhưng thường có thể làm số mũ trong độ phức tạp thời gian giảm một nửa; trong một số bài, điều này đã đủ để ta tính được kết quả trong giới hạn thời gian.

Ưu điểm của kỹ thuật này là thuật toán dễ cài đặt; quá trình tìm kiếm mà thuật toán dùng về cơ bản giống thuật toán thô, chỉ cần thêm một số cấu trúc dữ liệu khá đơn giản, chẳng hạn bảng băm. Điểm yếu là phạm vi sử dụng không quá rộng, và độ phức tạp thời gian sau tối ưu vẫn là cấp số mũ, nên quy mô dữ liệu có thể xử lý hiệu quả không tăng quá nhiều. Đồng thời, thuật toán phải tiêu tốn rất nhiều không gian.

Mô hình đồ thị có hướng và mô hình phương trình khái quát các đặc trưng phổ biến của những bài mà kỹ thuật *meet in the middle* có thể giải; dùng hai mô hình này có thể giải được một lớp bài toán. Mặt khác, vẫn có những bài không thể chuyển trực tiếp thành các mô hình trên, đòi hỏi ta sửa kỹ thuật *meet in the middle*. Từ đó có thể thấy cùng một kỹ thuật cũng có nhiều cách sử dụng khác nhau trong các bài khác nhau. Ta cần linh hoạt dùng kỹ thuật *meet in the middle* theo tình huống cụ thể để đạt mục đích tối ưu thuật toán.

9.6 Lời cảm ơn

Xin cảm ơn người thân và bạn bè đã động viên, ủng hộ tôi.

Xin cảm ơn các thầy cô Li Shu, Wu Xiaoshi và Shi Zhenlei đã hướng dẫn tôi.

Xin cảm ơn Jia Zhipeng, Xu Haoran và nhiều bạn học thi tin học khác đã giúp đỡ và gợi mở cho tôi.

9.7 Tài liệu tham khảo

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. *Introduction to Algorithms*.
2. vexorian. TopCoder Algorithm Problem Set Analysis.
3. Bruce Merry. USACO 2012 US Open Gold Division, Balanced Cow Subsets Solution.
4. Zhipeng Jia. TopCoder SRM 450–499 Solution.

10 Phân tích cơ sở và ứng dụng của xác suất trong thi tin học

Hu Yuanming

Trường Trung học Dương Châu, Giang Tô

Tóm tắt

Cùng với việc nội dung thi tin học ngày càng phong phú, những bài toán liên quan tới xác suất liên tục xuất hiện. Các thí sinh có trình độ nhất định đều có năng lực giải những bài xác suất ở mức khá, nhưng khi bị hỏi tới chứng minh hơi chặt chẽ của lời giải, không ít người lại lúng túng.

Bài viết này dựa trên trải nghiệm của tác giả khi học xác suất sơ cấp, cũng như những điểm khó về xác suất mà tác giả nhận thấy trong quá trình trao đổi với các thí sinh khác. Nội dung chính gồm các khái niệm cơ bản của xác suất, biến ngẫu nhiên và kỳ vọng, các vấn đề liên quan trên mạng chuyển tiếp xác suất, bất đẳng thức Markov và bất đẳng thức Chebyshev. Từ hai tầng lý thuyết và thực hành gắn bó chặt chẽ, bài viết giúp người đọc làm rõ mạch tư duy của xác suất và hiểu sâu hơn các lời giải có dùng xác suất.

10.1 Xác suất

10.1.1 Xác suất là gì

Nếu từng suy nghĩ kỹ về câu hỏi này, người đọc có thể thấy nó không dễ trả lời. Nói một cách trực quan, việc có xác suất lớn thì có khả năng xảy ra lớn; do đó xác suất là một thước đo khả năng xảy ra của sự việc.

Hệ tiên đề hóa của xác suất sẽ liên quan tới nhiều nội dung của lý thuyết độ đo; người đọc quan tâm có thể tra cứu sách chuyên khảo. Vì bài viết này chỉ thảo luận xác suất từ góc nhìn thi tin học, ta thường không gặp các tập hợp quá đặc biệt, chẳng hạn hợp của đếm được nhiều tập, cũng không gặp một số bài ngẫu nhiên liên tục. Bởi vậy lý thuyết xác suất dùng trong thi tin học có thể được giản lược đáng kể. Tuy nhiên, tác giả vẫn khuyên mọi người khi có thời gian nên tìm hiểu hệ tiên đề của xác suất; điều đó sẽ giúp ích rất nhiều cho việc hiểu xác suất.

10.1.2 Không gian xác suất

Xác suất sơ cấp dùng trong thi đấu có ba thành phần quan trọng: không gian mẫu Ω , tập các biến cố F , và độ đo xác suất P . Cái ta thường gọi là biến cố thực ra là một tập con nào đó của Ω ; người đọc nên luôn ghi nhớ điều này. Trong thi đấu, ta thường có thể coi mọi tập con của Ω đều là một biến cố, dù ở một số lĩnh vực ngoài phạm vi thi đấu, cách làm này có thể gây vấn đề. Tập hợp của mọi biến cố được ký hiệu là F ; chú ý đây là một tập hợp các tập hợp. Độ đo xác suất P là một hàm từ tập các biến cố tới tập số thực. Dĩ nhiên, không phải độ đo xác suất nào cũng hợp lý. Một độ đo xác suất hợp lý cần thỏa mãn ba tiên đề:

1. Với mọi biến cố A , $P(A) \geq 0$ (tính không âm).
2. $P(\Omega) = 1$ (tính chuẩn hóa).
3. Với hai biến cố A và B , nếu $A \cap B = \emptyset$ thì $P(A \cup B) = P(A) + P(B)$ (tính cộng được).

Ta gọi bộ ba (Ω, F, P) thỏa mãn các điều kiện trên là một không gian xác suất. Ví dụ, khi gieo ngẫu nhiên một con xúc xắc đều và xét mặt hướng lên, không gian mẫu là

$$\Omega = \{1, 2, 3, 4, 5, 6\}.$$

Biến cố “mặt hướng lên là số lẻ” là $\{1, 3, 5\}$. Tập các biến cố F là tập tất cả các tập con của Ω , và độ đo xác suất là

$$P(A) = \frac{|A|}{6}.$$

Dễ kiểm tra đây là một không gian xác suất hợp lý.

Khái niệm này hẳn khá quen thuộc với mọi người và thường được xem là hiển nhiên. Ở đây nó được nhắc lại chỉ vì một số độc giả có thể gặp khó khăn với nền tảng xác suất khi đọc các phần sau.

10.1.3 Xác suất có điều kiện

Xác suất có điều kiện là một khái niệm rất thường gặp trong OI, nhưng chỉ cần sơ suất một chút là có thể dẫn tới sai lầm. Khái niệm này không khó giải thích; hãy xét một ví dụ. Có hai đại học α và β . Trong đại học α , 99% sinh viên là nam; trong đại học β , 99% sinh viên là nữ. Giả sử hai trường có số sinh viên bằng nhau. Nếu chọn ngẫu nhiên một sinh viên từ hai trường, xác suất sinh viên đó là nam bằng bao nhiêu?

Đây rõ ràng là một bài thuộc mô hình cổ điển. Ta dễ thấy nam sinh chiếm 50% toàn bộ sinh viên, nên xác suất chọn được nam sinh là 50%.

Nếu sau khi nói chuyện với sinh viên được chọn, ta biết người đó thuộc đại học α , thì xác suất này rõ ràng trở thành 99%. Từ đó có thể thấy rằng sau khi có thêm thông tin, xác suất của một biến cố có thể thay đổi.

Ký hiệu xác suất biến cố A xảy ra trong điều kiện đã biết biến cố B xảy ra là $P(A | B)$. Gọi biến cố sinh viên được chọn thuộc đại học α là U_1 , thuộc đại học β là U_2 ; gọi biến cố sinh viên được chọn là nam là S_1 , là nữ là S_2 . Trong ví dụ trên, ta có thể viết

$$P(S_1 | U_1) = 99\%.$$

Chú ý rằng nhiều người mới học khi hiểu vấn đề này thường bị ràng buộc bởi cảm giác “trước sau”, cho rằng biến cố B nhất định xảy ra trước biến cố A . Điều đó không thỏa đáng trong một số tình huống. Biến cố chỉ là một tập con của không gian mẫu, bản thân nó không có thuộc tính thời gian; trong ví dụ này việc chọn sinh viên được hoàn thành một lần. Dĩ nhiên, trong một số trường hợp, hai biến cố thật sự có quan hệ thời gian rõ ràng, chẳng hạn xác suất thi đỗ sau khi ôn tập và xác suất thi đỗ khi không ôn tập hiển nhiên là khác nhau.

Công thức tính xác suất có điều kiện là

$$P(A | B) = \frac{P(AB)}{P(B)}.$$

Khi nghiên cứu các bài xác suất, ta thường dùng AB , hoặc “ A, B ”, để biểu diễn $A \cap B$. Vì vậy $P(A \cap B)$, $P(AB)$, $P(A, B)$ có cùng ý nghĩa.

Thực ra khi xét xác suất có điều kiện, ta đang coi biến cố B là không gian mẫu mới. Vì biến cố và không gian mẫu đều là tập hợp, và bản thân không gian mẫu cũng là một biến cố, cách làm này không có vấn đề. Độ đo xác suất mới thường được gọi là xác suất có điều kiện; công thức xác suất có điều kiện về bản chất cho thấy quan hệ giữa các độ đo xác suất trên hai không gian mẫu.

10.1.4 Công thức xác suất toàn phần

Công thức xác suất toàn phần có vai trò rất quan trọng trong các bài xác suất, nhưng nhiều thí sinh chưa hiểu sâu ý nghĩa của nó. Giả sử $B_1, B_2, B_3, \dots, B_n$ là một phân hoạch của không gian xác suất. Khi đó

$$P(A) = \sum_k P(A | B_k)P(B_k).$$

Công thức này là nền tảng khi dùng phân loại để nghiên cứu bài toán xác suất; gần như mọi bài xác suất đều liên quan tới nó. Khi vừa kết luận rằng nam sinh chiếm 50% toàn bộ sinh viên, thực ra ta đã dùng công thức xác suất toàn phần:

$$P(S_1) = P(S_1 | U_1)P(U_1) + P(S_1 | U_2)P(U_2) = 99\% \cdot 50\% + 1\% \cdot 50\% = 50\%.$$

10.1.5 Công thức Bayes

Xét đẳng thức

$$P(A | B)P(B) = P(AB) = P(B | A)P(A).$$

Bỏ qua hạng giữa và đồng thời chia hai vế cho $P(B)$, ta được công thức Bayes:

$$P(A | B) = \frac{P(B | A)P(A)}{P(B)}.$$

Công thức này rất hữu ích khi tính xác suất ngược.

Vì thông tin ta biết thường là một dãy xác suất có điều kiện $P(B | A_k)$, trong đó $A_1, A_2, \dots, A_n$ là một phân hoạch của không gian mẫu, nên ở tình huống này ta cần kết hợp công thức trên với công thức xác suất toàn phần để có

$$P(A_k | B) = \frac{P(B | A_k)P(A_k)}{\sum_j P(B | A_j)P(A_j)}.$$

Quay lại ví dụ trên, bây giờ câu hỏi đổi thành: biết sinh viên được chọn là nam, có thể tính xác suất người đó là sinh viên đại học α không?

Đại lượng cần tính là $P(U_1 | S_1)$. Dùng công thức Bayes, ta có

$$P(U_1 | S_1) = \frac{P(S_1 | U_1)P(U_1)}{P(S_1 | U_1)P(U_1) + P(S_1 | U_2)P(U_2)} = \frac{99\% \cdot 50\%}{99\% \cdot 50\% + 1\% \cdot 50\%} = 99\%.$$

10.2 Biến ngẫu nhiên và kỳ vọng

Kỳ vọng là một khái niệm thường xuất hiện trong các bài xác suất. Nó là một thuộc tính quan trọng của biến ngẫu nhiên, nhưng điểm này thường bị thí sinh bỏ qua, khiến kỳ vọng trở thành một tòa lâu đài thiếu nền móng là biến ngẫu nhiên. Muốn hiểu kỳ vọng rốt cuộc là gì, trước hết phải hiểu biến ngẫu nhiên.

10.2.1 Định nghĩa biến ngẫu nhiên

Biến ngẫu nhiên (*random variable*) là một tên gọi thú vị, vì thật ra nó không phải nguồn gốc của tính ngẫu nhiên (tính ngẫu nhiên đến từ không gian mẫu), cũng không phải là biến, mà là một hàm thực xác định trên không gian mẫu Ω . Tuy nhiên trong rất nhiều trường hợp, ta thường bỏ qua không gian mẫu và trực tiếp xét thuộc tính của biến ngẫu nhiên, chẳng hạn kỳ vọng, đến mức nhiều thí sinh chỉ biết khái niệm kỳ vọng mà không hiểu rõ biến ngẫu nhiên là gì. Ở đây cần đưa ra một định nghĩa rõ ràng:

$$X : \Omega \rightarrow \mathbb{R}$$

được gọi là một biến ngẫu nhiên.

Trong đa số trường hợp, sau khi có biến ngẫu nhiên, ta có thể bỏ qua việc quan tâm trực tiếp tới không gian mẫu ban đầu, mà tập trung vào xác suất biến ngẫu nhiên nhận từng giá trị thực. Chú ý rằng bài viết này chỉ xét trường hợp rời rạc. Quá trình này thực chất là chia lại không gian mẫu: các phần tử trong không gian mẫu cùng cho ra một giá trị thực dưới hàm này được gộp lại. Dĩ nhiên, trước đây khi hiểu biến ngẫu nhiên, người đọc rất có thể đã bắt đầu trực tiếp từ xác suất nó nhận từng giá trị.

10.2.2 Kỳ vọng của biến ngẫu nhiên

Kỳ vọng (*expectation*) là một cách khắc họa tình huống trung bình mà biến ngẫu nhiên thể hiện, đồng thời là khái niệm xuất hiện thường xuyên nhất trong thi đấu. Nhiều thí sinh học khái niệm này khi tách

rời khỏi biến ngẫu nhiên; đó là cách học không nên làm. Với một biến ngẫu nhiên, định nghĩa kỳ vọng của nó là

$$E[X] = \sum_{\omega} P(\omega)X(\omega) = \sum_x xP(X=x).$$

Ở đây $X=x$ biểu diễn một biến cố, tương đương với tập

$$\{\omega \mid \omega \in \Omega, X(\omega) = x\}.$$

Công thức đầu là định nghĩa từ góc nhìn đầu vào, tức không gian mẫu; công thức sau bắt đầu từ các đầu ra khác nhau, chia không gian mẫu theo những đầu ra này rồi lấy tổng có trọng số xác suất của các giá trị đầu ra.

Vì với một số phần tử của không gian mẫu, giá trị đầu ra của biến ngẫu nhiên có thể giống nhau, đôi khi ta không cần xét biến ngẫu nhiên từ góc độ không gian mẫu, mà xét trực tiếp biến cố “biến ngẫu nhiên nhận một giá trị cụ thể”. Tuy nhiên vẫn cần chú ý rằng không gian mẫu và độ đo xác suất là nền tảng của biến ngẫu nhiên. Nếu phát hiện bài toán không thể giải được từ tầng cao là kỳ vọng, ta cần quay lại tầng thấp hơn. Các chứng minh tính chất kỳ vọng phía sau đều bắt đầu từ không gian xác suất; nếu không có không gian xác suất, ta cũng không có cách chứng minh tính độc lập của hai biến ngẫu nhiên.

10.2.3 Tính độc lập của biến ngẫu nhiên và kỳ vọng của tích

Tính độc lập của biến ngẫu nhiên là thuộc tính của các biến cố ngẫu nhiên ở tầng đầu ra. Với hai biến ngẫu nhiên X_1, X_2 và hai số thực $x_1 \in X_1(\Omega), x_2 \in X_2(\Omega)$, nếu

$$P(X_1 = x_1, X_2 = x_2) = P(X_1 = x_1)P(X_2 = x_2),$$

thì ta nói X_1, X_2 độc lập với nhau.

Một tính chất quan trọng của hai biến ngẫu nhiên độc lập là kỳ vọng của tích bằng tích của kỳ vọng:

$$\begin{aligned} E[X_1 X_2] &= \sum_{x \in (X_1 X_2)(\Omega)} xP(X_1 X_2 = x) \\ &= \sum_{x \in (X_1 X_2)(\Omega)} xP(X_1 = x_1)P(X_2 = x_2) \\ &= \sum_{x \in (X_1 X_2)(\Omega)} \sum_{x_1 \in X_1(\Omega)} xP(X_1 = x_1)P\left(X_2 = \frac{x}{x_1}\right) \\ &= \sum_{x \in (X_1 X_2)(\Omega)} \sum_{x_1 \in X_1(\Omega)} x_1 \frac{x}{x_1} P(X_1 = x_1)P\left(X_2 = \frac{x}{x_1}\right) \\ &= \sum_{x_1 \in X_1(\Omega)} x_1 P(X_1 = x_1) \sum_{x \in (X_1 X_2)(\Omega)} \frac{x}{x_1} P\left(X_2 = \frac{x}{x_1}\right) \\ &= \sum_{x_1 \in X_1(\Omega)} x_1 P(X_1 = x_1) \sum_{x_2 \in X_2(\Omega)} x_2 P(X_2 = x_2) \\ &= E[X_1]E[X_2]. \end{aligned}$$

10.2.4 Tính tuyến tính của kỳ vọng

Tính tuyến tính, hay tính cộng được, là một tính chất quan trọng của kỳ vọng. Bất kể hai biến ngẫu nhiên X_1 và X_2 có độc lập hay không, luôn có

$$E[\alpha X_1 + \beta X_2] = \alpha E[X_1] + \beta E[X_2].$$

Trong thi đấu, tính chất này thường được dùng bằng cách tách một biến ngẫu nhiên “lớn” thành tổng của các biến ngẫu nhiên “nhỏ”. Khi đó kỳ vọng của biến ngẫu nhiên “lớn” bằng tổng kỳ vọng của từng biến ngẫu nhiên “nhỏ”.

10.2.5 Ứng dụng: *Maze* (phỏng theo Codeforces 123E)

Mô tả bài toán. Cho một cây. Xuất phát từ gốc S , dừng lại khi tới lá T . Hãy tính số bước kỳ vọng của thuật toán DFS. Mỗi lần DFS sẽ lấy các đỉnh chưa tới được từ đỉnh hiện tại, thực hiện *RandomShuffle*, rồi thử đi xuống theo thứ tự ngẫu nhiên đó. Chú ý rằng quá trình quay lui khi DFS cũng tính là một bước.

Lời giải. Với bài đầu tiên liên quan tới kỳ vọng, cần làm rõ một số chi tiết. Trước hết xác định không gian mẫu Ω là tập hợp mọi đường đi bắt đầu từ S và kết thúc ở T . Biến ngẫu nhiên X tác động trên Ω , biểu diễn độ dài của mỗi đường đi. Độ đo xác suất P ánh xạ một tập con nào đó của Ω tới số thực:

$$P(A) = \sum_{\omega \in A} P(\omega).$$

Ở đây $P(\omega)$ là xác suất đi ra đường đi ω ; ta có thể tính nó, nhưng cũng có thể tránh việc tính xác suất này.

Ta cần tính $E[X]$. Không khó thấy rằng khi quy mô dữ liệu đủ lớn, liệt kê từng đường đi là không khả thi, nên phải chọn hướng khác. Hãy xây dựng các biến ngẫu nhiên “nhỏ” X_e , trong đó e là một cạnh của cây, còn $X_e(\omega)$ là số lần đường đi ω đi qua cạnh e , tính cả hai chiều đi và về. Vì với mọi $\omega \in \Omega$,

$$X[\omega] = \sum_e X_e(\omega),$$

ta viết gọn $X = \sum_e X_e$. Dùng tính tuyến tính của kỳ vọng:

$$E[X] = \sum_e E[X_e].$$

Như vậy bài toán chuyển thành tính $E[X_e]$.

Gọi đường đi từ S tới T là đường đi chính. Chia các cạnh thành hai loại: cạnh nằm trên đường đi bắt buộc và cạnh không nằm trên đường đi bắt buộc. Trước hết xét cạnh trên đường đi bắt buộc. Số lần đi qua loại cạnh này chắc chắn là 1. Một mặt, vì cạnh nằm trên đường đi bắt buộc nên nếu không đi qua cạnh đó thì không thể tới T . Mặt khác, một khi đã đi qua cạnh đó, ta nhất định sẽ tới đích trước khi đi ngược lại qua cạnh này, nên cạnh đó bắt buộc và chỉ có thể đi đúng một lần.

Bây giờ xét một cạnh e không nằm trên đường đi bắt buộc. Loại cạnh này có hai khả năng: không đi tới, hoặc đi qua 2 lần cả đi lẫn về. Để tính kỳ vọng số lần đi qua, ta phải tính xác suất đi 0 lần và đi 2 lần. Xét đỉnh u trên đường đi bắt buộc gần cạnh này nhất. Giả sử cạnh này nằm trong cây con gốc v , với v là con của u , còn đỉnh đích nằm trong cây con gốc w , với w cũng là con của u . Ta có $v \neq w$, nếu không u không phải là đỉnh trên đường đi bắt buộc gần e nhất. Khi đó ta nhận thấy e được đi 0 lần hay 2 lần hoàn toàn phụ thuộc vào sau khi xuất phát từ u , thuật toán đi tới v trước hay w trước. Nếu đi tới v trước, mọi cạnh trong cây con gốc v đều được đi 2 lần; nếu đi tới w trước, đường đi không thể vào cây con v nữa. Trong một hoán vị ngẫu nhiên, xác suất một phần tử đứng trước một phần tử khác là $\frac{1}{2}$, nên xác suất cạnh này được đi 0 lần và 2 lần đều là $\frac{1}{2}$. Chú ý rằng lúc này ta thực ra đang xét bài toán ở tầng đầu ra của biến ngẫu nhiên. Do đó kỳ vọng số lần đi qua e là 1.

Tóm lại, kỳ vọng số lần đi qua mỗi cạnh đều là 1. Vì vậy ta thu được một kết luận khá kỳ diệu: bất kể hình dạng của cây ra sao, số cạnh kỳ vọng mà ta đi qua đều là $n - 1$.

10.2.6 Công thức kỳ vọng toàn phần

Trước hết xét một vấn đề tương tự xác suất có điều kiện: nếu biết thêm thông tin, chẳng hạn biến cố A chắc chắn xảy ra, thì biến ngẫu nhiên X trên không gian mẫu Ω sẽ thay đổi như thế nào?

Nếu ký hiệu biến ngẫu nhiên bị ràng buộc này là $X | A$, thì với mọi $x \in X(\Omega)$, ta có

$$P((X | A) = x) = \frac{P(X = x, A)}{P(A)}.$$

Đây là toàn bộ thông tin của biến ngẫu nhiên mới $X | A$, được hình thành khi biến ngẫu nhiên X trên Ω được đặt trên không gian mẫu mới A .

Với hai biến ngẫu nhiên X, Y , một công thức ban đầu có thể hơi khó nắm bắt là

$$E[E[X | Y]] = E[X].$$

Theo phân tích trên, $X | Y = y$ là một biến ngẫu nhiên trên không gian mẫu $Y = y$, do đó $E[X | Y = y]$ là một giá trị xác định. Nếu không chỉ định rõ giá trị của Y , thì $E[X | Y]$ là một biến ngẫu nhiên mới. Kỳ vọng của nó biểu diễn tổng có trọng số theo $P(Y = y)$ của các giá trị $E[X | Y = y]$, trong điều kiện Y nhận các đầu ra cụ thể y .

Ý nghĩa của công thức này tương tự công thức xác suất toàn phần, giúp ta phân loại để thảo luận bài toán kỳ vọng. Công thức xác suất toàn phần cũng là cơ sở của công thức này; trong phép suy diễn dưới đây, bước từ (5) sang (6) dùng công thức xác suất toàn phần. Để dễ hiểu, ta chứng minh như sau:

$$E[E[X | Y]] = \sum_{y \in Y(\Omega)} E[X | Y = y]P(Y = y) \quad (3)$$

$$= \sum_{y \in Y(\Omega)} \sum_{x \in X(\Omega)} xP(X = x | Y = y)P(Y = y) \quad (4)$$

$$= \sum_{y \in Y(\Omega)} \sum_{x \in X(\Omega)} xP(X = x, Y = y) \quad (5)$$

$$= \sum_{x \in X(\Omega)} x \sum_{y \in Y(\Omega)} P(X = x, Y = y) \quad (6)$$

$$= \sum_{x \in X(\Omega)} x \sum_{y \in Y(\Omega)} P(X = x | Y = y)P(Y = y) \quad (7)$$

$$= \sum_{x \in X(\Omega)} xP(X = x) \quad (8)$$

$$= E[X]. \quad (9)$$

Xét một ví dụ thực tế. Trong toàn bộ học sinh của một khối, tức không gian mẫu Ω , chọn đều một người theo độ đo xác suất

$$P(A) = \frac{|A|}{|\Omega|}.$$

Hỏi điểm bài kiểm tra lần trước X của người đó và cần tính $E[X]$. Cách thứ nhất là hỏi trực tiếp điểm của từng học sinh rồi lấy trung bình làm $E[X]$. Cách thứ hai là nhờ mỗi giáo viên chủ nhiệm tính điểm trung bình của lớp mình, rồi lấy tổng có trọng số theo xác suất học sinh được chọn thuộc từng lớp, tức số học sinh lớp đó chia cho tổng số học sinh của khối.

Cách thứ hai thực chất xây dựng một biến ngẫu nhiên mới Y , trong đó $Y(\omega)$ biểu diễn mã lớp của ω . Giá trị tính được là $E[E[X | Y]]$. Rõ ràng kỳ vọng thu được từ hai cách là như nhau; điều này cũng minh họa tính đúng đắn của công thức kỳ vọng toàn phần.

Người đọc có thể chú ý rằng ở đây Y thực ra chỉ liên quan tới thao tác so sánh bằng nhau, nên miền giá trị của nó không nhất thiết là số thực. Điều này sẽ được nhắc lại trong ví dụ phía sau.

10.3 Các vấn đề trên mạng chuyển tiếp xác suất

Mạng chuyển tiếp xác suất là mô hình của một lớp bài toán rất thường gặp trong OI. Bây giờ hãy xem cách giải các dạng vấn đề khác nhau trong mô hình này.

Trước hết cần hiểu mạng chuyển tiếp xác suất là gì. Mạng chuyển tiếp xác suất gọi tắt là mạng, là một mạng có hướng gồm tập đỉnh, hay tập trạng thái, ma trận xác suất chuyển tiếp, tức một hàm hai biến

$$G : V \times V \rightarrow [0, 1],$$

và đỉnh xuất phát v_0 . Với mỗi $u \in V$, ta có

$$\sum_v G[u, v] \leq 1.$$

Sau định nghĩa toán học, hãy xem ý nghĩa thực tế của mô hình này. Một chất điểm chuyển động ban đầu ở thời điểm 0 nằm tại v_0 . Trong mỗi đoạn thời gian, nếu chất điểm nằm tại đỉnh u , thì với mọi $v \in V$, nó có xác suất $G[u, v]$ chuyển tới v , và có xác suất

$$1 - \sum_v G[u, v]$$

biến mất, hoặc có thể hiểu là chuyển tới một đỉnh hư vô.

Những bạn quen chuỗi Markov có thể thấy mô hình này rất giống chuỗi Markov. Tuy nhiên, xét tới các dạng bài có thể gặp trong OI, tác giả đã điều chỉnh chuỗi Markov một chút. Mô hình này có tính phổ quát rất mạnh.

Tiếp theo xét hai ví dụ.

10.3.1 Ví dụ: *Museum* (Codeforces 113D)

Petya và Vasya đang đi du lịch. Họ quyết định tham quan một bảo tàng. Bảo tàng có n phòng, được nối bởi m hành lang, và từ bất kỳ phòng nào cũng có thể đi tới bất kỳ phòng khác. Hai người quyết định tách ra để xem những tác phẩm nghệ thuật mình quan tâm. Họ hẹn sáu giờ chiều gặp nhau tại một phòng. Tuy nhiên họ quên một điều quan trọng: họ chưa chọn phòng gặp mặt. Khi tới sáu giờ, họ bắt đầu chạy khắp bảo tàng để tìm nhau. Dù vậy, họ vẫn chưa xem đủ nhiều tác phẩm nghệ thuật, nên mỗi người hành động như sau: mỗi phút đưa ra một quyết định; với xác suất p_i , người đó không đi đâu trong phút này, tức ở lại phòng hiện tại; với xác suất $1 - p_i$, người đó chọn đều một phòng kề và đi theo hành lang tới đó. Ở đây i là số hiệu phòng hiện tại. Mỗi hành lang nối hai phòng khác nhau, và giữa hai phòng bất kỳ có nhiều nhất một hành lang.

Hai cậu bé hành động đồng thời. Mỗi hành lang có thể đi qua trong một phút. Hai người không thể gặp nhau trong hành lang, nhưng có thể đi từ hai hướng của cùng một hành lang. Ngoài ra, hai người có thể đồng thời đi qua cùng một hành lang mà vẫn không gặp nhau. Hai cậu bé di chuyển theo phương thức trên cho tới khi gặp nhau. Nói cụ thể hơn, nếu ở một thời điểm nào đó hai người chọn đi tới cùng một phòng, họ sẽ gặp nhau trong phòng đó.

Hiện tại hai cậu bé lần lượt ở hai phòng a, b . Hãy tính xác suất họ gặp nhau tại từng phòng. Ràng buộc $n \leq 22$.

Chuyển mô hình. Ta thử chuyển bài toán này thành mạng chuyển tiếp xác suất nói trên. Không khó thấy do có hai nhân vật, ta cần định nghĩa tập trạng thái V là $V_0 \times V_0$, trong đó V_0 là tập các phòng trong đề bài. Từ điều kiện đề bài, ta dễ tính xác suất chuyển từ mỗi trạng thái, tức vị trí của hai người, sang trạng thái khác; đó chính là ma trận G . Trạng thái ban đầu là $v_0 = (a, b)$. Do tính đặc thù của bài, ta còn định nghĩa tập trạng thái dừng

$$S = \{(a, a) \mid a \in V_0\}.$$

Sau đó có hai cách giải bài toán.

Cách 1: phương pháp lập. Nếu xử lý đặc biệt các chuyển tiếp của trạng thái trong S , đặt xác suất chuyển từ mỗi trạng thái này về chính nó bằng 1 và mọi xác suất chuyển tới trạng thái khác bằng 0, thì không khó thấy điều ta cần tính chính là vị trí của chất điểm sau đủ nhiều bước di chuyển. Sau đủ lâu,

chất điểm nhất định dừng trong S . Ký hiệu xác suất chất điểm ở từng điểm của mạng tại thời điểm t là x^t , trong đó x_v^t là xác suất chất điểm ở trạng thái v tại thời điểm t . Ta có

$$x^{t+1} = Gx^t.$$

Ở trạng thái ban đầu, $x_{v_0}^0 = 1$, còn các giá trị khác bằng 0.

Lúc này nghiệm cần tìm là x^∞ . Giá trị này không thể được tính chính xác trong thời gian hữu hạn, nhưng có thể nhận thấy khi t đủ lớn, x^t và x^∞ rất gần nhau. Ta có thể dùng lũy thừa nhanh để tính G^t với t càng lớn càng tốt, rồi dùng $x^t = G^t x^0$ để thu được một nghiệm xấp xỉ khá tốt.

Với phần lớn dữ liệu yếu, cách này có thể chấp nhận được. Nhưng khi dữ liệu mạnh và tốc độ hội tụ của x^t không lý tưởng, cách này sẽ bất lực. Vì vậy cần tìm một lời giải tốt hơn.

Cách 2: giải hệ phương trình tuyến tính. Khác với cách 1, ta dùng cách khác để xử lý các trạng thái trong S : đặt toàn bộ xác suất chuyển tiếp của chúng bằng 0, tức sau khi tới trạng thái trong S , bước tiếp theo chắc chắn chuyển tới “hư vô”. Mục đích của cách xử lý này là thuận tiện cho việc lập phương trình.

Bây giờ xét, trước khi rơi vào hư vô, kỳ vọng số lần chất điểm dừng tại mỗi điểm E_u , tức tổng xác suất chất điểm ở điểm này tại mọi thời điểm:

$$\sum_t x_u^t.$$

Ở đây ta dùng tính tuyến tính của kỳ vọng.

Với một điểm u khác v_0 , ta có

$$\begin{aligned} E_u &= x_u^0 + \sum_{t=1}^{\infty} x_u^t \\ &= x_u^0 + \sum_{t=1}^{\infty} \sum_v x_v^{t-1} G[v, u] \\ &= x_u^0 + \sum_v G[v, u] \sum_{t=0}^{\infty} x_v^t \\ &= x_u^0 + \sum_v G[v, u] E_v. \end{aligned}$$

Hiển nhiên $x_{v_0}^0 = 1$, và $x_u^0 = 0$ với $u \neq v_0$.

Như vậy ta đã lập được một hệ phương trình; giải hệ này là xong. Vì các trạng thái trong S chỉ có thể dừng lại một lần, kỳ vọng số lần chất điểm dừng ở các điểm này chính là xác suất chất điểm dừng ở điểm đó tại bước cuối.

Cách này mạnh hơn cách trước rất nhiều, có thể vượt qua toàn bộ dữ liệu. Độ phức tạp thời gian là $O(n^6)$, không bị quá thời gian. Tin rằng không ít thí sinh biết thuật toán này và có thể cài đặt đúng, thậm chí AC; nhưng qua khảo sát, tác giả nhận thấy phần lớn thí sinh chưa hiểu sâu ý nghĩa của ẩn số trong hệ phương trình này.

10.3.2 Ví dụ: di mê cung (SDOI 2012)

Morenan bị kẹt trong một mê cung. Mê cung có thể xem là đồ thị có hướng gồm N đỉnh, M cạnh; Morenan ở đỉnh xuất phát S , còn đích của mê cung là T . Đáng tiếc là Morenan chỉ biết từ một đỉnh ngẫu nhiên đi theo một cạnh có hướng xuất phát từ đỉnh đó để tới một đỉnh khác. Như vậy số bước của Morenan có thể rất lớn, cũng có thể là vô hạn, càng có thể không tới được đích; nếu không tới được đích thì số bước được xem là vô cùng lớn. Nhưng bạn phải tìm cách tính kỳ vọng số bước Morenan đi.

Lời giải. Bài này rất gần với mô hình trên, việc xây dựng mô hình không khó. Cần chú ý rằng toàn bộ cạnh đi ra khỏi đỉnh T nên bị xóa. Lời giải bài này có một số kỹ thuật. Dùng $E[X_u]$ biểu diễn kỳ vọng số bước cần đi từ đỉnh u tới đỉnh T . Đặc biệt,

$$E[X_T] = 0.$$

Theo trực giác, ta có

$$E[X_u] = \sum_v G[u, v] E[X_v] + 1.$$

Công thức này rất đơn giản: liệt kê bước tiếp theo đi theo hướng nào, lấy tổng có trọng số theo xác suất rồi cộng thêm 1 bước vừa đi.

Tuy nhiên đây chỉ là một công thức trực quan. Dù nó đúng, ta vẫn phải chứng minh nó trên lý thuyết, nếu không sẽ khó thuyết phục. Chứng minh cần dùng công thức kỳ vọng toàn phần

$$E[X_u] = E[E[X_u | Y_u]],$$

trong đó Y_u là một biến ngẫu nhiên biểu diễn đỉnh đạt tới sau bước đầu tiên từ u . Như đã nhắc khi giới thiệu công thức kỳ vọng toàn phần, ở đây cần mở rộng nhẹ khái niệm biến ngẫu nhiên: biến ngẫu nhiên này là ánh xạ từ Ω tới V . Không khó thấy

$$E[X_u | Y_u = v] = E[X_v].$$

Thay vào $E[X_u]$ sẽ thu được kết quả. Người đọc nên tự suy nghĩ kỹ xem không gian mẫu và độ đo xác suất cụ thể trong bài này là gì.

Sau khi có n đẳng thức, ta có thể giải bài toán bằng cách giải hệ phương trình.

10.4 Hai bất đẳng thức thường dùng

10.4.1 Bất đẳng thức Markov

Với biến ngẫu nhiên không âm X và số thực dương a , luôn có

$$P(X \geq a) \leq \frac{E[X]}{a}.$$

Bất đẳng thức này mô tả quan hệ giữa xác suất biến ngẫu nhiên thể hiện một giá trị tương đối lớn và kỳ vọng của nó. Nó rất hữu dụng khi ta chỉ biết kỳ vọng của một biến ngẫu nhiên. Chú ý rằng trong phần lớn trường hợp, dấu bằng không đạt được, và vế trái nhỏ hơn vế phải rất nhiều.

Ứng dụng: ước lượng thời gian chạy của thuật toán tăng dần ngẫu nhiên cho bài đường tròn nhỏ nhất bao phủ. Những bạn biết thuật toán này hẳn đều hiểu rằng độ phức tạp thời gian kỳ vọng của thuật toán là $\Theta(n)$. Nhưng luôn có người lo rằng tuy độ phức tạp kỳ vọng là tuyến tính, trong một số trường hợp thuật toán vẫn có thể suy biến thành bình phương và bị quá thời gian. Ở đây ta dùng bất đẳng thức Markov để phân tích xác suất xảy ra tình huống đó. Dùng biến ngẫu nhiên X biểu diễn thời gian chạy của thuật toán, ta có

$$P(X \geq n^2) \leq \frac{E[X]}{n^2} = \frac{1}{n}.$$

Nói cách khác, xác suất độ phức tạp thời gian của thuật toán suy biến thành $\Theta(n^2)$ là cấp $\Theta(1/n)$. Với $n = 200000$,

$$\frac{1}{n} = 5 \cdot 10^{-6}.$$

Đây chỉ là ước lượng thô nhất, nhưng xác suất suy biến của thuật toán đã nhỏ hơn xác suất một người chết vì tai nạn giao thông. Nói không thật nghiêm ngặt, nếu bạn vẫn không tin vào tính ổn định thời gian của thuật toán này, thì cả đời bạn đừng ra khỏi nhà. Nếu xét thêm các yếu tố khác, xác suất suy biến thường còn nhỏ hơn giá trị trên vài bậc độ lớn.

Ví dụ này cho thấy nếu ta đã đưa ra được độ phức tạp kỳ vọng của một thuật toán, xác suất thuật toán đó suy biến thường cực kỳ nhỏ. Vì vậy khi gặp một số thuật toán có yếu tố ngẫu nhiên trong thời gian chạy, chẳng hạn quicksort ngẫu nhiên hoặc Treap, mọi người có thể yên tâm sử dụng.

10.4.2 Bất đẳng thức Chebyshev

Ta gọi phương sai của biến ngẫu nhiên X là

$$\text{Var}[X] = E[(X - E[X])^2],$$

và độ lệch chuẩn σ là

$$\sqrt{\text{Var}[X]}.$$

Nếu biết phương sai của biến ngẫu nhiên, ta có thể ước lượng sâu hơn xác suất biến ngẫu nhiên lệch khỏi kỳ vọng của nó quá một biên độ nào đó. Với biến ngẫu nhiên X và số thực dương a , có

$$P(|X - E[X]| \geq a) \leq \frac{\text{Var}[X]}{a^2}.$$

Công thức trên cũng có thể viết bằng phép đổi biến $a = c\sigma$:

$$P(|X - E[X]| \geq c\sigma) \leq \frac{1}{c^2}.$$

Công thức này cho thấy khi chỉ biết độ lệch chuẩn, xác suất biến ngẫu nhiên lệch khỏi kỳ vọng của nó ít nhất c lần độ lệch chuẩn không vượt quá $1/c^2$.

Thực ra bất đẳng thức này còn diễn giải trực quan hơn về phương sai: biến ngẫu nhiên có phương sai càng nhỏ thì biên độ lệch khỏi kỳ vọng càng nhỏ.

10.5 Tổng kết

Nội dung xác suất sơ cấp được đề cập trong bài này thật ra không phức tạp. Tuy nhiên khi mới học, nhiều người thường không nắm rõ mạch tư duy, bỏ qua quan hệ logic bên trong, cho rằng nhiều định nghĩa đơn giản không cần chú ý và trực tiếp dựa vào trực giác không đáng tin cậy của mình để hiểu. Bài viết này xuất phát từ “xác suất”, viên đá nền của lý thuyết xác suất; thông qua thảo luận về biến ngẫu nhiên rời rạc, bài viết tập trung nghiên cứu đặc trưng số học cơ bản của biến ngẫu nhiên là kỳ vọng và áp dụng nó vào quá trình giải bài. Cuối cùng, thông qua hai bất đẳng thức ước lượng biến ngẫu nhiên liên quan tới giá trị trung bình, biểu diễn bằng thống kê bậc một là kỳ vọng, và mức độ phân tán, biểu diễn bằng thống kê bậc hai là phương sai, bài viết nêu lên công dụng của hai đặc trưng thống kê này.

Bài viết không đề cập nhiều bài tập; một số nội dung là những chi tiết mà thí sinh có thể đã biết kết luận nhưng chưa hiểu sâu. Nhiều vấn đề trong xác suất có chiều sâu triết học đáng kể, chẳng hạn quan hệ giữa ngẫu nhiên và tất nhiên, cục bộ và toàn cục, hữu hạn và vô hạn. Khi làm bài xong, người đọc nên thử suy nghĩ kỹ về một vài vấn đề nền tảng nhất của xác suất. Biết kết quả và còn biết vì sao có kết quả chắc chắn sẽ giúp dùng xác suất để nâng cao năng lực tư duy một cách hiệu quả.

Do năng lực của tác giả còn hạn chế, nếu trong bài có sai sót mong người đọc chỉ giáo. Chứng minh của nhiều định lý và bất đẳng thức trong bài có thể tìm thấy trong bất kỳ sách nhập môn xác suất nào; do giới hạn dung lượng, bài viết không trình bày lại.

10.6 Lời cảm ơn

Xin cảm ơn CCF đã cung cấp một sân chơi để thể hiện bản thân.

Xin cảm ơn thầy Ni Zhenxiang đã hướng dẫn tôi trong thời gian dài.

Xin cảm ơn Zhang Yicheng, Xu Haoran và Jia Zhipeng đã đưa ra những ý kiến quý báu cho bài viết của tôi.

Xin cảm ơn cha mẹ và nhà trường đã bồi dưỡng tôi.

10.7 Tài liệu tham khảo

1. Su Chun. *Xác suất luận*. Nhà xuất bản Khoa học.
2. Stephen Fletcher Hewson. *Cây cầu toán học: một chuyến thưởng ngoạn tới toán học cao cấp*. Nhà xuất bản Giáo dục Khoa học Kỹ thuật Thượng Hải.
3. <http://tsinsen.com/A1475>, Codeforces 113D Museum.

11 Bàn về một số cách giải không kinh điển cho bài cấu trúc dữ liệu

Xu Haoran

Trường Ngoại ngữ Nam Kinh, Giang Tô

Tóm tắt

Bài cấu trúc dữ liệu là dạng bài rất thường gặp trong thi tin học. Những cách giải kinh điển như cây đoạn, cây cân bằng, chia khối và thuật toán Mo đã quá quen thuộc. Bài viết này giới thiệu một số cách làm ít phổ biến hơn nhưng rất hữu dụng cho bài cấu trúc dữ liệu. Nội dung chính gồm chia để trị theo thời gian và các mở rộng của nó, thuật toán gom nhóm nhị phân, và thuật toán chia đôi tổng thể. Bài viết cũng chứa nhiều kinh nghiệm và nội dung nguyên gốc của tác giả, với mong muốn gợi mở thêm hướng suy nghĩ cho người đọc.

11.1 Gom nhóm nhị phân, chia để trị theo thời gian và mở rộng

11.1.1 Bắt đầu từ giao nửa mặt phẳng động

Giao nửa mặt phẳng động là một bài toán cấu trúc dữ liệu kinh điển. Ta có một mặt phẳng và cần thực hiện một dãy thao tác:

1. chèn một nửa mặt phẳng;
2. xóa nửa mặt phẳng đã được chèn ở một lần nào đó;
3. hỏi một điểm có nằm trong giao của các nửa mặt phẳng hiện tại hay không.

Bài toán này có một thuật toán kinh điển nhưng cài đặt cực kỳ phức tạp: dùng cây đoạn lồng cây cân bằng bền vững, duy trì bằng thao tác cắt và gộp cây cân bằng bền vững để bảo đảm độ phức tạp. Phần phân trường hợp khi gộp có rất nhiều chi tiết.

Nếu ví giao nửa mặt phẳng động với một con quái vật lớn, thì cách làm trên giống như một cây gậy sắt lớn: quả thật có thể đập chết quái vật, nhưng hơi quá nặng tay. Thực ra, bằng cách mở rộng chia để trị theo thời gian, ta có thể giải bài toán này một cách khéo léo hơn. Cách làm giống như một con dao sắc:

nhìn bề ngoài không mạnh, nhưng đâm thẳng vào chỗ cốt lõi. Các phần dưới đây sẽ từng bước đi tới lời giải đó.

11.1.2 Bài cấu trúc dữ liệu có “cập nhật độc lập, cho phép ngoại tuyến”

Trước hết, ta quy ước trong bài cấu trúc dữ liệu: thao tác cần trả lời được gọi là *truy vấn*; thao tác có thể ảnh hưởng tới đáp án của truy vấn được gọi là *cập nhật*.

Ta nhận thấy khá nhiều bài cấu trúc dữ liệu thỏa mãn hai điều kiện sau. Khi đó chia để trị theo thời gian là một công cụ rất hữu ích.

1. Đóng góp của các cập nhật đối với truy vấn là độc lập; các cập nhật không ảnh hưởng hiệu quả của nhau.
2. Bài toán cho phép dùng thuật toán ngoại tuyến.

Ta nên khai thác hai điều kiện này như thế nào? Giả sử cần xử lý một dãy thao tác S . Ta chia đều toàn bộ dãy thao tác thành hai nửa. Khi đó có hai tính chất:

1. Các cập nhật trong nửa sau hiển nhiên không ảnh hưởng gì tới kết quả của các truy vấn trong nửa trước.
2. Các truy vấn trong nửa sau chỉ chịu ảnh hưởng từ hai phía: toàn bộ cập nhật trong nửa trước, và các cập nhật trong nửa sau đứng trước chính truy vấn đó.

Vì cập nhật ở nửa sau hoàn toàn không ảnh hưởng tới truy vấn ở nửa trước, các truy vấn ở nửa trước thực chất độc lập với nửa sau. Đây là một bài toán con cùng dạng với bài toán gốc, nên có thể xử lý đệ quy.

Tiếp theo xét truy vấn ở nửa sau. Trong hai nguồn ảnh hưởng của nó, phần “các cập nhật trong nửa sau đứng trước truy vấn đó” cũng hoàn toàn không liên quan tới nửa trước: theo giả thiết, các cập nhật độc lập với nhau và không ảnh hưởng lẫn nhau. Do đó phần này cũng là một bài toán con cùng dạng với bài toán gốc và có thể xử lý đệ quy.

Nguồn ảnh hưởng còn lại là “toàn bộ cập nhật trong nửa trước”. Phần này tuy có liên quan mật thiết tới nửa trước, nhưng có một tính chất rất tốt: đối với mọi truy vấn ở nửa sau, tập cập nhật tạo ảnh hưởng là như nhau. Chính là toàn bộ cập nhật ở nửa trước. Như vậy cập nhật động trong bài toán gốc đã biến mất, và bài toán được chuyển thành một dạng đơn giản hơn: ban đầu cho tất cả cập nhật, sau đó trả lời một số truy vấn ngoại tuyến.

Phân tích độ phức tạp: giả sử độ phức tạp của phiên bản “không có cập nhật động” là $O(f(n))$. Theo định lý Master, chia để trị theo thời gian có

$$T(n) = 2T\left(\frac{n}{2}\right) + O(f(n)), \quad T(n) \leq O(f(n) \log n).$$

Vì vậy, miễn bài cấu trúc dữ liệu thỏa mãn hai yêu cầu “cập nhật độc lập” và “cho phép ngoại tuyến”, ta có thể trả giá thêm một nhân tử $\log n$ để loại bỏ cập nhật động trong bài gốc, biến nó thành phiên bản đơn giản hơn không có cập nhật động.³

Ví dụ 1: các đường tròn cùng đi qua một điểm. Trên một mặt phẳng, cần hỗ trợ các thao tác sau. Để đơn giản hóa, giả sử mọi điểm đều nằm phía trên trục x và hoành độ khác 0:

1. cho một điểm, chèn đường tròn có tâm tại điểm đó và đi qua gốc tọa độ;

³Lập luận bằng định lý Master ở đây thật ra cần $O(f(n)) \geq O(n)$. Nhưng vì dãy thao tác đã có độ dài n , độ phức tạp xử lý một lần gần như không thể nhỏ hơn $O(n)$; nếu thật sự không thỏa mãn thì cần phân tích riêng theo bài cụ thể.

2. hỏi một điểm có nằm trong tất cả các đường tròn hay không.

Cách chính thức của bài này là nghịch đảo các đường tròn, rồi chuyển thành bài duy trì bao lồi: dùng splay để duy trì toàn bộ bao lồi động, hỗ trợ chèn điểm mới và hỏi điểm có nằm trong bao lồi hay không. Mã lệnh khá phức tạp, độ phức tạp là $O(n \log n)$.

Thực ra bài này có cách làm đơn giản hơn. Ta trước hết biến đổi một chút. Điều kiện “điểm (x_0, y_0) nằm trong đường tròn tâm (x, y) và đi qua gốc tọa độ” tương đương với

$$(x_0 - x)^2 + (y_0 - y)^2 \leq x^2 + y^2,$$

tức

$$2x_0x + 2y_0y \geq x_0^2 + y_0^2.$$

Nói cách khác, tập tâm (x, y) thỏa điều kiện là một nửa mặt phẳng $2x_0x + 2y_0y \geq x_0^2 + y_0^2$. Bài toán trở thành:

1. chèn một điểm mới;
2. cho một nửa mặt phẳng, hỏi có phải mọi điểm đều nằm trong nửa mặt phẳng này hay không.

Dễ thấy các thao tác cần hỗ trợ thỏa mãn “cập nhật độc lập, cho phép ngoại tuyến”: mỗi điểm được chèn có quyền phủ quyết độc lập đối với đáp án truy vấn, không chịu can thiệp bởi yếu tố khác. Vì vậy ta áp dụng trực tiếp chia để trị theo thời gian, đưa bài toán về dạng:

Ban đầu cho một tập điểm. Cần trả lời một loạt truy vấn: cho một nửa mặt phẳng, hỏi có phải mọi điểm đều nằm trong nửa mặt phẳng này hay không.

Bài toán này rất đơn giản. Xét điểm có hình chiếu nhỏ nhất theo hướng vuông góc với nửa mặt phẳng. Nếu điểm này cũng nằm trong nửa mặt phẳng, thì mọi điểm đều nằm trong đó. Hơn nữa, chỉ các điểm trên bao lồi mới có thể là điểm cần tìm. Vì vậy ta dựng bao lồi của tập điểm ban đầu. Quan sát tiếp, hai cạnh kề điểm cần tìm trên bao lồi có hệ số góc vừa “keo” hệ số góc của nửa mặt phẳng. Do đó, sau khi sắp xếp các cạnh của bao lồi theo hệ số góc, ta có thể tìm điểm cần xét bằng tìm kiếm nhị phân theo hệ số góc của nửa mặt phẳng.

Với cách trực tiếp, thời gian dựng và trả lời trong toàn bộ chia để trị là $O(n \log^2 n)$, mã lệnh rất ngắn. Độ phức tạp này nhiều hơn cách chính thức một nhân tử $\log n$.

Vậy chia để trị theo thời gian có thật sự không đạt được độ phức tạp như lời giải chính thức không? Câu trả lời là có thể đạt được. Nút thắt của thuật toán là dựng bao lồi, sắp xếp hệ số góc và tìm kiếm nhị phân. Nhưng ta đã bỏ quên một tính chất quan trọng: đây là thuật toán chia để trị. Sau khi xử lý xong hai bài toán con của nửa trước và nửa sau, ta đã có bao lồi của từng nửa; vì sao không gộp tuyến tính chúng lại? Việc sắp xếp hệ số góc cũng có thể làm tương tự bằng trộn. Truy vấn cũng không phải vấn đề: ta trộn các truy vấn theo hệ số góc. Khi cả hệ số góc của bao lồi và hệ số góc truy vấn đều có thứ tự, chỉ cần quét một lần tuyến tính là tìm được điểm cần xét. Sau các tối ưu này, độ phức tạp xử lý mỗi tầng trở thành tuyến tính, và tổng độ phức tạp đạt $O(n \log n)$, bằng lời giải chính thức. Dù vậy, cách tối ưu này không có nhiều ý nghĩa thực tế: do hằng số, bản $O(n \log n)$ đôi khi chưa chắc nhanh hơn bản $O(n \log^2 n)$, trong khi mã lệnh dài hơn rất nhiều.

Ví dụ 2: bài toán kinh điển. Đây là phiên bản đơn giản của bài toán ban đầu. Cho một nửa mặt phẳng, cần hỗ trợ:

1. chèn một nửa mặt phẳng;
2. hỏi một điểm có nằm trong giao của các nửa mặt phẳng hiện tại hay không.

Bài này là phiên bản bỏ thao tác xóa của giao nửa mặt phẳng động. Dễ thấy nó cũng thỏa mãn “cập nhật độc lập, cho phép ngoại tuyến”. Dùng chia để trị theo thời gian, bài toán được rút gọn thành:

Ban đầu cho một số nửa mặt phẳng. Cần trả lời một loạt truy vấn: cho một điểm, hỏi điểm đó có nằm trong giao của các nửa mặt phẳng hiện tại hay không.

Bài toán tĩnh này hẳn mọi người đều biết cách làm: dùng thuật toán giao nửa mặt phẳng để tìm giao của toàn bộ nửa mặt phẳng, sau đó chọn tùy ý một điểm bên trong giao và sắp xếp các đỉnh của đa giác giao theo góc cực quanh điểm đó. Khi truy vấn, chỉ cần tìm kiếm nhị phân để xác định điểm hỏi rơi vào khoảng góc nào. Bản trực tiếp cho độ phức tạp $O(n \log^2 n)$ sau khi tính cả chia để trị.

So với cách dùng cây cân bằng để duy trì thứ tự các đỉnh của giao nửa mặt phẳng, cách này có mã lệnh nhỏ hơn nhiều. Ngoài ra, cũng có thể dùng kỹ thuật tương tự ví dụ trước để tối ưu độ phức tạp xuống $O(n \log n)$, bằng với cách duy trì bằng cây cân bằng.

11.1.3 Bài cấu trúc dữ liệu có “cập nhật độc lập, yêu cầu trực tuyến”

Có một số bài cấu trúc dữ liệu tuy thỏa mãn tính “cập nhật độc lập”, nhưng người ra đề dùng nhiều cách như mã hóa thao tác hoặc bài tương tác để buộc thí sinh dùng thuật toán trực tuyến. Khi đó chia để trị theo thời gian không còn hiệu lực. Nhưng ta vẫn có một phương pháp tốt để đối phó: thuật toán gom nhóm nhị phân.

Ta tách một số thành tổng các lũy thừa của 2, theo thứ tự từ lớn tới nhỏ. Ví dụ:

$$\begin{aligned} 21 &= 16 + 4 + 1, \\ 22 &= 16 + 4 + 2, \\ 23 &= 16 + 4 + 2 + 1, \\ 24 &= 16 + 8. \end{aligned}$$

Ý nghĩa của việc này là gì? Ta có thể dùng cách tách trên để chia các cập nhật thành nhóm. Chẳng hạn, với 21 cập nhật đầu tiên, ta gom 16 cập nhật đầu thành một nhóm, cập nhật thứ 17 tới 20 thành nhóm thứ hai, và cập nhật thứ 21 thành nhóm thứ ba. Với mỗi nhóm, ta dùng một cấu trúc dữ liệu để duy trì và hỗ trợ truy vấn trực tuyến.

Giả sử với một nhóm m phần tử, độ phức tạp dựng cấu trúc và hỗ trợ truy vấn lần lượt là $O(f(m))$ và $O(g(m))$. Khi truy vấn, chỉ có $O(\log n)$ nhóm, ta truy vấn từng nhóm rồi gộp kết quả, nên độ phức tạp một truy vấn là $O(g(n) \log n)$. Cách làm đúng vì tiền đề lớn là đóng góp của cập nhật cho truy vấn độc lập, các cập nhật không ảnh hưởng lẫn nhau; do đó chia cập nhật thành nhóm tùy ý và xử lý riêng từng nhóm không làm thay đổi đáp án.

Khi có một cập nhật mới, ta so sánh cách phân nhóm hiện tại với cách phân nhóm sau khi thêm cập nhật đó. Sẽ có một số nhóm cũ biến mất, và một nhóm mới xuất hiện. Ta xóa các nhóm không còn tồn tại và dựng thô bạo nhóm mới. Tổng chi phí của việc dựng lại các nhóm có thể phân tích bằng lowbit. Khi thêm cập nhật thứ k , số phần tử trong các nhóm được dựng lại là $\text{lowbit}(k)$, tức giá trị của bit 1 thấp nhất trong biểu diễn nhị phân của k .⁴

Xét các giá trị có thể có của $\text{lowbit}(k)$ và số lần chúng xuất hiện, ta thu được

$$\begin{aligned} \sum_{1 \leq k < n} O(f(\text{lowbit}(k))) &= \sum_{1 \leq i < \log n} \left(\frac{n}{2^i} + O(1) \right) O(f(2^i)) \\ &\leq \sum_{1 \leq i < \log n} O(f(n)) = O(f(n) \log n). \end{aligned}$$

⁴Ví dụ $22 = (10110)_2$, nên $\text{lowbit}(22) = (10)_2 = 2$.

Vì vậy, bằng gom nhóm nhị phân, ta trả thêm một nhân tử $\log n$ để loại bỏ cập nhật động trong bài gốc, chuyển thành phiên bản đơn giản không có cập nhật; đồng thời vẫn đáp ứng yêu cầu thuật toán trực tuyến.⁵

Ví dụ 3: bài toán kinh điển. Đây là phiên bản mạnh hơn của ví dụ 2. Cho một mặt phẳng, cần hỗ trợ trực tuyến:

1. chèn trực tuyến một nửa mặt phẳng;
2. hỏi trực tuyến một điểm có nằm trong giao của các nửa mặt phẳng hiện tại hay không.

Như bài trước, thao tác của bài này thỏa mãn yêu cầu “cập nhật độc lập”, chỉ khác ở chỗ bài yêu cầu thuật toán trực tuyến. Vì vậy ta áp dụng trực tiếp gom nhóm nhị phân, đưa bài toán về:

Ban đầu cho một số nửa mặt phẳng. Cần trả lời trực tuyến một loạt truy vấn: cho một điểm, hỏi điểm đó có nằm trong giao của các nửa mặt phẳng hiện tại hay không.

Bài toán này dùng được ngay cách giải trong ví dụ 2, vì cách đó vốn hỗ trợ truy vấn trực tuyến. Sau khi tính thêm chi phí gom nhóm nhị phân, độ phức tạp trở thành $O(n \log^2 n)$ cho cập nhật và $O(\log^2 n)$ cho mỗi truy vấn. Tuy không bằng cách kinh điển dùng cây cân bằng để đạt $O(\log n)$ mỗi thao tác, thuật toán này có hằng số nhỏ hơn không ít, mã ngắn, dễ cài đặt, và hầu như không có phân trường hợp dễ gây lỗi.

Nhân tiện nêu một mở rộng nhỏ: nếu thay vì phân rã nhị phân, ta dùng phân rã tam phân hoặc cơ số cao hơn thì sao? Có thể thấy khi cơ số tăng, độ phức tạp một truy vấn sẽ tăng, nhưng tổng độ phức tạp cập nhật giảm. Trong một số bài rất đặc biệt, chẳng hạn dữ liệu bảo đảm phần lớn thao tác là cập nhật, điều này có thể hữu dụng.

11.1.4 Hỗ trợ thao tác xóa và thay đổi

Có một lớp bài cấu trúc dữ liệu cho phép thuật toán ngoại tuyến, trong đó cập nhật gồm cả chèn và xóa. Dù thao tác chèn có đóng góp độc lập đối với truy vấn, và các thao tác chèn không ảnh hưởng lẫn nhau, thao tác xóa có thể hủy hiệu lực của một thao tác chèn trước đó, khiến cập nhật không còn hoàn toàn độc lập. Bài giao nửa mặt phẳng động ở đầu bài viết chính là ví dụ như vậy.

Cách kinh điển cho lớp bài này thường cực kỳ phức tạp, đa phần cần các cấu trúc như cây lồng cây, đôi khi còn cần cấu trúc dữ liệu bền vững. Lượng mã thường rất lớn và chi tiết rất rườm rà. Nhưng nếu phát huy đầy đủ sức mạnh của chia để trị theo thời gian, ta vẫn có thể thu được cách làm đơn giản và đẹp.

Ví dụ 4: giao nửa mặt phẳng động. Ta quay lại bài toán ban đầu. Cần thực hiện một dãy thao tác:

1. chèn một nửa mặt phẳng;
2. xóa nửa mặt phẳng đã được chèn ở một lần nào đó;
3. hỏi một điểm có nằm trong giao của các nửa mặt phẳng hiện tại hay không.

Bây giờ ta tiếp tục xét chia để trị theo thời gian. Các cập nhật, tức chèn và xóa, không còn hoàn toàn độc lập, vì một thao tác chèn có thể bị thao tác xóa phía sau hủy bỏ. Dù vậy, ta vẫn thử chia dãy thao tác thành hai nửa.

⁵Tương tự phần trước, phân tích này cũng ngầm cần $O(f(n)) \geq O(n)$. Vì dãy đã có độ dài n , độ phức tạp xử lý một lần gần như không thể nhỏ hơn tuyến tính; nếu thật sự gặp ngoại lệ thì cần phân tích riêng theo bài cụ thể.

Truy vấn trong nửa trước hiển nhiên hoàn toàn không liên quan tới chèn và xóa ở nửa sau, nên đó là bài toán con cùng dạng với bài toán gốc và có thể đệ quy. Truy vấn trong nửa sau chỉ liên quan tới hai phần:

1. các thao tác chèn trong nửa trước mà tới thời điểm truy vấn vẫn chưa bị xóa;
2. các thao tác chèn trong nửa sau, đứng trước truy vấn và chưa bị xóa.

Vì thao tác chèn trong nửa sau rõ ràng không thể bị xóa ở nửa trước, phần thứ hai hoàn toàn không liên quan tới nửa trước; nó cũng là bài toán con cùng dạng với bài toán gốc và có thể đệ quy. Do đó phần thực sự cần xử lý là:

Đóng góp của các thao tác chèn trong nửa trước còn chưa bị xóa đối với các truy vấn trong nửa sau.

Nhiệm vụ thực tế trở thành: ban đầu cho một tập nửa mặt phẳng, cần hỗ trợ xóa một nửa mặt phẳng và hỏi một điểm có nằm trong tất cả nửa mặt phẳng hay không. Bài này nếu giải trực tiếp thì rất khó. Nhưng nó chỉ có thao tác xóa, không có thao tác chèn. Vì vậy một kỹ thuật kinh điển phát huy tác dụng: đảo ngược thời gian.

Vì đang dùng thuật toán ngoại tuyến, ta biết trước toàn bộ thông tin cần thiết của bài toán đã chuyển đổi: tập nửa mặt phẳng ban đầu, dãy xóa và dãy truy vấn. Ta tìm trước những nửa mặt phẳng cuối cùng không bị xóa, rồi xử lý dãy thao tác theo thứ tự ngược. Truy vấn không đổi, còn xóa nửa mặt phẳng biến thành chèn nửa mặt phẳng. Nhờ đảo ngược thời gian, bài toán lại trở thành:

1. chèn một nửa mặt phẳng;
2. hỏi một điểm có nằm trong tất cả các nửa mặt phẳng hay không.

Đây chính là ví dụ 2. Nếu dùng phiên bản đơn giản của ví dụ 2 làm thủ tục con, bài toán giao nửa mặt phẳng động có thể được giải trong $O(n \log^3 n)$; nếu dùng phiên bản đã tối ưu bằng trộn, độ phức tạp giảm xuống $O(n \log^2 n)$, cùng bậc với cách kinh điển. Cụ thể, với phiên bản trực tiếp, nếu độ dài dãy đang xử lý là n , ta có

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n \log^2 n), \quad T(n) = O(n \log^3 n).$$

Thông qua việc phát huy chia để trị theo thời gian, ta dùng một cách làm đơn giản để giải một bài mà trong cách kinh điển rất khó. Từ đây, chỉ cần thao tác chèn có đóng góp độc lập và bài toán cho phép ngoại tuyến, ngay cả khi có xóa hoặc thay đổi thao tác, ta cũng có thể kết hợp chia để trị theo thời gian và đảo ngược thời gian, trả giá thêm hai nhân tử $\log n$, để đưa bài về phiên bản hoàn toàn tĩnh: không chèn động, không xóa động, không thay đổi động.

Nếu bài giao nửa mặt phẳng động bị sửa thành yêu cầu thuật toán trực tuyến thì liệu chia để trị theo thời gian hoặc gom nhóm nhị phân còn tiếp tục dùng được không? Với vấn đề này, tác giả chưa tìm được cách giải. Bước “đảo ngược thời gian” đã quyết định bài toán phải cho phép ngoại tuyến. Tuy nhiên, ít nhất sau khi áp dụng gom nhóm nhị phân một lần, thao tác cần hỗ trợ đã không còn chèn động; với một số bài, tính chất này có thể vẫn hữu dụng. Có lời giải mạnh hơn hay không còn chờ người đọc khám phá.

11.2 Thuật toán chia đôi tổng thể và ứng dụng

Chia đôi đáp án là kỹ thuật thường gặp. Nhưng trong bài cấu trúc dữ liệu, phương pháp này thường không hiệu quả, vì ta thường cần tiền xử lý một số thứ để trả lời nhanh tại điểm chia đôi hiện tại, trong khi bất kỳ giá trị đáp án nào cũng có thể bị chọn làm điểm chia đôi; độ phức tạp tiền xử lý vì vậy trở nên không chấp nhận được. Để xử lý lớp vấn đề này, ta có một phương pháp tốt: chia đôi tổng thể.

Để chuẩn hóa vấn đề, ta đặt các quy ước sau. Giá trị đáp án đang được dùng để chia đôi gọi là *chuẩn phán định*. Với mỗi cập nhật, khi kết hợp cùng một truy vấn cụ thể và chuẩn phán định, ta tính được đóng góp của cập nhật đó đối với đáp án phán định của truy vấn. Đáp án phán định của truy vấn là tổng đóng góp của các cập nhật; mỗi truy vấn có một yêu cầu đối với đáp án phán định. Dựa trên quan hệ giữa đáp án phán định thực tế và yêu cầu này, ta xác định đáp án thật của truy vấn nằm phía trên hay phía dưới chuẩn phán định hiện tại.

Chia đôi tổng thể cần bài cấu trúc dữ liệu thỏa mãn các tính chất sau:

1. đáp án của truy vấn có tính chia đôi được;
2. đóng góp của các cập nhật đối với đáp án phán định độc lập với nhau; các cập nhật không ảnh hưởng hiệu quả của nhau;
3. nếu một cập nhật có đóng góp cho đáp án phán định, thì giá trị đóng góp đó là xác định và không phụ thuộc vào chuẩn phán định;
4. các đóng góp thỏa mãn tính giao hoán, kết hợp và cộng được;
5. bài toán cho phép dùng thuật toán ngoại tuyến.

Tính chia đôi được của đáp án hiển nhiên là tiền đề. Vì đóng góp của cập nhật đối với chuẩn phán định là độc lập, và giá trị đóng góp nếu có không phụ thuộc vào chuẩn phán định, nên một khi ta đã tính đóng góp của một số cập nhật đối với truy vấn, đóng góp đó sẽ không bao giờ thay đổi. Ta không cần tính lại phần này khi chuẩn phán định thay đổi; chỉ cần ghi lại tổng đóng góp hiện tại, rồi khi tiếp tục chia đôi, cộng trực tiếp đóng góp mới vào.

Nhờ vậy, độ phức tạp xử lý không còn liên hệ trực tiếp với độ dài toàn dãy, mà chỉ liên hệ với độ dài của dãy con đang được xử lý.

Gọi $T(C, S)$ là độ phức tạp khi độ dài đoạn giá trị đáp án đang chia đôi là C , độ dài dãy thao tác đang xử lý là S . Giả sử độ phức tạp xử lý một lần là $O(f(n))$. Khi chia dãy đang xử lý thành hai phần kích thước S_1 và $S - S_1$, ta có

$$T(C, S) = T(C/2, S_1) + T(C/2, S - S_1) + O(f(S)).$$

Suy ra

$$T(C, n) \leq O(f(n) \log C).$$

Như vậy độ phức tạp trở nên chấp nhận được. Bằng chia đôi tổng thể, ta chỉ trả thêm một nhân tử $\log C$ để thực hiện tìm kiếm nhị phân trong điều kiện có tiền xử lý, cùng bậc nhân tử với tìm kiếm nhị phân thông thường.

Trong nguồn, tác giả đưa một khung giả mã cho chia đôi tổng thể. Ý tưởng của khung đó là: với một đoạn giá trị đáp án $[L, R]$, lấy M làm trung điểm và dùng thủ tục xử lý để tính đóng góp của các cập nhật thuộc nửa $[L, M]$. Mỗi truy vấn đang giữ hai trường: yêu cầu phán định và tổng đóng góp đã tích lũy. Nếu đóng góp tích lũy cộng với đóng góp mới đã đủ đạt yêu cầu, truy vấn đi tiếp sang nửa trái; ngược lại, đóng góp mới được cộng vào tổng tích lũy và truy vấn đi sang nửa phải. Điểm mấu chốt là đóng góp đã tính không bị tính lại ở các tầng đệ quy sau.

Ví dụ 5: bài toán kinh điển. Cho một dãy số, cần trả lời một loạt truy vấn:

Cho l, r, k , hỏi giá trị lớn thứ k trong dãy con từ phần tử thứ l tới phần tử thứ r .

Ta xét dùng chia đôi tổng thể. Giả sử chuẩn phán định hiện tại là s . Khi đó chỉ cần đếm trong mỗi đoạn truy vấn có bao nhiêu phần tử lớn hơn s . Nếu số lượng này ít nhất k , đáp án chắc chắn lớn hơn s ;

ngược lại, đáp án không lớn hơn s , và phần các phần tử lớn hơn s luôn tạo cùng một đóng góp cho truy vấn. Vì vậy khi truy vấn đi sang nửa còn lại, ta trừ số lượng này khỏi k và các tầng sau không cần thống kê lại phần đó.

Vậy làm sao đếm nhanh số phần tử lớn hơn s trong từng đoạn truy vấn? Nhiều người có thể nghĩ ngay tới cách mở một mảng, đánh dấu vị trí có phần tử lớn hơn s bằng 1, rồi lấy tổng tiền tố. Một lần xử lý là $O(n)$ để dựng mảng và $O(1)$ cho mỗi truy vấn; nghe có vẻ hoàn hảo, nhưng thực ra không đúng với yêu cầu độ phức tạp của chia đôi tổng thể.

Điểm sai nằm ở chỗ độ phức tạp xử lý tuyệt đối không được phụ thuộc tuyến tính vào độ dài toàn dãy ban đầu; nó chỉ được phụ thuộc vào độ dài đoạn thao tác hiện đang xử lý. Nếu mỗi tầng đều mất $O(n)$, tổng độ phức tạp sẽ suy biến gần như vét cạn. Cách đúng là: với các phần tử lớn hơn s trong đoạn đang xử lý, sắp xếp vị trí của chúng; sau đó dùng tìm kiếm nhị phân trên danh sách vị trí để trả lời mỗi truy vấn. Nếu z là kích thước đoạn đang xử lý, chi phí tương ứng là $O(z \log z)$ cho xử lý và $O(\log z)$ cho mỗi truy vấn. Tổng độ phức tạp là

$$O((n + Q) \log n \log C),$$

trong đó C là quy mô miền giá trị.

Ví dụ 6: ma trận nhân. Cho một ma trận $n \times n$, cần trả lời nhiều truy vấn:

Hỏi giá trị lớn thứ k trong một ma trận con.

Cách làm rất giống ví dụ trước, chỉ khác là từ một chiều chuyển thành hai chiều. Áp dụng chia đôi tổng thể, bài toán được chuyển thành:

Cho một ma trận, trong đó một số phần tử được đánh dấu. Cần trả lời một loạt truy vấn: trong một ma trận con có bao nhiêu phần tử được đánh dấu.

Yêu cầu phụ là độ phức tạp không được phụ thuộc tuyến tính vào kích thước toàn ma trận, mà chỉ được phụ thuộc vào số phần tử được đánh dấu.

Bài toán này có thể giải bằng cây BIT hai chiều. Có một chi tiết dễ sai: vì độ phức tạp không được phụ thuộc tuyến tính vào tổng số phần tử, nên sau khi truy vấn xong không thể đơn giản xóa trắng toàn bộ cây BIT hai chiều; đó là thao tác phụ thuộc vào kích thước toàn ma trận. Ta phải lần lượt hoàn tác các cập nhật theo thứ tự ngược, vì thao tác này chỉ phụ thuộc vào số phần tử đã đánh dấu. Độ phức tạp là

$$O((n^2 + Q) \log^2 n \log C).$$

Ví dụ 7: ZJOI 2013, truy vấn số lớn thứ k . Có n tập số, mỗi tập có thể chứa nhiều phần tử trùng nhau. Cần hỗ trợ:

1. thêm một số c vào mọi tập từ tập thứ l tới tập thứ r ;
2. lấy tất cả số trong các tập từ l tới r rồi gộp lại, hỏi số lớn thứ k là bao nhiêu.

Nếu áp dụng một lần chia đôi tổng thể, bài toán chuyển thành:

1. cộng 1 cho một đoạn của một dãy;
2. hỏi tổng trên một đoạn của dãy.

Yêu cầu phụ vẫn là độ phức tạp không được phụ thuộc tuyến tính vào độ dài dãy, mà chỉ được phụ thuộc vào số thao tác.

Nếu không có yêu cầu phụ này, bài toán quá quen thuộc: cây đoạn với lazy tag. Nhưng với yêu cầu phụ, ta cần một cách nghĩ khác. Thao tác cộng đoạn vẫn có tính “cập nhật độc lập”. Vì vậy ta áp dụng trực tiếp chia để trị theo thời gian, đưa bài toán về dạng tĩnh:

Ban đầu cho một số đoạn. Cần trả lời nhiều truy vấn: cho một đoạn, hỏi tổng độ dài giao của đoạn truy vấn với các đoạn đã cho ban đầu.

Mỗi đoạn ban đầu đóng góp cho đáp án dưới dạng một hàm bậc nhất từng đoạn. Sắp xếp rồi quét một lần là đủ. Độ phức tạp là $O(n \log^3 n)$; nếu trong chia để trị theo thời gian dùng trộn để thay cho sắp xếp lặp lại, có thể tối ưu xuống $O(n \log^2 n)$. Bằng việc kết hợp chia đôi tổng thể và chia để trị theo thời gian, ta giải được bài này.

Chia đôi tổng thể là một công cụ xử lý rất hiệu quả, có thể đơn giản hóa nhiều bài toán. Chẳng hạn, ta có thể tăng cường bài ở đầu bài viết thành:

1. chèn một nửa mặt phẳng có trọng số;
2. xóa một nửa mặt phẳng đã chèn;
3. hỏi trong các nửa mặt phẳng phủ một điểm, trọng số lớn nhất là bao nhiêu.

Nếu không dùng chia đôi tổng thể, bài này gần như không thể giải. Sau khi chia đôi tổng thể, bài toán được chuyển thành giao nửa mặt phẳng động ở đầu bài viết, và do đó được giải quyết.

11.3 Tổng kết

Chia để trị theo thời gian, gom nhóm nhị phân và chia đôi tổng thể đều là những công cụ hữu hiệu để giải bài cấu trúc dữ liệu. Ứng dụng chúng đúng cách và phát huy đầy đủ sức mạnh của chúng có thể làm giảm đáng kể độ khó của bài toán và độ khó khi lập trình.

Bản chất của các thuật toán này là khai thác các tính chất đặc biệt ẩn trong thao tác của bài cấu trúc dữ liệu, chẳng hạn “cập nhật độc lập”, “đóng góp độc lập” và “cho phép ngoại tuyến”, để chuyển hóa và đơn giản hóa bài toán. Điều này gợi ý rằng khi giải bài, ta nên đào sâu tính chất riêng của đề và tận dụng chúng, từ đó thu được thuật toán đẹp và hiệu quả hơn.

11.4 Lời cảm ơn

- Xin cảm ơn CCF đã cung cấp một nền tảng trao đổi.
- Xin cảm ơn cha mẹ và nhà trường đã bồi dưỡng tôi.
- Xin cảm ơn Jia Zhipeng, Luo Yuping, Peng Tianyi, Hu Yuanming, Wang Yuetong và nhiều bạn khác đã trao đổi với tôi qua mạng hoặc ngoài đời, đồng thời cho tôi nhiều giúp đỡ và cổ vũ.
- Xin cảm ơn Luo Yuping, Long Haomin và các bạn khác đã giúp tôi ở mảng truy vấn số lớn thứ k trên đoạn.
- Xin cảm ơn giáo viên chủ nhiệm và các thầy cô khác đã ủng hộ tôi.

11.5 Tài liệu tham khảo

1. Chen Danqi. *Từ Cash bàn về ứng dụng của một lớp thuật toán chia để trị*.
2. Bài giảng của Gu Xizhou tại WC2013.

12 Ứng dụng cây cân bằng theo trọng lượng và cây cân bằng hậu tố trong Olympic Tin học

Chen Lijie

Trường Ngoại ngữ Hàng Châu, Chiết Giang

Tóm tắt

Cây cân bằng theo trọng lượng là một khái niệm cây cân bằng có giá trị cả về nghiên cứu lẫn thực tiễn. Cây cân bằng hậu tố là một cấu trúc dữ liệu hậu tố rất có giá trị, xây dựng dựa trên cây cân bằng theo trọng lượng. Tuy vậy, trong các hoạt động OI những năm gần đây ta hiếm khi thấy chúng xuất hiện. Để cải thiện tình trạng đó, bài viết này giới thiệu khái niệm cây cân bằng theo trọng lượng và các loại thường dùng từ nhiều góc độ, đồng thời chọn một số ứng dụng kinh điển để thảo luận sâu hơn.

12.1 Cấu trúc bài viết

Trước hết, bài viết giới thiệu ngắn gọn khái niệm cây cân bằng theo trọng lượng, đồng thời giới thiệu một số cây thuộc loại này:

1. Treap;
2. skip list;
3. scapegoat tree.

Sau đó, bài viết giới thiệu vài ứng dụng của cây cân bằng theo trọng lượng:

1. truy vấn số lớn thứ k trên đoạn động;
2. bài toán duy trì thứ tự của một dãy.

Tiếp theo là cây cân bằng theo trọng lượng bền vững. Phần cuối là trọng tâm của bài viết: dùng cây cân bằng theo trọng lượng để duy trì cấu trúc cây cân bằng hậu tố, rồi giải một số bài toán.

12.2 Khái niệm cây cân bằng theo trọng lượng

Cây cân bằng thông thường phụ thuộc vào phép quay. Khi ta chỉ thống kê một số thông tin có thể gộp nhanh, phép quay không gây vấn đề gì. Nhưng nếu thông tin cần duy trì phức tạp hơn, độ phức tạp sẽ trở thành trở ngại.

Hình minh họa trong nguồn cho thấy một phép quay cục bộ có thể làm thay đổi toàn bộ thông tin của một cây con. Hãy xét một ví dụ đơn giản: ta muốn tại mỗi nút của cây cân bằng duy trì một tập hợp, chứa tất cả các số nằm trong cây con của nút đó. Khi ấy chi phí của một phép quay có thể đạt tới $O(n)$, và cây cân bằng quay truyền thống không còn phát huy tác dụng.

Cây cân bằng theo trọng lượng thì khác. Trong mỗi thao tác, kích thước cây con lớn nhất bị ảnh hưởng chỉ là $O(\log n)$ trong trường hợp xấu nhất, theo nghĩa ngẫu nhiên, hoặc theo kỳ vọng. Vì vậy, ngay cả khi duy trì thông tin theo cách nói trên, độ phức tạp vẫn có thể chấp nhận được.

12.2.1 Một số cây cân bằng theo trọng lượng không dùng phép quay

Một số cây cân bằng, hoặc cấu trúc dữ liệu có thể thực hiện các thao tác của cây cân bằng truyền thống, không phụ thuộc vào cơ chế quay. Nhờ đó chúng không chịu ảnh hưởng bởi điểm yếu cục bộ của phép

quay.

Bài viết chọn hai cấu trúc có giá trị thực tiễn nhất định trong OI để giới thiệu.

Skip list. Skip list đã được các đàn anh đội tuyển quốc gia trình bày chi tiết trong luận văn trước đây, nên bài viết này không nhắc lại. Bạn đọc quan tâm có thể xem tài liệu *Segment skip list: một mở rộng của skip list* trong phần tham khảo.

Scapegoat tree. Scapegoat tree dựa trên thao tác dựng lại thô bạo.

Ta định nghĩa một hệ số cân bằng a . Với mỗi nút t trong scapegoat tree, ta cần thỏa mãn

$$\max(\text{size}_l, \text{size}_r) \leq a \cdot \text{size}_t,$$

trong đó l, r lần lượt chỉ cây con trái và cây con phải.

Cách cài đặt rất ngắn gọn: sau mỗi thao tác, tìm nút cao nhất trên đường đi lên mà không còn thỏa mãn tính chất cân bằng, rồi dựng lại cây con gốc tại nút đó thành một cây nhị phân hoàn toàn.

Độ phức tạp của scapegoat tree. Xét một cây con t vừa được dựng lại. Qua tính toán, có thể thấy phải chèn thêm ít nhất $\Omega(|t|)$ nút vào cây con đó thì nó mới bị dựng lại lần nữa; ở đây chỉ xét việc dựng lại toàn bộ cây con, không xét việc dựng lại ở các hậu duệ.

Mặt khác, chi phí dựng lại là $|t|$. Ta có thể dùng thế năng để phân tích: mỗi khi chèn một nút, tiện thể cộng 1 đơn vị thế năng cho tất cả tổ tiên của nó. Khi một nút bị dựng lại, trên nút đó sẽ có đủ thế năng để trả cho việc dựng lại. Do đó độ phức tạp khấu hao của phép chèn là $O(\log n)$.

Phân tích cho phép xóa về cơ bản tương tự, xin để bạn đọc tự suy nghĩ.

Đồng thời, từ tính chất cân bằng ta chứng minh được chiều cao của scapegoat tree là $O(\log_{1/a} n)$. Vì vậy thao tác tìm kiếm hiển nhiên cũng có độ phức tạp $O(\log n)$.

Chọn hệ số cân bằng a . Ta có thể nhận thấy khi hệ số cân bằng a lớn hơn một chút, số lần dựng lại giảm đi, do đó thời gian chèn giảm; nhưng chiều cao cây cũng lớn hơn, nên thời gian tìm kiếm tăng.

Khi a nhỏ hơn, số lần dựng lại tăng, do đó thời gian chèn tăng; nhưng chiều cao cây giảm, nên thời gian tìm kiếm cũng giảm. Thông thường chọn a khoảng 0.6 tới 0.7 là đủ đáp ứng phần lớn nhu cầu.

Ưu và nhược điểm của scapegoat tree. Scapegoat tree tuy không dựa trên cơ chế quay, nhưng ý tưởng rất rõ ràng, lượng mã rất nhỏ, tốc độ cũng không chậm. Về độ phức tạp thời gian, độ phức tạp cài đặt và độ phức tạp tư duy, nó đều không thua các cây cân bằng truyền thống như Treap và Splay. Trong thi đấu thực tế, dùng nó là một lựa chọn rất hợp lý.

Dĩ nhiên, do hạn chế của cơ chế cân bằng, scapegoat tree không hỗ trợ được một số thao tác rất phức tạp, chẳng hạn thao tác trích riêng một đoạn thường gặp trong Splay. Đồng thời, vì nó là một cấu trúc khấu hao bằng thế năng, ta cũng không thể bền vững hóa nó một cách đơn giản. Ngoài ra, khi cài đặt cần lưu trường size để biểu diễn kích thước cây con, nhưng trong OI điều này không phải vấn đề lớn.

Nhìn chung, scapegoat tree rất xuất sắc trong các ứng dụng cây cân bằng đơn giản.

12.2.2 Cây cân bằng theo trọng lượng dùng phép quay

Ngay cả khi dùng cơ chế quay, cũng có rất nhiều cây cân bằng là cân bằng theo trọng lượng. Có thể hiểu như sau: phần lớn phép quay chỉ là những “điều chỉnh nhỏ”, xảy ra ở tầng thấp, nên cây con liên quan không quá lớn; phép quay ở tầng cao thì xảy ra ít hơn. Vì vậy độ phức tạp khấu hao hoặc kỳ vọng tổng thể vẫn là $O(\log n)$.

Treap. Treap là một cấu trúc dữ liệu rất thường gặp.

Đúng như tên gọi, Treap = Tree + Heap. Nó duy trì một khóa ngẫu nhiên cho mỗi nút, đồng thời bảo đảm hình dạng của cây giống như cây BST thường nhận được khi chèn các nút theo thứ tự tăng dần của khóa ngẫu nhiên đó.

Ta đều biết rằng với dữ liệu ngẫu nhiên, chiều cao kỳ vọng của BST thường là $O(\log n)$. Vì vậy chiều cao kỳ vọng của Treap cũng là $O(\log n)$, và không phụ thuộc vào thứ tự dữ liệu vào. Các thao tác của Treap đều là thuật toán kinh điển, nên ở đây không nhắc lại.

Tính cân bằng theo trọng lượng của Treap. Khi chèn một số, có thể phát sinh một số phép quay. Giả sử nút mới được quay lên tới vị trí của tổ tiên k . Khi đó ta cần dùng thời gian size_k để dựng lại thông tin. Nhưng tình huống này xảy ra chỉ khi khóa ngẫu nhiên của nút mới là nhỏ nhất trong cây con của k , với xác suất $1/\text{size}_k$. Vì vậy chi phí kỳ vọng là 1. Đồng thời, một nút có nhiều nhất $O(\log n)$ tổ tiên theo kỳ vọng, nên tổng chi phí kỳ vọng của một lần chèn là $O(\log n)$.

Tiếp theo xét xóa một số x . Ta có thể trực tiếp dựng lại cây con gốc tại nút đó. Vì một nút có $O(\log n)$ tổ tiên theo kỳ vọng, tổng số quan hệ (tổ tiên, con) là $O(n \log n)$; nói cách khác, tổng kích thước cây con của mọi nút có kỳ vọng $O(n \log n)$. Do đó kích thước cây con của một nút có kỳ vọng $O(\log n)$, và độ phức tạp xóa một số là $O(\log n)$ theo kỳ vọng.

12.3 Ứng dụng của cây cân bằng theo trọng lượng

Ta hãy xem một vài ứng dụng của cây cân bằng theo trọng lượng để nhấn mạnh vai trò của chúng.

12.3.1 Bài toán duy trì thứ tự của một dãy

Mô tả bài toán. Ta có một dãy nút và cần hỗ trợ hai thao tác:

1. chèn một nút mới y vào sau nút x ;
2. hỏi quan hệ trước sau của hai nút a, b .

Cách 1. Ta có thể dùng một cây cân bằng đơn giản để duy trì dãy này. Khi chèn thì chèn trực tiếp. Khi hỏi quan hệ lớn nhỏ, chỉ cần tìm rank của từng nút rồi so sánh.

Độ phức tạp chèn là $O(\log n)$. Độ phức tạp truy vấn là $O(\log n)$.

Cách 2. Ta vẫn dùng cây cân bằng để duy trì dãy, nhưng khác ở chỗ mỗi nút được gán một nhãn tag_i . Khi hỏi quan hệ trước sau của a, b , chỉ cần so sánh tag_a và tag_b .

Làm thế nào để duy trì nhãn này? Ta cho mỗi nút tương ứng với một khoảng số thực. Gốc tương ứng với khoảng $(0, 1)$. Nếu nút i tương ứng với khoảng (l, r) , đặt

$$\text{tag}_i = \frac{l + r}{2}.$$

Cây con trái của nó tương ứng với (l, tag_i) , cây con phải tương ứng với (tag_i, r) . Dễ thấy các nhãn như vậy thỏa yêu cầu thứ tự.

Khi so sánh, ta chỉ cần so sánh các giá trị tag . Đồng thời, mẫu số của tag_i là $2^{\text{depth}_i + 1}$, trong đó depth_i là độ sâu của i , tức khoảng cách từ i tới gốc. Vì vậy chỉ cần cây cân bằng, tức độ sâu lớn nhất không quá lớn, thì độ chính xác được bảo đảm; dùng kiểu double là đủ.

Khi chèn, có thể phải tính lại giá trị tag cho cả một cây con. Miễn dùng cây cân bằng theo trọng lượng, thao tác này vẫn đạt độ phức tạp $O(\log n)$. Độ phức tạp truy vấn là $O(1)$.

Cách 3. Đây là một bài toán kinh điển. Huang Jiatai đã tra cứu luận văn và phát hiện một thuật toán rất tốt, trong đó cả chèn lẫn truy vấn đều là $O(1)$. Tuy vậy nó không liên quan tới chủ đề của bài viết. Bạn đọc quan tâm có thể xem tài liệu *Two Simplified Algorithms for Maintaining Order in a List* trong phần tham khảo.

12.3.2 Truy vấn số lớn thứ k trên đoạn động

Mô tả bài toán. Ta có một dãy s độ dài n , cần hỗ trợ các thao tác:

1. chèn số x vào sau số thứ i ;
2. sửa s_i thành x ;
3. xóa số thứ i ;
4. hỏi số lớn thứ k trong đoạn $[l, r]$.

Cách 1. Ta dùng cây cân bằng theo trọng lượng để duy trì dãy. Tại mỗi nút, dùng một cây cân bằng khác để lưu mọi số trong cây con của nút đó.

Khi đó chèn, xóa và sửa đều có thể xử lý trong $O(\log^2 n)$. Với truy vấn, ta nhị phân đáp án a , rồi trong $O(\log^2 n)$ trả lời có bao nhiêu số trong đoạn nhỏ hơn a . Tổng độ phức tạp là $O(\log^3 n)$.

Cách 2. Ta có thể tại mỗi nút duy trì một cây đoạn theo miền giá trị, từ đó cải thiện độ phức tạp xuống $O(\log^2 n)$. Về cây đoạn theo miền giá trị, có thể xem luận văn năm trước của tác giả, *Nghiên cứu cấu trúc dữ liệu bền vững*, trong đó có phần giải thích chi tiết.

Cách 3. Ta đưa ra một cách làm khác khá thú vị: mở một cây đoạn chung trên toàn bộ miền giá trị. Tại nút ứng với khoảng giá trị $[l, r]$, lưu một cây cân bằng gồm vị trí của tất cả các số có giá trị nằm trong $[l, r]$.

Khi trả lời truy vấn, ta đi một lượt trên cây đoạn miền giá trị; độ phức tạp là $O(\log^2 n)$. Khi chèn một số, ta cần chèn số đó vào dãy, đồng thời chèn nó vào các nút của cây đoạn chứa giá trị đó.

Lúc này cần chú ý: vì đã chèn thêm một số, vị trí của tất cả các số phía sau đều tăng 1. Hiển nhiên ta không thể trực tiếp sửa toàn bộ vị trí của chúng. Nhưng thực ra ta không cần vị trí chính xác; trong cây cân bằng, ta chỉ cần có thể so sánh nhanh quan hệ trước sau giữa hai vị trí.

Do đó ta dùng lời giải của bài toán duy trì thứ tự dãy để duy trì dãy này. Như vậy ta có thể so sánh nhanh quan hệ trước sau của hai số, từ đó hoàn thành các thao tác trên cây cân bằng. Độ phức tạp các thao tác khác cũng là $O(\log^2 n)$.

12.3.3 K-D tree động

K-D tree là một cấu trúc dữ liệu hình học kinh điển và hỗ trợ nhiều thao tác. Trước hết, ta nhắc lại ngắn gọn khái niệm K-D tree.

K-D tree dựa trên việc liên tục chia tập điểm trên mặt phẳng. Mỗi lần, ta sắp xếp tất cả các điểm theo trục x hoặc trục y , rồi chia thành hai phần trái-phải hoặc trên-dưới càng đều càng tốt, sau đó đệ quy chia tiếp. Các trục chia được dùng luân phiên: trước hết trục x , rồi trục y , rồi tiếp tục như vậy.

Giả sử hiện tại ta đã có một K-D tree và cần hỗ trợ thao tác chèn một điểm. Nếu tìm thô bạo vị trí rồi chèn trực tiếp, tính chất của K-D tree sẽ bị phá vỡ. Ta có thể mô phỏng tư tưởng của scapegoat tree: đặt một hệ số cân bằng a , yêu cầu phần lớn nhất sau mỗi phép chia không vượt quá a nhân với tổng kích thước. Như vậy có thể bảo đảm độ phức tạp của chèn và truy vấn.

12.3.4 MAXPATH

Đây là một bài gốc của tác giả.

Mô tả bài toán. Tìm một đường đi dài nhất. Ràng buộc $n \leq 50000$.

Cách 1. Bài này có một lời giải $O(n \log^3 n)$ dựa trên chia để trị và đồ thị Voronoi. Hằng số và lượng mã đều rất lớn. Vì không liên quan tới chủ đề bài viết, tác giả không trình bày thêm; bạn đọc quan tâm có thể xem bài giảng WC2013 của Gu Xizhou.

Cách 2. Trước hết ta làm DP, đặt dp_i là độ dài chuỗi dài nhất kết thúc tại i . Khi đó phương trình DP là

$$dp_i = \max dp_j \quad (j < i, |p_i p_j| < R_i).$$

Xét công thức này, ý tưởng đầu tiên là duy trì một cấu trúc dữ liệu hỗ trợ chèn một điểm có trọng số và hỏi trọng số lớn nhất trong một hình tròn. Nhưng đây là bài rất khó. Ta đổi cách nghĩ: sắp xếp toàn bộ $j < i$ trước đó theo thứ tự giá trị dp_j .

Sau đó, lần lượt từ lớn tới nhỏ tìm phần tử đầu tiên có khoảng cách tới i nhỏ hơn R_i . Cách này vẫn có độ phức tạp $O(n^2)$, nhưng lúc này bài toán đã có thể dùng những cấu trúc dữ liệu nói ở trên.

Ta dùng cây cân bằng theo trọng lượng để duy trì thứ tự của mọi j theo dp_j . Trên mỗi nút, duy trì một K-D tree để lưu tất cả các điểm nằm trong cây con của nút đó.

Khi trả lời truy vấn, ta hỏi trên K-D tree xem khoảng cách từ điểm gần nhất bên trong nó tới i có nhỏ hơn R_i hay không; nhờ vậy biết cần đi sang con nào. Sau khi tính được dp_i , ta chèn trực tiếp nó vào cây cân bằng. Vì cũng cần chèn vào K-D tree, ta dùng K-D tree động đã nói ở trên.

Chi phí hỏi điểm gần nhất trên K-D tree có thể xem là $O(\sqrt{n})$, nên tổng độ phức tạp là $O(n^{1.5} \log n)$. Tuy độ phức tạp cao hơn cách 1, hằng số và độ phức tạp mã lệnh lại nhỏ hơn rất nhiều; lượng mã và độ phức tạp của đồ thị Voronoi lớn hơn K-D tree đáng kể.

12.4 Cây cân bằng theo trọng lượng bền vững

Ta xét cây cân bằng theo trọng lượng bền vững. Vì bản thân Treap đã cân bằng theo trọng lượng, ta chỉ cần cài đặt một Treap bền vững.

Về các chi tiết của Treap bền vững, có thể tham khảo luận văn năm trước của tác giả, *Nghiên cứu cấu trúc dữ liệu bền vững*; ở đây không nhắc lại. Độ phức tạp thời gian và không gian đều là $O(\log n)$ theo kỳ vọng cho mỗi bước.

Scapegoat tree là một cấu trúc khá hao. Nếu ta chỉ cần bền vững hóa một phần, tức chỉ được sửa cấu trúc hiện tại và chỉ truy vấn các cấu trúc lịch sử, thì có thể dùng cách bền vững hóa cây cân bằng đơn giản để thực hiện.

Nhưng nếu muốn bền vững hóa hoàn toàn, tức cũng hỗ trợ sửa trên cấu trúc lịch sử, thì vì độ phức tạp của scapegoat tree là $O(\log n)$ theo nghĩa khá hao, một thao tác chi phí cao có thể bị thực hiện nhiều lần và phá vỡ bảo đảm độ phức tạp.

12.5 Cây cân bằng hậu tố

12.5.1 Định nghĩa cây cân bằng hậu tố

Xét một chuỗi s độ dài n . Định nghĩa S_i là hậu tố gồm $s_i, s_{i+1}, \dots, s_n$. Thứ tự giữa các hậu tố được định nghĩa bằng thứ tự từ điển. Cây cân bằng hậu tố là một cây cân bằng duy trì thứ tự của các hậu tố này.

12.5.2 Xây dựng cây cân bằng hậu tố

Ta lần lượt xét cách xây dựng cây cân bằng hậu tố từ góc độ ngoại tuyến và trực tuyến.

Thuật toán ngoại tuyến. Cho một chuỗi s , trước hết ta tìm mảng hậu tố, tức thứ tự sắp xếp của tất cả hậu tố. Từ mảng hậu tố, ta dễ dàng xây dựng được cây cân bằng hậu tố.

Độ phức tạp là $O(n)$ hoặc $O(n \log n)$, tùy thuật toán xây dựng mảng hậu tố được dùng.

Thuật toán trực tuyến. Giống cây hậu tố và tự động hóa hậu tố, cây cân bằng hậu tố cũng có một thuật toán trực tuyến thêm từng ký tự một, nên có thể giải một số vấn đề mà chỉ dùng mảng hậu tố thì không giải được.

Trái với cây hậu tố và tự động hóa hậu tố, thuật toán trực tuyến của cây cân bằng hậu tố không thêm ký tự ở cuối chuỗi, mà thêm ở đầu chuỗi.

Xét chuỗi hiện tại S , và thêm ký tự c vào đầu nó. Khi đó thực chất tương đương với việc chèn một hậu tố mới cS vào cây cân bằng hậu tố. Nếu ta so sánh nhanh được thứ tự của hai hậu tố, chỉ cần áp dụng trực tiếp thuật toán chèn của cây cân bằng.

Hash. Trước hết, ta có thể dùng phương pháp hash kinh điển: tiền xử lý hash của chuỗi, rồi dùng tìm kiếm nhị phân để tìm LCP của hai hậu tố trong $O(\log n)$, từ đó so sánh thứ tự của chúng.

Khi đó độ phức tạp thời gian là $O(n \log^2 n)$, độ phức tạp mã lệnh rất nhỏ và tương đối dễ viết.

Áp dụng bài toán duy trì thứ tự dãy. Chú ý rằng nếu dùng lời giải của bài toán duy trì thứ tự dãy đã nói ở trên, ta có thể so sánh hai hậu tố trong $O(1)$.

Như vậy độ phức tạp trở thành $O(n \log n)$, cải thiện lớn so với phương pháp hash.

Bền vững hóa. Ta xét bền vững hóa cây cân bằng hậu tố. Miễn dùng Treap, ta rất dễ mở rộng thuật toán thành thuật toán bền vững, từ đó có được cây cân bằng hậu tố bền vững.

Khi đó độ phức tạp thời gian không đổi, còn độ phức tạp không gian trở thành $O(n \log n)$.

12.5.3 Ưu thế của cây cân bằng hậu tố

Trước hết, về mặt khái niệm, cây cân bằng hậu tố chỉ là một ứng dụng tổng hợp của mảng hậu tố và cây cân bằng, nên dễ hiểu hơn tự động hóa hậu tố và cây hậu tố phức tạp.

Độ phức tạp mã lệnh thì lớn hơn một chút, nhưng ý tưởng rất rõ ràng và không dễ viết sai. Đồng thời, so với tự động hóa hậu tố và cây hậu tố, nó còn hỗ trợ thêm một số thao tác khác.

Chẳng hạn, tự động hóa hậu tố và cây hậu tố đều có thể thêm một ký tự ở cuối, nhưng chúng không hỗ trợ xóa một ký tự ở cuối. Ngược lại, cây cân bằng hậu tố có thể thực hiện thao tác này.

Cây cân bằng hậu tố cũng có thể hỗ trợ duy trì động tất cả hậu tố của một Trie, điều mà tự động hóa hậu tố không thể làm hiệu quả. Ngoài ra, cây cân bằng hậu tố hoàn toàn không phụ thuộc vào kích thước bảng chữ cái, trong khi các cài đặt đơn giản của tự động hóa hậu tố và cây hậu tố đều phải xét yếu tố này.

Cuối cùng, cây cân bằng hậu tố có thể bền vững hóa; đây là điều tự động hóa hậu tố và cây hậu tố khó đạt tới. Tiếp theo ta dùng một vài ví dụ để cho thấy tác dụng của cây cân bằng hậu tố.

12.6 Ứng dụng của cây cân bằng hậu tố

12.6.1 COT4 online

Bài này được sửa từ bài kinh điển COT4 của Huang Jiatai; so với bài gốc, có thể xem đây là một phiên bản yếu hơn.

Ý chính đề bài. Cho một tập chuỗi S , ban đầu chỉ chứa một chuỗi rỗng. Cần hỗ trợ các thao tác:

1. lấy một chuỗi $s \in S$, xét một ký tự c , rồi đưa cả sc và s vào tập S ;
2. với một chuỗi truy vấn q và một chuỗi $s \in S$, in số lần q xuất hiện như một chuỗi con của s .

Số thao tác $n \leq 200000$, tổng độ dài các chuỗi truy vấn $Q \leq 500000$.

Lời giải. Ta chú ý rằng đưa sc vào S thực chất là thêm một ký tự vào sau s . Ta dùng cây cân bằng hậu tố bền vững để duy trì mọi hậu tố của s ; khi thêm ký tự, thực hiện một lần chèn bền vững.

Tiếp theo là hỏi một chuỗi q xuất hiện bao nhiêu lần. Thực ra đây chính là số hậu tố có thứ tự từ điển nhỏ hơn $q\#$ trừ đi số hậu tố nhỏ hơn q , trong đó $\#$ là một ký tự ảo lớn nhất. Vì vậy ta chỉ cần tìm kiếm nhị phân và so sánh trên cây cân bằng; độ phức tạp là $O(|q| \log n)$, chấp nhận được. Nếu duy trì thêm một số thông tin phụ, có thể cải thiện độ phức tạp truy vấn thành $O(|q| + \log n)$, nhưng cách đó phức tạp hơn.

Độ phức tạp mỗi thao tác cập nhật là $O(\log n)$, nên tổng độ phức tạp là $O((n + Q) \log n)$.

12.6.2 STRQUERY

Đây là một bài gốc của tác giả.

Ý chính đề bài. Ta có một chuỗi s , cần hỗ trợ bốn loại thao tác:

1. chèn hoặc xóa một ký tự ở đầu trái;
2. chèn hoặc xóa một ký tự ở đầu phải;
3. chèn hoặc xóa một ký tự ở chính giữa, tại vị trí $|s|/2$;
4. hỏi chuỗi q xuất hiện bao nhiêu lần.

Tổng cộng có n thao tác, với $n \leq 150000$; tổng độ dài các chuỗi truy vấn không vượt quá 1500000.

Lời giải. Trước hết, cây cân bằng hậu tố của ta hỗ trợ chèn hoặc xóa một ký tự ở đầu trái. Gọi cấu trúc này là TL .

Nếu đảo ngược toàn bộ chuỗi và cũng đảo ngược chuỗi truy vấn, ta sẽ hỗ trợ được chèn hoặc xóa một ký tự ở đầu phải. Gọi cấu trúc này là TR .

Bây giờ ta xét cách xây dựng một cấu trúc hỗ trợ chèn và xóa ở hai đầu. Ta duy trì một cấu trúc TL và một cấu trúc TR , đặt

$$TB = TL + TR.$$

Các thao tác chèn, xóa lần lượt thực hiện trên TL và TR . Nếu một trong hai cấu trúc bị xóa tới 0 ký tự, ta chia đều cấu trúc còn lại thành hai nửa; theo nghĩa khẩu hao, thao tác này không ảnh hưởng độ phức tạp.

Khi truy vấn trên TB , ta trước hết hỏi q xuất hiện bao nhiêu lần trong TL và TR , sau đó hỏi nó xuất hiện bao nhiêu lần trong phần vượt qua ranh giới. Độ dài phần vượt qua ranh giới nhiều nhất là $2|q| - 2$, nên chỉ cần lấy trực tiếp đoạn đó ra rồi chạy KMP.

Tiếp theo, ta xây dựng một cấu trúc TM hỗ trợ chèn ở chính giữa. Ta duy trì hai cấu trúc TB , gọi là l, r . Sau mỗi lần chèn, ta trao đổi vài ký tự giữa l và r , sao cho vị trí ngay sau l luôn là chính giữa. Như vậy có thể chèn ở chính giữa. Truy vấn được xử lý giống như truy vấn trên TB .

Do đó ta thu được một cấu trúc TM hỗ trợ bài toán này, với độ phức tạp $O(n \log n)$.

12.7 Lời cảm ơn

- Xin cảm ơn Huang Jiatai đã giúp đỡ tôi.
- Xin cảm ơn CCF đã cung cấp cơ hội quý báu này.
- Xin cảm ơn các huấn luyện viên đã chỉ dẫn.

12.8 Tài liệu tham khảo

1. Liu Wenjia, Huang Liang. *Nghệ thuật thuật toán và thi tin học*. Nhà xuất bản Đại học Thanh Hoa.
2. Liu Rujia. *Nhập môn kinh điển về thi thuật toán*. Nhà xuất bản Đại học Thanh Hoa.
3. Michael A. Bender, Richard Cole, Erik D. Demaine, Martin Farach-Colton, and Jack Zito. *Two Simplified Algorithms for Maintaining Order in a List*.
4. BHA. *Segment skip list: một mở rộng của skip list*. Luận văn đội tuyển quốc gia năm 2009.
5. Chen Lijie. *Nghiên cứu cấu trúc dữ liệu bền vững*. Bài tập đội tuyển quốc gia năm 2012.

13 Bàn về bài toán đếm trên vòng

Gao Shenghan

Trường Trung học cao cấp Thường Châu, Giang Tô

13.1 Tóm tắt

Trong toán học và tin học, ta thường gặp các bài toán đếm; trong đó có rất nhiều bài là bài toán đếm trên vòng. Khi giải loại bài này, ta thường dùng bổ đề Burnside và định lý Polya, đồng thời phải phối hợp với các thuật toán khác như quy hoạch động hoặc truy hồi. Bài viết này phân tích ngắn gọn các công cụ đó và tổng kết một số kinh nghiệm để tham khảo.

13.2 Dẫn nhập

Trong các bài đếm trên vòng của tin học, điều kiện “sau khi quay hoặc lật mà giống nhau thì xem là cùng một trường hợp” xuất hiện rất thường xuyên. Ví dụ:

Một dãy vòng gồm n phần tử, mỗi phần tử có thể tô một trong các màu $1, 2, \dots, m$. Hai dãy nhận được từ nhau bằng phép quay được xem là cùng một phương án. Hỏi tổng số phương án là bao nhiêu?

Với các bài như vậy, phần đếm cơ bản thường có thể xử lý bằng tổ hợp hoặc truy hồi. Tuy nhiên khi có phép quay và phép lật, ta cần suy luận thêm và thường phải tối ưu công thức. Trong phần lập luận toán học, bổ đề Burnside và định lý Polya là hai công cụ rất hữu ích. Trước hết ta nhắc lại các khái niệm cần dùng.

13.3 Cơ sở lý thuyết

13.3.1 Phép toán

Với một tập không rỗng S , một phép toán trên S là một ánh xạ từ $S \times S$ vào S , ký hiệu là $*$. Ta viết $a * b$, hoặc ngắn gọn là ab ; tập S cùng phép toán này được ký hiệu là $(S, *)$.

Các tính chất thường dùng gồm:

1. Nếu với mọi $a, b \in A$ đều có $a * b \in A$, ta nói A đóng dưới phép toán $*$.

2. Nếu với mọi $a, b, c \in A$ đều có

$$(a * b) * c = a * (b * c),$$

ta nói phép toán thỏa luật kết hợp trên A .

3. Nếu với mọi $a, b \in A$ đều có $a * b = b * a$, ta nói phép toán thỏa luật giao hoán trên A .

4. Nếu $e \in S$ và với mọi $a \in S$ đều có $a * e = e * a = a$, thì e là phần tử đơn vị của $*$.

5. Nếu $a, b \in S$ và $a * b = b * a = e$, thì b là phần tử nghịch đảo của a dưới $*$, ký hiệu là a^{-1} .

6. Nếu với mọi $a, b, c \in A$, từ $a * b = a * c$ suy ra $b = c$, ta nói phép toán thỏa luật khử trái trên A .

13.3.2 Nhóm

Gọi G là một tập không rỗng có phép toán hai ngôi. Nếu G thỏa:

1. phép toán đóng trên G ;
2. phép toán thỏa luật kết hợp;
3. phần tử đơn vị thuộc G ;
4. với mọi phần tử của G , nghịch đảo của nó cũng thuộc G ,

thì G là một nhóm. Số phần tử của G được ký hiệu là $|G|$ và gọi là bậc của nhóm; từ đó ta có nhóm hữu hạn và nhóm vô hạn.

13.3.3 Hoán vị

Một hoán vị σ là song ánh từ tập $\{1, 2, \dots, n\}$ vào chính nó. Ta thường viết

$$\sigma = \begin{pmatrix} 1 & 2 & 3 & \cdots & n \\ a_1 & a_2 & a_3 & \cdots & a_n \end{pmatrix}.$$

Ở đây $a_1, a_2, \dots, a_n$ là một hoán vị của $1, 2, \dots, n$, và mỗi cột mô tả một ánh xạ độc lập.

Tập tất cả hoán vị trên n phần tử được ký hiệu là S_n , có bậc $n!$. Không khó kiểm tra rằng S_n là một nhóm, gọi là nhóm đối xứng bậc n . Một nhóm hoán vị là một nhóm con của nhóm đối xứng; mọi phần tử của nó đều là một phép hoán vị. Nhóm này chứa phần tử đơn vị

$$\begin{pmatrix} 1 & 2 & 3 & \cdots & n \\ 1 & 2 & 3 & \cdots & n \end{pmatrix}.$$

Nghịch đảo của hoán vị

$$\begin{pmatrix} 1 & 2 & 3 & \cdots & n \\ a_1 & a_2 & a_3 & \cdots & a_n \end{pmatrix}$$

là

$$\begin{pmatrix} a_1 & a_2 & a_3 & \cdots & a_n \\ 1 & 2 & 3 & \cdots & n \end{pmatrix}.$$

Hoán vị có thể nhân bằng phép hợp thành ánh xạ. Vì mỗi hoán vị có nghịch đảo duy nhất, nếu a, b, c là hoán vị và $a * b = a * c$, thì $a^{-1} * (a * b) = a^{-1} * (a * c)$, suy ra $b = c$. Do đó phép nhân hoán vị thỏa luật khử.

13.3.4 Bổ đề Burnside

Ta dùng hai khái niệm sau.

1. Với nhóm hoán vị G tác động lên tập X , gọi Z_k là tập các phép hoán vị trong G giữ nguyên phần tử k . Đây là lớp hoán vị bất động của k .
2. Nếu tồn tại $g \in G$ biến phần tử k thành phần tử l , thì k và l thuộc cùng một lớp tương đương. Lớp tương đương chứa k được ký hiệu là E_k .

Trong các bài toán cụ thể, ta thường biết nhóm hoán vị và muốn đếm số lớp tương đương trên tập các cấu hình. Lớp hoán vị bất động thường dễ tính hơn, nên ta sẽ biểu diễn số lớp tương đương qua đại lượng đó.

Với mọi k , ta có

$$|E_k| \cdot |Z_k| = |G|.$$

Lý do như sau. Giả sử $E_k = \{a_1, a_2, \dots, a_t\}$. Với mỗi a_j , chọn một phép hoán vị $p_j \in G$ đưa k tới a_j . Nhờ tính đóng và luật khử của phép nhân hoán vị, tập $p_j Z_k$ liệt kê đúng các hoán vị đưa k tới a_j , không lặp và không sót. Lấy tổng theo j thu được công thức trên; đồng thời nếu $i, j \in E_k$ thì $|Z_i| = |Z_j|$.

Gọi l là số lớp tương đương. Tổng các số phần tử bất động có thể tính theo hai cách:

$$\sum_{x \in X} |Z_x| = \sum_{\text{các lớp } E} \sum_{x \in E} |Z_x| = l|G|.$$

Mặt khác, nếu X_g là tập các cấu hình không đổi sau khi áp dụng g , thì

$$\sum_{g \in G} |X_g| = \sum_{x \in X} |Z_x|.$$

Do đó ta nhận được bổ đề Burnside:

$$l = \frac{1}{|G|} \sum_{g \in G} |X_g|. \quad (11)$$

13.4 Các trường hợp đặc biệt

Phép quay và phép lật trong phần dẫn nhập chính là các trường hợp đặc biệt của bổ đề Burnside. Ta xét cụ thể cách suy ra công thức và cách dùng chúng.

13.4.1 Phép quay

Với phép quay một đơn vị, ta có hoán vị

$$\sigma = \begin{pmatrix} 1 & 2 & \cdots & n-1 & n \\ 2 & 3 & \cdots & n & 1 \end{pmatrix}.$$

Quay k đơn vị tương ứng với

$$\begin{pmatrix} 1 & 2 & 3 & \cdots & n \\ 1+k & 2+k & 3+k & \cdots & n+k \end{pmatrix}, \quad (12)$$

trong đó chỉ số được lấy theo modulo n , tức $n+1 = 1, n+2 = 2, \dots$. Các phép quay này tạo thành một nhóm hoán vị bậc n .

Định lý Polya phát biểu rằng: nếu G là nhóm hoán vị của n đối tượng, mỗi đối tượng có thể tô bằng m màu, thì số cách tô màu không tương đương là

$$l = \frac{1}{|G|} \left(m^{c(g_1)} + m^{c(g_2)} + \dots + m^{c(g_{|G|})} \right), \quad (13)$$

trong đó $c(g_i)$ là số chu trình của hoán vị g_i . Định lý này giải quyết trực tiếp nhiều bài tô màu vòng.

Từ Burnside, với bài toán quay vòng ta có

$$l = \frac{1}{n} \sum_{i=1}^n X_{\sigma^i}.$$

Ở đây X_{σ^i} là số cấu hình không đổi sau khi quay i ô. Các cấu hình đó chính là các dãy có chu kỳ $\gcd(n, i)$. Nếu gọi $f[k]$ là số dãy tuyến tính độ dài k thỏa điều kiện nội bộ, thì

$$l = \frac{1}{n} \sum_{i=1}^n f[\gcd(n, i)].$$

Nhóm các hạng tử theo $k = \gcd(n, i)$ cho công thức tối ưu hơn:

$$l = \frac{1}{n} \sum_{k|n} f[k] \cdot \varphi\left(\frac{n}{k}\right), \quad (14)$$

với φ là hàm Euler.

Ngay cả khi chưa biết Burnside và Polya, ta cũng có thể hiểu công thức này theo chu kỳ. Khi đếm dãy vòng độ dài n bằng cách cắt vòng tại từng vị trí, một vòng thường bị đếm n lần. Nhưng nếu vòng có chu kỳ ngắn nhất là k , thì các cách cắt chỉ tạo ra k dãy tuyến tính khác nhau. Vì vậy các cấu hình có chu kỳ ngắn cần được cộng bù. Liệt kê mọi ước k của n , tính số dãy có chu kỳ ngắn nhất đúng bằng k , rồi dùng bao hàm-loại trừ hoặc hàm Mobius, ta cũng đi tới đúng công thức (14). Bản chất của cách làm này vẫn là lập luận tương tự định lý Polya.

13.4.2 Phép lật

Phép lật một dãy dài n có thể viết dưới dạng hoán vị

$$\begin{pmatrix} 1 & 2 & 3 & \cdots & n \\ n & n-1 & n-2 & \cdots & 1 \end{pmatrix}. \quad (15)$$

Đây là phép đảo ngược thứ tự, tức một trường hợp rất đặc biệt của phép hoán vị. Nếu chỉ xét một chuỗi tuyến tính dưới quan hệ đảo ngược, thông thường mỗi chuỗi được đếm đúng hai lần: chuỗi gốc và chuỗi đảo. Riêng chuỗi đối xứng palindrome chỉ được đếm một lần, nên cần cộng thêm số trường hợp palindrome.

Một palindrome độ dài n được xác định bởi nửa đầu của nó. Nếu n chẵn, độ dài phần quyết định là $n/2$; nếu n lẻ, độ dài này là $(n+1)/2$. Vì vậy, nếu $f[i]$ là số chuỗi tuyến tính độ dài i thỏa điều kiện, thì

$$l = \frac{1}{2} \left(f[n] + f\left[\frac{n + \text{odd}(n)}{2}\right] \right), \quad (16)$$

trong đó $\text{odd}(n) = 1$ khi n lẻ và bằng 0 khi n chẵn.

13.4.3 Lật rồi quay

Trường hợp vừa cho phép quay vừa cho phép lật sẽ được phân tích trong các bài cụ thể ở phần sau.

13.5 Ví dụ

13.5.1 Codeforces 93D Flags

Ý chính đề bài. Cho một dãy độ dài n , tô bằng bốn màu: trắng, vàng, đỏ, đen. Hai vị trí kề nhau phải khác màu; trắng và vàng không được kề nhau; đỏ và đen không được kề nhau; đồng thời không được xuất hiện một bộ ba liên tiếp đen, đỏ, trắng. Hai dãy nhận được từ nhau bằng phép lật được xem là cùng một loại. Với truy vấn $[l, r]$, hỏi có bao nhiêu loại dãy có độ dài trong đoạn đó, lấy modulo để xuất kết quả. Ràng buộc $1 \leq l \leq r \leq 10^9$.

Phân tích thuật toán. Trước hết chuyển bài toán thành tính số loại dãy có độ dài từ 1 đến n . Nếu bỏ qua phép lật, bài toán có thể giải bằng truy hồi: do cần cấm bộ ba đen–đỏ–trắng liên tiếp, trạng thái phải ghi ít nhất hai màu cuối. Phần chuyển trạng thái có thể tối ưu bằng lũy thừa ma trận.

Theo công thức cho phép lật, ta phải xét thêm các dãy palindrome. Bài này có một điểm đặc biệt: khi n chẵn, hai vị trí giữa của palindrome bằng nhau, trái với điều kiện hai vị trí kề nhau phải khác màu. Vì vậy trường hợp chẵn không có palindrome hợp lệ; còn khi n lẻ, số palindrome tương ứng với số dãy hợp lệ trên nửa quyết định. Do đó toàn bộ bài toán quy về các tổng số dãy không xét phép lật theo độ dài từ 1 đến n .

13.5.2 SPOJ LARGE

Ý chính đề bài. Có t bộ dữ liệu. Trên một bàn tròn có n người ngồi, chỉ phân biệt giới tính. Yêu cầu không được có quá m bạn nữ liên tiếp. Bàn tròn có thể quay; hỏi số cách sắp chỗ ngồi, lấy modulo. Ràng buộc

$$1 \leq n, m, t \leq 1000.$$

Phân tích thuật toán. Đây là mô hình phép quay, nhóm hoán vị có bậc n ; nói trực quan là quay $1, 2, 3, \dots, n$ vị trí.

Trước hết bỏ qua phép quay và tạm thời bỏ qua việc đầu cuối nối nhau. Ta có thể dùng trạng thái theo số bạn nữ liên tiếp ở cuối dãy, hoặc tương đương theo vị trí gần nhất có bạn nam. Chuyển trạng thái có thể tối ưu bằng tổng tiền tố.

Sau đó đưa tính chất vòng vào: khi nối đầu với cuối, vẫn không được có quá m bạn nữ liên tiếp. Từ các giá trị tuyến tính đã tính, dùng bao hàm–loại trừ kết hợp truy hồi để thu được mảng $f[i]$, trong đó $f[i]$ là số cách hợp lệ trên một vòng kích thước i nếu chưa đồng nhất các phép quay. Cuối cùng áp dụng công thức (14), liệt kê các ước k của n và dùng mảng f đã tiền xử lý để cộng đáp án.

13.5.3 Bài tự đặt ring

Ý chính đề bài. Phiên bản rút gọn: đếm số xâu nhị phân dạng vòng độ dài n có chứa một xâu cho trước s độ dài k , lấy modulo. Ràng buộc

$$1 \leq k \leq n \leq 10^9, \quad k \leq 30.$$

Phân tích thuật toán. So với bài trước, khâu cắm hoặc khâu cần xuất hiện không còn có cấu trúc đều đặn, nên việc xử lý phức tạp hơn.

Ta có thể bắt chước cách làm tuyến tính: dùng con trỏ thất bại của KMP để xây dựng chuyển trạng thái, tức trạng thái ghi độ dài tiền tố của s đang khớp. Nhưng khi thêm tính chất vòng, nếu tiếp tục dùng bao hàm–loại trừ trực tiếp thì công thức sẽ rất rối. Vì vậy cần đổi góc nhìn.

Khi cắt vòng thành dãy, một lần xuất hiện của s có thể nằm hoàn toàn trong dãy, hoặc vắt qua điểm cắt: một hậu tố của phần cuối ghép với một tiền tố của phần đầu tạo thành s . Với bài giới tính ở trên, điều kiện vắt qua điểm cắt chỉ phụ thuộc vào tổng số bạn nữ ở hai phía và phần giữa có cùng số cách xếp. Trong bài khâu tùy ý, tính chất đó không còn đúng.

Cách xử lý là tiền xử lý một mảng boolean hai chiều p , ghi nhận với mỗi độ dài hậu tố và tiền tố rằng khi ghép chúng có tạo ra s hay không. Sau đó duyệt mọi kiểu ghép đầu–cuối có thể và chạy truy hồi tương ứng. Vì một hậu tố có thể là tiền tố của một hậu tố khác, cần khử trùng lặp. Trạng thái có thể ghi là: độ dài hậu tố đúng bằng a , độ dài tiền tố đúng bằng b . Dựa vào mảng p , ta thống kê được số trường hợp mà đoạn ghép qua điểm cắt tạo ra s ; phần truy hồi chính vẫn có thể tăng tốc bằng lũy thừa ma trận.

Đến đây, phần đếm trên vòng cơ bản đã xong, nhưng để lấy đáp án theo phép quay ta vẫn phải áp dụng công thức (14). Một chi tiết quan trọng xuất hiện khi độ dài ước $l < k$. Theo ý nghĩa của $f[l]$ trong công thức, ta đang xét một khối độ dài l được lặp lại đủ nhiều lần để tạo thành vòng độ dài n . Khâu s vẫn có thể xuất hiện trong chuỗi lặp này, kể cả khi $l < k$. Đây là bài toán chu kỳ khâu kinh điển; dùng mảng thất bại KMP đã tính, có thể xử lý chi tiết này bằng một lượng kiểm tra rất nhỏ.

13.5.4 Tiếp tục mở rộng

Có hai hướng mở rộng tự nhiên:

1. thêm phép lật;
2. đổi thành bài nhiều khâu: cho nhiều khâu, vòng độ dài n không được chứa bất kỳ khâu nào trong số đó.

Thêm phép lật. Khi phép lật kết hợp với phép quay, các trường hợp sẽ phức tạp hơn. Cách chắc chắn nhất vẫn là viết ra tất cả hoán vị có thể: n phép quay đã xét ở mục 2.1, cộng với các phép lật rồi quay. Tuy nhiên nếu nhìn trực tiếp trên vòng, ta có thể xem “lật rồi quay” như một phép lật quanh một trục; sự khác biệt chính nằm ở tính chẵn lẻ của n .

Nếu n lẻ, mỗi phép lật có một trục đi qua đúng một phần tử và cạnh đối diện. Có n trục như vậy. Một cấu hình bất động phải đối xứng qua phần tử trung tâm, nên $(n + 1)/2$ phần tử quyết định toàn bộ vòng. Tổng đóng góp của các phép lật là

$$n \cdot f\left[\frac{n+1}{2}\right].$$

Cộng với phần phép quay rồi chia cho $|G| = 2n$ sẽ cho đáp án.

Nếu n chẵn, có hai loại trục. Loại thứ nhất đi qua hai khe đối diện giữa các phần tử; có $n/2$ phép lật khác nhau thuộc loại này và mỗi cấu hình được quyết định bởi $n/2$ phần tử. Loại thứ hai đi qua hai phần tử đối diện; cũng có $n/2$ phép lật khác nhau và mỗi cấu hình được quyết định bởi $n/2 + 1$ phần tử. Vì vậy đóng góp từ các phép lật là

$$\frac{n}{2} \left(f\left[\frac{n}{2}\right] + f\left[\frac{n}{2} + 1\right] \right),$$

rồi lại cộng với phần phép quay và chia cho $2n$.

Phần truy hồi gần như không đổi so với bài *ring*: trạng thái vẫn ghi a, b , tức hậu tố và tiền tố. Chỉ cần xử lý riêng phép quay và phép lật. Vì phép lật làm thay đổi cách hai đầu được ghép lại, ta sửa quy tắc ghép, tức sửa mảng boolean p . Chi tiết $l < k$ cũng được xử lý tương tự: cần kiểm tra một khối ngắn sau

khi lặp và lật có chứa xâu dài k hay không. Phần này không chi phối độ phức tạp tổng thể, nên có thể dùng cách kiểm tra trực tiếp.

Nhiều xâu. Từ một xâu sang nhiều xâu, phần truy hồi và phần xử lý tiền tố–hậu tố chỉ cần thay KMP bằng máy tự động Aho–Corasick. Vì có thể xảy ra trường hợp độ dài ước l nhỏ hơn một số xâu mẫu, với mỗi l ta cần xét lại automaton phù hợp với các xâu có thể xuất hiện trong chuỗi lặp độ dài l . Một cách đơn giản là với từng xâu mẫu, dùng cùng ý tưởng chu kỳ như bài một xâu: nếu một khối độ dài l lặp lại có thể phủ kín xâu mẫu, thì rút điều kiện về mẫu tương ứng trên khối độ dài l , rồi cắm mẫu rút gọn đó trong automaton.

13.6 Tổng kết

Trong thi lập trình, ta có thể gặp nhiều bài toán có điều kiện tương tự: nhìn qua tưởng khó bắt đầu, thậm chí khá lạ, nhưng nếu nắm được bản chất của hoán vị và hiểu ý nghĩa của từng đại lượng trong công thức, nhiều bài tưởng khó sẽ trở nên rõ ràng hơn.

13.7 Tài liệu tham khảo

1. Yang Binbin. *Đếm hai lớp hệ lục giác dạng vòng*. Luận văn thạc sĩ, Đại học Hạ Môn.
2. Dong Jinhui. *Chỉ số chu trình của nhóm hoán vị sinh bởi nhóm quay của khối mười hai mặt đều*. Tạp chí Khoa Toán và Khoa học thông tin, Học viện Sư phạm Hoàng Cương.
3. Seymour Lipschutz, Marc Lars Lipson. *Schaum's Outline of Discrete Mathematics*. Bản dịch tiếng Trung của Cao Aiwen và Cao Kun, Nhà xuất bản Đại học Thanh Hoa.
4. Fu Wenjie. *Nguyên lý Polya và ứng dụng*. Luận văn đội tuyển quốc gia năm 2001.
5. Chen Yuxi. *Ứng dụng của phương pháp đếm Polya*. Luận văn đội tuyển quốc gia năm 2008.

14 Ứng dụng của phương pháp chia khối

Wang Ziyu

Trường Trung học số 1 Shengli, Sơn Đông

Tóm tắt

Phương pháp chia khối là một cách giải bài toán khá tổng quát và có tỉ lệ hiệu quả trên công sức cài đặt cao. Trong nhiều trường hợp, ngay cả khi ta chưa khai thác sâu cấu trúc riêng của bài toán, chia khối vẫn cho lời giải đủ nhanh. Bài viết này thông qua một số ví dụ để giới thiệu các ứng dụng khác nhau của chia khối, đồng thời thử rút ra những điểm chung của chúng.

14.1 Dẫn nhập

Trong tin học có một lớp bài yêu cầu thống kê nhanh thông tin trên một đoạn của dãy, hoặc trên cấu trúc tương tự như cây, và có thể kèm theo các thao tác sửa đổi. Cách giải truyền thống thường chia thành hai hướng: dùng cây đoạn hoặc chia khối dãy. Cây đoạn đôi khi bị giới hạn bởi loại thông tin cần gộp; khi đó chia khối trở thành cách khả thi hơn, như bài ở mục 2. Mặt khác, có những bài vẫn giải được bằng cây đoạn hoặc phương pháp khác, nhưng chia khối đơn giản hơn, đôi khi còn cho độ phức tạp tốt hơn, như hai bài đầu tiên ở mục 1.

Chia khối dãy thể hiện tư tưởng phân tán đều các yếu tố chi phối hiệu suất. Tư tưởng đó không chỉ dùng để duy trì dữ liệu. Những phương pháp như tái xây dựng định kỳ, hay kết hợp hai kiểu thuật toán để xử lý bài toán, cũng có cùng tinh thần; các mục sau sẽ thảo luận những dạng này.

14.2 Nhóm ví dụ thứ nhất

14.2.1 Bài 1: UNTITLE

Đề bài. Cho dãy $A[1..n]$, với $n \leq 50000$. Cần hỗ trợ hai loại thao tác:

1. $0 \ x \ y \ k$: tăng mọi phần tử của $A[x..y]$ thêm k ;
2. $1 \ x \ y$: hỏi giá trị lớn nhất của $S[x..y]$, trong đó $S[i] = \sum_{j=1}^i A[j]$.

Lời giải. Đây là một bài cấu trúc dữ liệu điển hình: phải duy trì một dãy, hỗ trợ sửa đoạn và hỏi đoạn. Với loại bài này, nếu thông tin của hai đoạn con có thể gộp nhanh, ta thường dùng cây đoạn; nếu không, chia khối là lựa chọn tự nhiên.

Ta thử duy trì dãy $S[x]$. Xét thao tác $0, x, y, k$, giả sử nó phủ các khối $[l_1, r_1], [l_2, r_2], \dots, [l_j, r_j]$. Trên mỗi khối liên tiếp, lượng tăng của $S[i]$ có dạng

$$\Delta(i) = \text{delt} + (i - l) \cdot K,$$

trong đó delt và K là hằng số đối với khối đó. Vì vậy với mỗi khối ta có thể lưu hai giá trị này làm nhãn lười.

Với một khối, truy vấn cần tính

$$C + \max_i (S[i] + iK).$$

Đặt $x_i = i, y_i = S[i]$. Khi đó $y_i + Kx_i$ có thể hiểu là tung độ giao với trục y của đường thẳng có hệ số góc $-K$ đi qua điểm (x_i, y_i) . Do đó, tìm giá trị lớn nhất tương đương với trượt một đường thẳng hệ số góc $-K$ từ rất cao xuống cho tới khi nó chạm tập điểm. Điểm chạm đầu tiên luôn nằm trên bao lồi trên của tập điểm.

Vì thế, trong mỗi khối ta lưu bao lồi trên của các điểm $(i, S[i])$. Hình trong bản gốc minh họa một tập điểm của một khối: với $k = -0.32$, điểm tối ưu là điểm G , còn các điểm có thể trở thành tối ưu chính là các điểm màu đỏ trên bao lồi trên. Sau khi có bao lồi, do mỗi điểm trên bao lồi cho một hàm tuyến tính theo K , có thể tìm điểm tốt nhất bằng tìm kiếm tam phân trên bao lồi trong $O(\log N)$.

Nếu dùng cây đoạn, việc gộp bao lồi của hai nút không đủ đơn giản để cài đặt hiệu quả. Vì vậy ta chia dãy thành M khối. Mỗi khối duy trì bao lồi của tập điểm tương ứng với S . Một thao tác sửa chỉ đụng trực tiếp tối đa hai khối biên; với các khối này ta sửa thô và xây lại bao lồi, tốn $O(M)$. Các khối nằm hoàn toàn trong đoạn chỉ cần sửa nhãn lười, tổng cộng $O(N/M)$. Do đó chi phí sửa là

$$O\left(M + \frac{N}{M}\right).$$

Truy vấn cũng xử lý thô tối đa hai khối biên, còn các khối trọn vẹn được hỏi trên bao lồi như trên, nên có độ phức tạp

$$O\left(M + \frac{N}{M} \log N\right).$$

Lấy $M = \sqrt{N}$, ta được sửa trong $O(\sqrt{N})$ và hỏi trong $O(\sqrt{N} \log N)$.

Nếu chia cây thành $\sqrt{N}$ khối, loại tư tưởng này có thể mở rộng lên cấu trúc cây, tạo thành cấu trúc thường gọi là cây dạng khối. Bản gốc bỏ qua các chi tiết cài đặt.

14.2.2 Bài 2: RMQ tĩnh

Đề bài. Cho dãy $A[1..N]$. Mỗi truy vấn cho l, r , cần tính

$$\min_{l \leq i \leq r} A[i].$$

Cách $O(N) - O(\sqrt{N})$. Chia dãy thành $\sqrt{N}$ khối; gọi vị trí đầu mỗi khối là điểm then chốt. Tiền xử lý giá trị nhỏ nhất trong từng khối, từ đó trong $O(N)$ thời gian tiền xử lý giá trị nhỏ nhất giữa mọi cặp điểm then chốt, đồng thời tiền xử lý giá trị nhỏ nhất từ mỗi vị trí tới hai điểm then chốt gần nó nhất.

Khi hỏi (l, r) , nếu l và r không nằm trong cùng một khối, các bảng trên cho đáp án trong $O(1)$. Nếu chúng ở cùng khối, quét thô $A[l..r]$, tốn $O(\sqrt{N})$.

Cách $O(N \log \log N) - O(1)$. Với trường hợp l, r cùng khối, không nhất thiết phải quét thô. Ta xây đệ quy cấu trúc vừa nêu cho từng khối, cho tới khi độ dài dãy con bằng 1. Cấu trúc này giống một cây mà bậc ở các tầng khác nhau; độ sâu là $O(\log \log N)$, và chi phí tiền xử lý mỗi tầng là $O(N)$, nên tổng thời gian tiền xử lý là $O(N \log \log N)$.

Khi hỏi, nếu hai đầu mút không ở cùng khối thì trả lời ngay; nếu cùng khối thì đi xuống cấu trúc con. Cách trực tiếp mất $O(\log \log N)$. Có thể nhị phân tầng nông nhất mà hai đầu mút không còn ở cùng khối để giảm xuống $O(\log \log \log N)$.

Để đạt $O(1)$, xét trường hợp tồn tại số nguyên z sao cho

$$N = 2^{2^z}.$$

Khi đó độ dài khối ở mỗi tầng là cố định. Với truy vấn độ dài L , gọi i là tầng đầu tiên mà độ dài khối không vượt quá L . Tầng nông nhất tách hai đầu mút chỉ có thể là i hoặc $i - 1$, nên kiểm tra hai tầng là đủ. Tiền xử lý ánh xạ từ mỗi độ dài truy vấn tới i , ta có thời gian hỏi $O(1)$. Nếu N không đúng dạng trên, tăng độ dài khối lên giá trị nhỏ nhất giữ được tính chất này; thời gian tiền xử lý vẫn là $O(N \log \log N)$.

Cách $O(N \log^* N) - O(1)$. Một biến thể khác chia dãy thành các khối dài $\log N$. Ta tiền xử lý giá trị nhỏ nhất trong mỗi khối, dùng các giá trị đó xây sparse table trong $O(N)$, nên truy vấn bằng qua khối trả lời được trong $O(1)$. Truy vấn nằm hoàn toàn trong một khối được xử lý bằng cách xây đệ quy cùng cấu trúc cho mỗi khối, tới khi độ dài bằng 1. Vì mỗi lần đệ quy giảm kích thước xuống mức logarit, độ sâu là $\log^* N$, còn tổng chi phí mỗi tầng là $O(N)$, nên tiền xử lý là $O(N \log^* N)$. Kỹ thuật trả lời truy vấn tương tự biến thể trước cũng cho $O(1)$.

Trong thi đấu, cách $O(N \log \log N) - O(\log \log N)$ thường có tỉ lệ hiệu quả trên công sức tốt nhất. Nếu tận dụng phép toán bit $O(1)$ của máy tính, còn có thể tối ưu nó thành $O(1)$; bản gốc không trình bày chi tiết. RMQ còn có những cách giải nhanh khác.

Hai cách đầu cũng mở rộng được cho một lớp bài tổng quát hơn: không có sửa đổi, truy vấn phép gộp thông tin trên đoạn, trong đó phép gộp có tính cộng theo đoạn nhưng không có phép trừ đoạn. Một ví dụ là bài tổng đoạn con liên tiếp lớn nhất trên đoạn.

14.3 Nhóm ví dụ thứ hai

14.3.1 Bài 3: *Fairy*

Đề bài. Cho một đồ thị. Hỏi có bao nhiêu cạnh sao cho sau khi xóa cạnh đó, đồ thị còn lại là đồ thị hai phía. Số đỉnh $N \leq 100000$, số cạnh $M \leq 100000$.

Lời giải. Bài này có lời giải tuyến tính, nhưng tư duy và cài đặt tương đối phức tạp. Ta xét một cách làm đơn giản hơn bằng chia khối.

Để kiểm tra một đồ thị có hai phía hay không, có thể dùng DSU kèm chẵn lẻ: duy trì parity của khoảng cách từ mỗi đỉnh tới đại diện thành phần liên thông. Khi thêm cạnh (u, v) , nếu u, v đã ở cùng thành phần và parity của chúng so với đại diện giống nhau, cạnh đó tạo ra chu trình lẻ.

Chia danh sách cạnh thành các khối. Với mỗi khối, trước hết thêm tất cả cạnh nằm ngoài khối vào DSU để thu được cấu trúc nền. Sau đó, với từng cạnh trong khối, thêm tất cả cạnh còn lại của khối vào DSU, kiểm tra có mâu thuẫn hay không, rồi hoàn tác các cạnh vừa thêm bằng một ngăn xếp.

Cần chú ý rằng độ phức tạp $O(\alpha(N))$ của DSU đến từ phân tích khấu hao, nên không thể máy móc viết tổng thời gian là $O(N\sqrt{N}\alpha(N))$; nếu không xử lý thêm, bound sẽ thành $O(N\sqrt{N}\log N)$. Tuy nhiên, sau khi đã thêm tất cả cạnh ngoài khối, ta có thể co các thành phần lại. Khi đó hai giai đoạn trước và sau co không còn phụ thuộc vào nhau, và phân tích khấu hao lại cho độ phức tạp $O(N\sqrt{N}\alpha(N))$.

14.4 Nhóm ví dụ thứ ba

14.4.1 Candy Park

Đề bài. Cho một cây có N đỉnh; mỗi đỉnh có một trọng số, và có C loại trọng số khác nhau. Có Q thao tác, mỗi thao tác hoặc sửa trọng số của một đỉnh, hoặc hỏi trọng số của đường đi nối hai đỉnh s_i, t_i .

Trọng số của một đường đi được định nghĩa như sau. Cho các mảng V và W . Nếu trọng số loại i xuất hiện c_i lần trên đường đi, thì giá trị đường đi là

$$\sum_i V[i] \sum_{j=1}^{c_i} W[j].$$

Trong bản gốc, dữ liệu được chia thành ba loại: cây là một đường thẳng và không có sửa đổi; cây tổng quát nhưng không có sửa đổi; cây tổng quát và có sửa đổi.

Bài này có cách chuẩn là xử lý truy vấn ngoại tuyến và chuyển trạng thái giữa các truy vấn, như lời giải bài Winter Camp trong tập bài đội tuyển. Ở đây tác giả trình bày một loạt thuật toán trực tuyến khác.

14.4.2 Trường hợp cây là đường thẳng, không sửa đổi

Với cây là một đường thẳng và không có sửa đổi, chia dãy thành $\sqrt{N}$ khối. Tiền xử lý đáp án từ điểm đầu của mỗi khối, gọi là điểm then chốt, tới mọi vị trí; chi phí là $O(N\sqrt{N})$.

Xét truy vấn $[l, r]$. Gọi L là điểm đầu khối gần l nhất ở bên phải. Đáp án cho $[L, r]$ đã có sẵn. Ta chỉ cần lần lượt đưa các điểm trong $[l, L - 1]$ vào thống kê và cập nhật đáp án.

Để cập nhật khi thêm một trọng số w , cần biết w đã xuất hiện bao nhiêu lần trong đoạn hiện tại. Dùng tổng tiền tố, việc này chuyển thành hiệu giữa số lần w xuất hiện trong $[1, r]$ và trong $[1, l]$. Ta duy trì $okr[i][w]$, là số lần trọng số đã rời rạc hóa w xuất hiện trong i phần tử đầu.

Lưu trực tiếp mảng hai chiều này tốn $O(N^2)$, nhưng $okr[i]$ và $okr[i - 1]$ chỉ khác nhau ở một vị trí. Vì vậy dùng danh sách khối bền vững: chia mỗi mảng $okr[i]$ thành $\sqrt{N}$ khối và lưu con trỏ tới từng khối. Khi xây $okr[i]$, hầu hết con trỏ được chép từ $okr[i - 1]$; chỉ khối chứa trọng số mới cần tạo lại. Như vậy mỗi phiên bản chỉ dùng thêm $O(\sqrt{N})$ bộ nhớ, và truy cập một phần tử vẫn coi như $O(1)$.

Phần mã giả trong bản gốc gồm hai thủ tục. Thủ tục khởi tạo chọn kích thước khối, tiền xử lý đáp án từ từng điểm then chốt, rồi xây các phiên bản okr bền vững. Thủ tục truy vấn tìm điểm then chốt L , lấy đáp án đã tiền xử lý cho $[L, r]$, sau đó thêm thô các điểm từ l tới $L - 1$ bằng cách tra số lần xuất hiện qua okr . Tổng độ phức tạp tiền xử lý là $O(N\sqrt{N})$, còn mỗi truy vấn tốn $O(\sqrt{N})$.

14.4.3 Trường hợp cây tổng quát, không sửa đổi

Với cây không còn là đường thẳng, cách làm tương tự. Chia cây thành nhiều nhất $\sqrt{N}$ khối, sao cho chiều cao mỗi khối không quá $\sqrt{N}$. Như trường hợp trên, tiền xử lý đáp án từ mỗi điểm then chốt tới mọi đỉnh. Đồng thời duy trì $okr[i][w]$, là số lần trọng số w xuất hiện trên đường từ gốc tới đỉnh i . Khi cần thêm một đỉnh vào đường đi, số lần xuất hiện có thể tính bằng công thức kiểu đường đi trên cây dựa vào các mảng từ gốc. Độ phức tạp vẫn là

$$O((N + Q)\sqrt{N}).$$

14.4.4 Trường hợp có sửa đổi

Khi có thao tác sửa, ta chia chính dãy thao tác thành các khối để xử lý. Giả sử mỗi khối thao tác có kích thước xấp xỉ $Q^{2/3}$. Với các truy vấn trong cùng một khối, trước hết áp dụng tất cả thao tác sửa xuất hiện trước khối vào cây, rồi dùng cách của mục trước để tính đáp án trong trạng thái chỉ xét các sửa đổi trước khối. Sau đó, với từng truy vấn, xét thêm các sửa đổi trong cùng khối nhưng xuất hiện trước truy vấn đó, và tính phần ảnh hưởng của chúng lên đáp án.

Ở bước thứ nhất, cần làm nhẹ cấu trúc okr . Nếu vẫn dùng danh sách khối bền vững hai tầng như trên, chi phí sẽ quá lớn. Ta chia mỗi mảng okr thành $N^{1/3}$ nhóm ở tầng một; mỗi nhóm có $N^{2/3}$ phần tử và lại được duy trì bằng danh sách khối bền vững ở tầng hai. Hình trong bản gốc minh họa một mảng khối ba tầng với 26 phần tử. Mỗi lần chỉ cần sửa một mảng khối tầng hai và tạo mới một mảng đáy kích thước $N^{1/3}$, nên chi phí xây một phiên bản okr giảm xuống $O(N^{1/3})$; truy cập một phần tử vẫn xem như $O(1)$. Vì thế, phần xử lý một truy vấn ở bước này tốn $O(N^{2/3})$.

Ở bước thứ hai, với mỗi thao tác sửa nằm trước truy vấn hiện tại trong cùng khối, kiểm tra đỉnh bị sửa có nằm trên đường đi đang hỏi hay không. Nếu có, cập nhật phần chênh lệch của đáp án. Dùng thuật toán LCA $O(1)$ cho truy vấn tổ tiên chung, bước này tốn $O(Q^{2/3})$ cho mỗi truy vấn. Tổng độ phức tạp là

$$O(Q^{1/3}(N^{4/3} + Q^{4/3})).$$

So với cách chuẩn, ưu điểm của hướng này là không cần xử lý ngoại tuyến. Bằng tư tưởng tương tự, có thể giải trực tuyến nhiều bài mà phiên bản không sửa đổi đã giải được nhưng phiên bản có sửa đổi chưa có cách trực tiếp, hoặc chỉ có một loại thao tác khó xử lý. Ví dụ được nêu trong bản gốc là bài hỏi thứ hạng trên đoạn với chèn và sửa.

14.4.5 Bài tương tự: *HARD Kth*

Đề bài. Cho dãy $A[1..n]$, hỗ trợ ba thao tác:

1. $M\ x\ y$: đổi $A[x]$ thành y ;
2. $I\ x\ y$: chèn y vào sau $A[x]$;
3. $Q\ l\ r\ k$: hỏi phần tử nhỏ thứ k trong $A[l..r]$.

Số thao tác $q \leq 1.75 \cdot 10^5$, độ dài dãy ban đầu $n < 3.5 \cdot 10^4$. Yêu cầu thuật toán trực tuyến.

Lời giải. Nếu chỉ có sửa và hỏi, có thể dùng BIT của cây đoạn động trong $O((n + q) \log^2 n)$. Mỗi nút BIT duy trì một cây đoạn động chứa các giá trị trong đoạn mà nút đó phụ trách. Khi sửa, cập nhật trên $O(\log n)$ cây đoạn. Khi hỏi, nhị phân đáp án và đếm số phần tử nhỏ hơn giá trị đang thử; vì các cây đoạn có cấu trúc giống nhau, mỗi lần hỏi tốn $O(\log^2 n)$.

Khi có thêm chèn, chia các thao tác thành m khối. Trước khi xử lý một khối, áp dụng tất cả thao tác chèn của các khối trước vào dãy rồi xây cấu trúc BIT của cây đoạn cho dãy hiện tại. Với mỗi truy vấn trong khối, trước hết quy đổi khoảng hỏi thực sự sau khi tính các chèn trong chính khối; trong mỗi bước nhị phân đáp án, quét các thao tác chèn của khối hiện tại để hiệu chỉnh số lượng. Độ phức tạp có dạng

$$O\left(m(n \log n) + \frac{q^2}{m} \log n\right).$$

Chọn $m = \sqrt{q}$, ta được

$$O((q + n)\sqrt{q} \log n).$$

14.5 Nhóm ví dụ thứ tư

14.5.1 Bài 5

Đề bài. Cho N tập hợp xâu; ban đầu mỗi tập chỉ chứa một xâu. Cần hỗ trợ:

1. MERGE $A B$: gộp hai tập A, B thành $A \cup B$;
2. QUERY $S s$: hỏi các xâu trong tập S xuất hiện bao nhiêu lần trong xâu s .

Tổng độ dài các xâu là $Slen \leq 10^5$.

Lời giải. Dùng tư tưởng hàng đợi nhị thức để duy trì automaton Aho–Corasick có thể cho lời giải $O(N \log N)$, nhưng tư duy và cài đặt khá nặng. Ở đây xét một cách làm đơn giản hơn, có độ phức tạp

$$O(N\sqrt{Slen} + Qlen\sqrt{Slen}),$$

trong đó $Qlen$ là tổng độ dài các xâu truy vấn.

Đặt $L = \lfloor \sqrt{Slen} \rfloor$. Chia các xâu mẫu thành hai loại: dài hơn L và không dài hơn L . Số xâu dài hơn L không vượt quá $Slen/L$. Với các xâu ngắn trong từng tập, tiền xử lý hash và lưu theo độ dài trong bảng hash; với các xâu dài, tính trước mảng *next* của KMP.

Khi gộp hai tập, chỉ cần gộp các bảng hash theo kiểu nhỏ vào lớn. Khi hỏi, với các xâu dài ta chạy KMP thô trên xâu truy vấn, nhờ mảng *next* đã có sẵn; chi phí là $O(dlen \cdot Slen/L)$, trong đó $dlen$ là độ dài xâu truy vấn hiện tại. Với các xâu ngắn, duyệt vị trí bắt đầu trong xâu truy vấn và duyệt độ dài mẫu, rồi tra hash tương ứng; chi phí là $O(dlen \cdot L)$. Chọn $L = \sqrt{Slen}$, bài toán được giải.

14.5.2 Bài 6: KAR

Đề bài. Nếu hai đoạn có giao, ta nói chúng khớp với nhau. Cho N đoạn $A[1..N]$. Có M truy vấn, mỗi truy vấn cho một đoạn $[l, r]$. Cần xác định một dãy con liên tiếp dài nhất của A sao cho mọi đoạn trong dãy con đó đều khớp với $[l, r]$. Ràng buộc:

$$N \leq 50000, \quad M \leq 200000.$$

Lời giải. Với một tập các đoạn, tất cả đoạn trong tập đều khớp với truy vấn $[l, r]$ khi và chỉ khi

$$\max L_i \leq r \quad \text{và} \quad \min R_i \geq l.$$

Vì vậy, để kiểm tra một tập đoạn có khớp hoàn toàn với truy vấn hay không, chỉ cần biết cặp

$$(first, second) = (\max L_i, \min R_i).$$

Nếu đáp án của truy vấn không vượt quá K , ta có thể tiền xử lý các tập $S[j]$: với mỗi độ dài j , $S[j]$ chứa các cặp (*first*, *second*) của mọi dãy con liên tiếp độ dài j . Để kiểm tra đáp án của truy vấn $[l, r]$ có ít nhất j hay không, trong $S[j]$ chỉ cần tìm phần tử có *first* $\leq r$ và *second* lớn nhất, rồi kiểm tra nó có không nhỏ hơn l hay không.

Một phần tử là vô dụng nếu tồn tại phần tử khác có *first* không lớn hơn và *second* không nhỏ hơn. Loại bỏ các phần tử vô dụng và sắp xếp theo *first*, mỗi lần kiểm tra có thể tìm bằng nhị phân. Sau đó nhị phân đáp án. Phần này tốn

$$O(NK + Q \log^2 N).$$

Nếu đáp án lớn hơn K , sau khi chia dãy A thành các khối dài không quá K , đáp án chắc chắn bằng qua ít nhất hai khối. Với mỗi khối, lưu các cặp (*first*, *second*) của tiền tố độ dài i và hậu tố độ dài i . Khi xử lý một truy vấn, dùng nhị phân để biết từ đầu mỗi khối có thể kéo dài bao nhiêu đoạn khớp, và từ cuối mỗi khối theo chiều ngược lại có thể kéo dài bao nhiêu đoạn. Dựa trên thông tin này, quét toàn bộ các khối để lấy đáp án. Phần này tốn $O(Q(N/K) \log N)$.

Tổng độ phức tạp là

$$O\left(NK + Q\left(\log^2 N + \frac{N}{K} \log N\right)\right).$$

Chọn $K = \sqrt{N \log N}$, ta được

$$O\left((N + Q)\sqrt{N \log N} + Q \log^2 N\right).$$

14.6 Tổng kết

Ưu điểm của chia khối là không đòi hỏi phân tích quá sâu các tính chất riêng của bài toán, nên có tính tổng quát cao. Nhược điểm là cài đặt đôi khi rườm rà, và độ phức tạp thời gian không phải lúc nào cũng đủ tốt. Trong thực tế thi đấu, đây vẫn là một phương pháp có tỉ lệ hiệu quả trên công sức rất cao.

14.7 Lời cảm ơn

Xin cảm ơn Luo Yuping, Huang Jiatai và Xing Zhiming đã giúp đỡ trong quá trình viết bài. Xin cảm ơn nhiều bạn đã cung cấp các ví dụ.

14.8 Tài liệu tham khảo

1. Chen Lijie. *Nghiên cứu cấu trúc dữ liệu bền vững*. Bài tập đội tuyển quốc gia năm 2012.
2. EPR. *Candy Park: lời giải*. Bài tập đội tuyển quốc gia năm 2013.
3. Anders Kaseorg. *Optimal worst-case fully persistent arrays*. TFP 2009.
4. *Ternary Search*. Wikipedia.